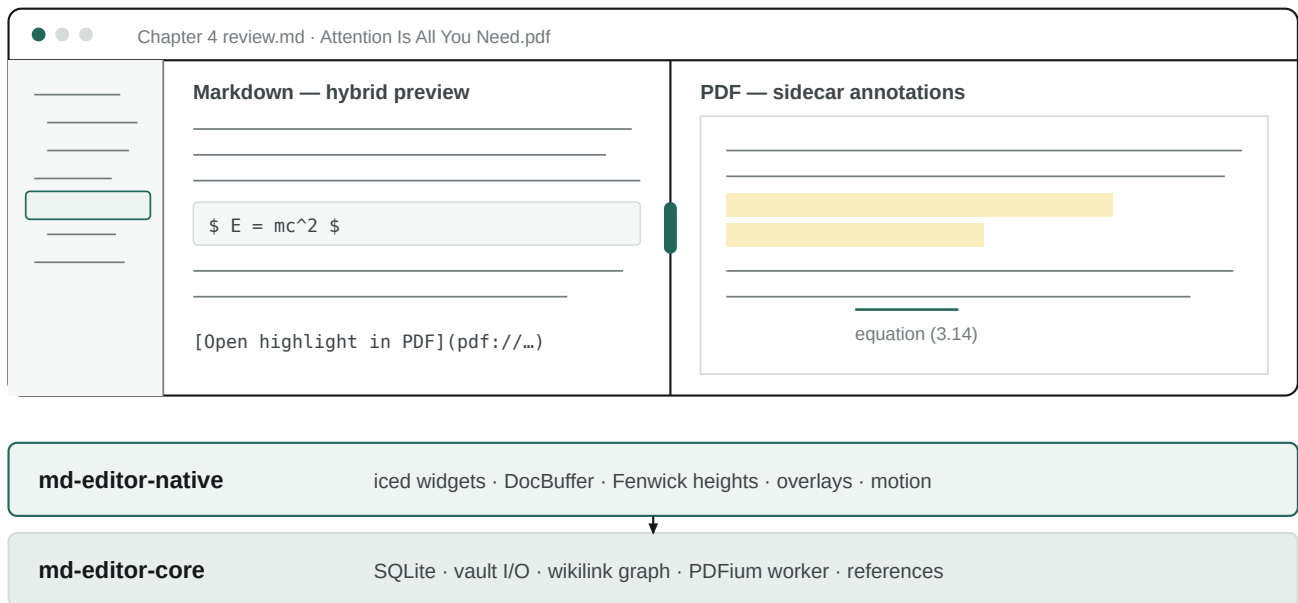




MD Editor

The Field Handbook — a guided tour of the codebase for the developer who just cloned it.

Audience — new contributors; junior developers with some Rust and no prior knowledge of this repository.

Source — generated from the project wiki (17 pages) and verified against the source tree at Suranjan77/md-editor.

Revised — September 2026: the editor chapters rewritten for the split renderer, incremental highlighting, typographic rhythm and caret motion; counts, tokens and constants brought up to date.

Covers — architecture, the core engine, the markdown editor, the PDF subsystem, design systems, testing, and the rules that keep it all standing up.

Licence — the project is GPL-3.0. This handbook ships with it.

How to use this handbook

This is not a reference manual you look things up in. It is a book you read in order, with the repository open beside it, running the commands as they appear.

MD Editor is roughly 26,000 lines of Rust across two crates. That is small enough to hold in your head eventually and large enough to be intimidating on day one. The path through it that this book takes is deliberate: *outside in*. You will see what the program does for a person, then the shape of the two crates, then the engine that owns the disk, then the editor that owns the screen, then the PDF machinery, and only at the end the rules that a contributor is expected to keep. By that point every rule will already have a story attached to it — which is the only way rules survive.

Reading paths

IF YOU ARE...	READ
New to the project, and want to ship a first change this week	Part I in full, then Chapter 30 (tests), Chapter 31 (invariants) and Appendix D (starter tasks). Come back for the deep parts when a task drags you into them.
Fixing something in the editor	Chapters 11–17. The layout and draw contract in Chapters 15–17 is where most performance regressions are born.
Fixing something in the PDF pane	Chapters 18–25, and then Chapter 32's rule 7. Read Chapter 23 twice; it is the bug this subsystem keeps trying to have.
Reviewing someone else's change	Chapter 31 (the five durability invariants) and Chapter 32 (the rules and the bug behind each one).
Preparing a release	Chapter 29 (build & PDFium), Chapter 33 (shipping), and the checklists inside them.

Conventions

Five kinds of aside appear throughout. They are worth learning to recognise, because each one is making a different kind of promise to you.

WHY IT IS LIKE THIS

The reasoning behind a decision that looks arbitrary. If you ever want to change the code these describe, you have to defeat the argument in the box first.

TRAP

A mistake this codebase has actually made, or one it is shaped to invite. These are the paragraphs that save you an afternoon.

TRY IT

Something to run or change right now, in the checkout in front of you. Reading about a Fenwick tree is not the same as watching one stop your editor from stalling.

CHECK YOURSELF

A question with an answer directly underneath. If the answer surprises you, re-read the section — the material after it assumes you got this one.

NOTE

Context, history, or a pointer somewhere else in the book.

Source references look like `core/src/vault.rs::write_file` — a path relative to the repository root, and where useful the function inside it. Every factual claim in this book is meant to be checkable against the file named beside it. Where a claim and the code disagree, **the code is right and this book has a bug**; the same rule the wiki holds itself to.

What you need in front of you

- A clone of the repository, on the default branch.
- A stable Rust toolchain, 1.85 or newer — the workspace is on the 2024 edition.
- A C compiler, because `rusqlite` builds SQLite from source.
- Network access for the first build only, so the build script can fetch PDFium.
- A desktop session, if you want to *run* the app. The core crate's tests need no display at all.

Chapter 3 walks the whole setup. If a command in this book fails, that chapter is where the fix lives.

Contents

PART I · ORIENTATION

1	What you are looking at	7
2	The map: two crates, three kinds of thread	10
3	Your first hour	13
4	The language of this codebase	16

PART II · THE ENGINE — MD-EDITOR-CORE

5	AppState and the lock discipline	19
6	The database	21
7	Where the database lives	24
8	The atomic save	26
9	Wikilinks and the backlink graph	29
10	Full-text search	31

PART III · THE MARKDOWN EDITOR

11	The shell: one loop, one router	34
12	Subscriptions and the zero-idle-CPU rule	36
13	DocBuffer, transactions, and word-sized undo	38
14	Hybrid preview and incremental highlighting	42
15	The Fenwick height tree	45
16	Measure once: the line cache and the layout key	48
17	The widget: rhythm, rows, and the draw pass	51

PART IV · THE PDF SUBSYSTEM

18	One thread, two queues	59
19	Two coordinate systems	61
20	Finding a table of contents that isn't there	62
21	Resolving “see equation (3.14)”	63
22	Sidecar annotations and document identity	66
23	Generations, pending sets, and the stranded page	69
24	Stable page slots	71
25	From highlight to note and back	74

PART V · THE SURFACE

26	Tokens and motion	77
27	The command palette and the fuzzy scorer	81

28	The study tracker	83
----	-------------------------	----

PART VI · WORKING HERE

29	The build system and PDFium	86
30	The test suite, and what each part protects	88
31	The five durability invariants	92
32	The rules, and the bug behind each one	94
33	Shipping a build	97

APPENDICES

A	Keyboard shortcuts	101
B	Every named constant	104
C	The message vocabulary	108
D	Starter tasks, graded	110
E	Where to go next	113

A NOTE ON FIGURES

Every diagram in this book was drawn for it. Where the wiki carried a generic box-and-arrow graph, the figure here tries instead to show the *mechanism* — the thing that actually moves, the state that actually changes, the failure the design is holding off. If a figure does not tell you something the surrounding prose left out, it has not earned its ink.

Orientation

Before any code: what this program is for, what shape it has, and how to get it running on your machine. By the end of this part you should be able to open any file in the repository and know roughly why it exists.

1. What you are looking at
2. The map: two crates, three kinds of thread
3. Your first hour
4. The language of this codebase

What you are looking at

The product, and the four decisions everything else follows from

MD Editor is a desktop workspace for people who read papers and write notes about them: a markdown editor on one side, a PDF reader on the other, a search index across both, and a study timer underneath. It is written in Rust, it runs entirely on your machine, and it stores your work as ordinary files in an ordinary folder.

That description is short enough to fit on a landing page, and it hides everything interesting. What makes this codebase worth reading is not the feature list — plenty of programs edit markdown — but the four commitments the author made early and then refused to bend, because every non-obvious piece of code in the repository is downstream of one of them.

1.1 The four pillars

Local-first, and no proprietary format

A **vault** is a directory. You open one with a native folder picker and MD Editor indexes what it finds: markdown (`.md`, `.markdown`), PDFs, and images (`.jpeg`, `.jpg`, `.png`, `.svg`, `.webp`, `.avif` — the list in `core/src/vault.rs`, which is not quite the list the README prints; see Appendix B). Nothing is imported, converted, or moved. Your notes stay readable by `cat`, editable by Vim, and syncable by Git, rsync, or Syncthing.

There is no account, no licence key, and no telemetry. The *only* network access anywhere in the project is the build script fetching PDFium, and that happens once, at compile time (Chapter 29).

Zero-configuration portability

Everything that is not your content — settings, window size, per-document scroll and cursor positions, PDF highlights, cached PDF text, resolved cross-references, study history — lives in a single SQLite file called `md_editor_settings.sqlite`. By default that file sits *beside the executable*, so the whole application is one folder you can drop on a USB stick. Chapter 7 covers what happens when that directory is read-only.

Durability and continuity

Typing must survive a crash, a power cut, and a window closed in a hurry. The app therefore treats disk as the source of truth and commits aggressively: an autosave fires 400 ms after you stop typing, every write is atomic, and navigating away from a note flushes it first — refusing to navigate if the write fails. Chapters 8 and 31 are entirely about this.

Non-destructive sidecar annotations

A paper you are reading is reference material, and the app treats it as immutable. Highlight a passage and MD Editor writes a row in SQLite — never a byte into your `.pdf`. This is the decision with the longest tail of consequences, and Figure 1.1 is the shape of it.

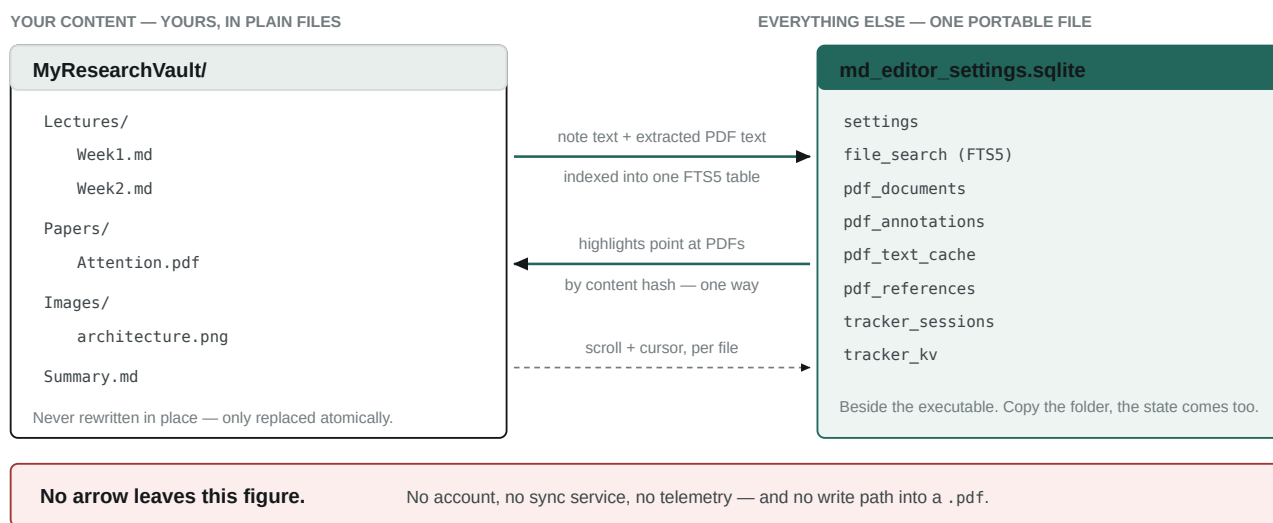


Figure 1.1 — The sidecar split. Content on the left is plain and portable; state on the right is one file. The arrow that *does not exist* is the important one: annotations reference PDFs, but nothing ever writes back into them. That single missing arrow is why annotations must be keyed by a content hash rather than a filename, why a “drifted highlight” report has to exist at all (both Chapter 22), and why your highlights do not travel with a PDF you email to someone.

WHY IT IS LIKE THIS

Every one of these pillars costs something. Sidecar annotations mean your highlights do not open in Acrobat. Portability means the database is in a slightly unusual place. Aggressive autosave means the code has to be careful about writes in a way a “save when the user asks” editor never does.

They were chosen anyway because the failure they prevent is worse than the cost they impose: a research vault you cannot read without the tool that made it is a vault you will lose.

1.2 What it actually does, briefly

You will understand the code faster if you have used the program for ten minutes. The feature surface, in one pass:

AREA	WHAT A USER GETS	WHERE IT LIVES
Markdown editing	Hybrid live preview — formatted text that reveals its raw syntax on the line the cursor is on, with no mode switch. Inline and display LaTeX, images, tables that scroll on their own, task lists you click.	<code>native/src/editor/</code>
Smart typing	Auto-paired brackets and quotes that know what a contraction is, list continuation with ordered-list increment, and an undo that takes back a word rather than a character.	<code>native/src/editor/buffer.rs</code>

AREA	WHAT A USER GETS	WHERE IT LIVES
Linking	<code>[[Wikilinks]]</code> that resolve across subfolders by filename, and a backlinks panel that shows both notes and PDF highlights pointing at the open document.	<code>core/src/file_index.rs</code>
PDF reading	Continuous scroll, fit-to-width, a recovered table of contents when the file has no bookmarks, recognised cross-references (“see equation (3.14)”) you can right-click to preview.	<code>core/src/pdf.rs</code> , <code>references.rs</code>
Study workflow	Highlights in five cycling colours, quick notes, and “Link to Note”, which writes a markdown section quoting the passage with a <code>pdf://</code> deep link back to the exact page.	<code>native/src/pdf_notes.rs</code>
Search	Three of them: in-note with regex and replace, PDF search with a loose-whitespace mode for phrases that wrap across lines, and a global FTS5 search covering notes and extracted PDF text together.	<code>core/src/vault.rs</code> , <code>native/src/search.rs</code>
Tracker	A timer, a session log, and a configurable curriculum of phases, projects, gates and reading lists.	<code>core/src/tracker.rs</code>

TRY IT

Before reading Chapter 2, build and run the app on a scratch folder with one markdown file and one PDF in it (Chapter 3 has the commands). Type a sentence, press `Ctrl + Z` once, and watch a whole word-run disappear rather than one character. That behaviour is Chapter 13, and it is much easier to read the code once you have felt the thing it produces.

CHECK YOURSELF

You copy your vault folder to a new laptop but forget the application folder. What survives?

Answer: every note, PDF and image — and none of the highlights, study history, scroll positions or cached text, because those live in the settings database beside the executable, not in the vault. Whether that split is right is a genuine design question; Chapter 22 explains why the annotations were nevertheless not put inside the PDFs.

The map: two crates, three kinds of thread

The one structural picture the rest of the book assumes

MD Editor is a Cargo workspace with exactly two members. Almost every question of the form “where does this belong?” is answered by which of the two it is, and by which thread it is allowed to run on. Learn this chapter properly and the other thirty-one become local details.

2.1 The two crates

`md-editor-core` — the headless engine

Owns the disk and the database. Vault traversal, atomic writes, the SQLite schema, the full-text index, the wikilink graph, the PDFium worker thread, the cross-reference resolver, and the study tracker's persistence. It has no idea a window exists. All 56 of its tests run in a container with no display server.

`md-editor-native` — the desktop interface

Owns the screen. The Iced application loop, the custom markdown editor widget, the PDF page widget and its overlays, design tokens, motion, the command palette. It reaches the engine only through thread-safe methods on a shared `Arc<AppState>`, or through async tasks.

The dependency arrow points one way: `native` depends on `core`, and `core` must never learn about `native`. Three rules make that concrete, and they are the first three things a reviewer will check:

1. **No circular dependency.** `md-editor-core` never depends on `md-editor-native`.
2. **No UI in core.** The core crate must not take a dependency on a GUI or text-shaping toolkit — not `iced`, not `winit`, not `ratex-render`.
3. **Pure FFI containment.** `pdfium-render` is reachable only through `core/src/pdf.rs`, and exactly one `Pdfium` binding may exist per process.

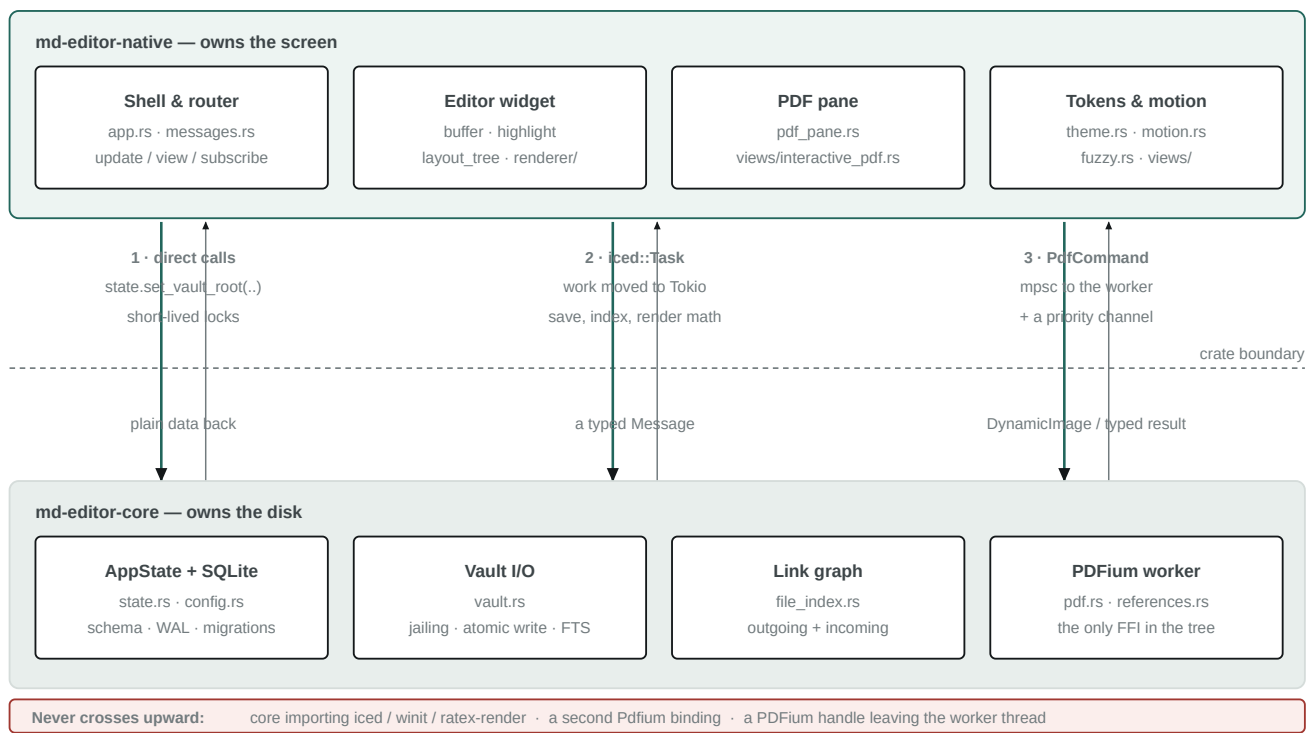


Figure 2.1 — What actually crosses the crate boundary. Three kinds of traffic, and nothing else. Sage arrows go down (a request), grey arrows come back (a result). Notice that the return path is never a PDFium handle or an iced type — it is decoded image data, plain structs, or a `Message`. That is what makes the engine testable without a window.

2.2 Three kinds of thread

Threading is where a desktop app is won or lost. MD Editor has three execution contexts and a strict rule about each.

1. **The UI main thread** — the Iced event loop. Window events, typing, clicks, layout, drawing. *It never blocks on disk or on PDF work.*
2. **The Tokio async runtime** — background work dispatched as `iced::Task`: autosave writes, vault indexing, FTS queries, PDF page renders, and syntax highlighting for large documents. Results come back as ordinary `Message` values.
3. **One dedicated PDFium worker thread** — spawned by `PdfRenderer::new()`. Because `pdfium-render` keeps its bindings in a process-global cell, *all* PDF work is serialized onto this single thread. It is fed by two `std::sync::mpsc` channels: a **priority** channel for the page the reader is actually looking at, and a **standard** channel for everything else.

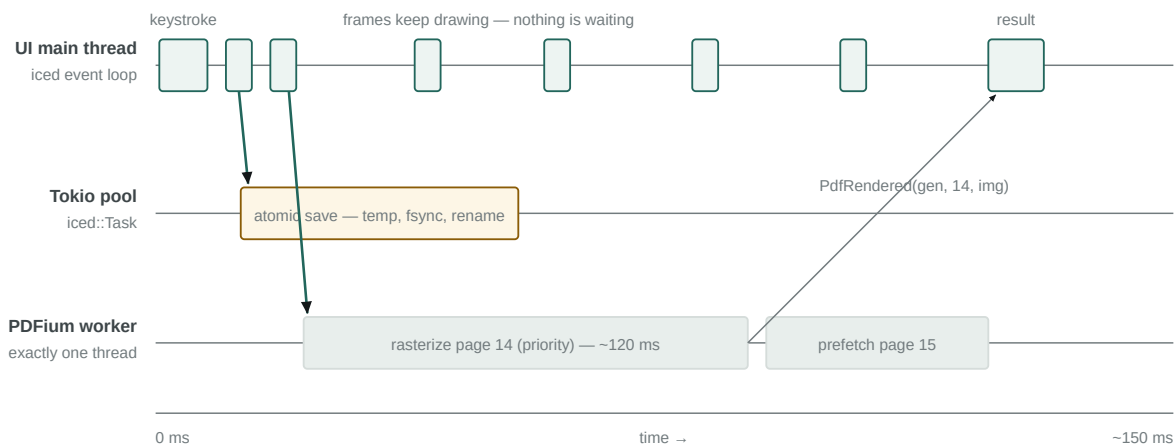


Figure 2.2 — Nobody waits. One keystroke fans out into two background jobs, and the UI lane keeps producing frames the whole time. The worker lane is the interesting one: it is a *single* lane by necessity, which is why Chapter 18 has to talk about queue priority rather than parallelism.

TRAP

The PDFium worker **cannot interrupt an operation already in flight**. Priority only decides what runs *next*. If you add a worker command that takes half a second — say, extracting text for a whole 500-page document in one go — every page render queues behind it and the reader watches a blank page. Chapter 21 shows the chunking pattern that exists precisely to avoid this.

CHECK YOURSELF

You want to add “export the current note to HTML”. Which crate, and which thread?

Answer: the conversion itself is pure text-to-text with no GUI dependency, so it belongs in `md-editor-core` where it can be unit-tested headlessly. The file write goes through the existing atomic-write path in `core/src/vault.rs`. The native side contributes a menu entry and an `iced::Task` so the work runs on Tokio rather than on the UI thread — even though it is fast, because “it is fast” is how every stall starts.

Your first hour

Build it, run it, and learn where the code actually is

Everything in this chapter is meant to be typed, not read. If a command fails, the fix is almost always in the prerequisites list — this is a project with a C compiler and a downloaded native library in its build path, and both can bite on a fresh machine.

3.1 Prerequisites

- **Rust** — stable 1.85 or newer, because the workspace is on the **2024 edition**.
- **A C compiler** — `gcc` or `clang`. `rusqlite` compiles a bundled SQLite from source, so there is no system SQLite to install and no version skew to debug.
- **Network access, once** — `core/build_pdfium.rs` downloads a prebuilt PDFium for your platform on the first build and caches it under `target/pdfium/`.
- **A desktop session** — only to *run* the app. Tests do not need one.

DEBIAN / UBUNTU

```
sudo apt update
sudo apt install -y build-essential pkg-config libxkbcommon-dev libfontconfig1-dev
sudo apt install -y libwayland-dev libx11-dev # windowing
```

ARCH

```
sudo pacman -S --needed base-devel pkgconf libxkbcommon fontconfig
```

3.2 The commands you will use every day

```
cargo fmt --check           # formatting — must be clean
cargo check --workspace     # fast typecheck
cargo test --workspace      # 215 tests: 56 core + 159 native
cargo clippy --workspace    # see the warning below
cargo run                   # launch a debug build
cargo build --release        # optimized binary
```

TRAP

`cargo fmt --check` and `cargo test --workspace` pass cleanly on main. **Clippy does not.** The tree carries 10 warnings (2 in core, 8 in native), mostly `too_many_arguments` in view functions — the editor itself is `warning-clean` — so `-D warnings` fails on a fresh checkout. Treat 10 as the baseline: compare the count before and after your change and fix anything *your* code adds. Do not open a pull request that raises it, and do not be alarmed when it is not zero.

The first build is slow — SQLite compiles from source and PDFium downloads — and every build after that is normal. The binary lands at `target/release/md-editor` (`target\release\md-`

`editor.exe` on Windows) with the PDFium shared library copied beside it. It accepts a single path argument, which is how the Linux desktop entry hands it a file:

```
./target/release/md-editor ~/vault/Summary.md
```

3.3 Where the code actually is

The repository has 72 Rust files and about 33,000 lines. That distribution is very uneven, and knowing the shape of it is worth more than memorising the tree.

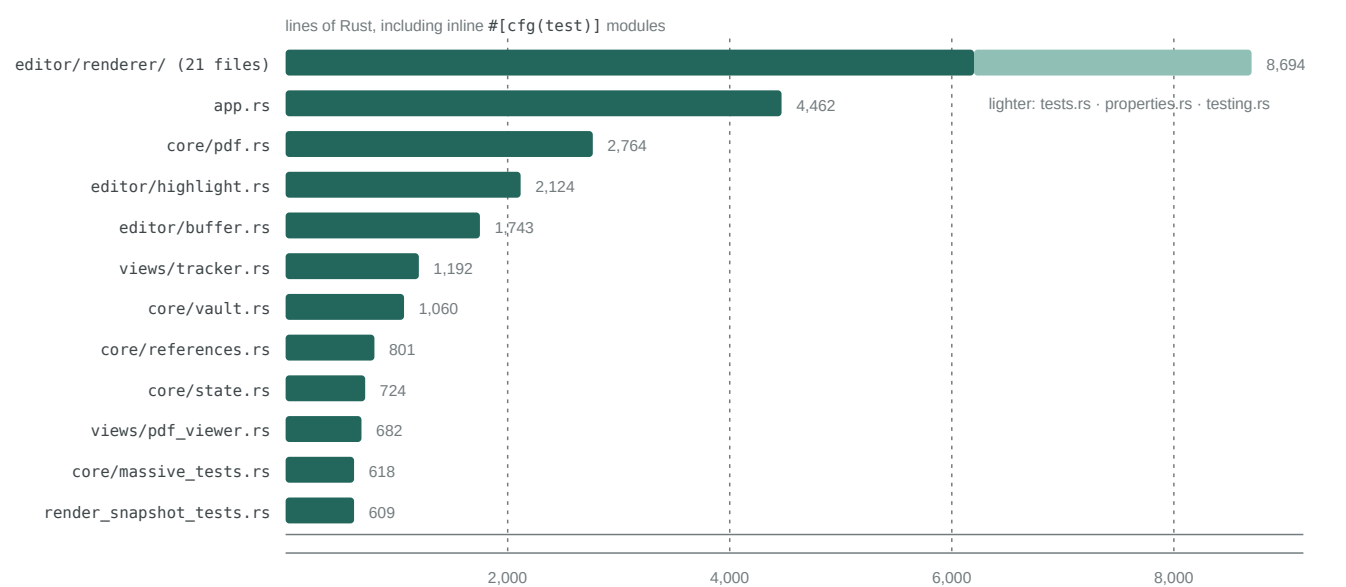


Figure 3.1 — Where the mass is. The eight largest entries are just over two thirds of the codebase, and the editor's `renderer/` directory and `app.rs` alone are two fifths. `renderer/` was a single 4,400-line `renderer.rs` until it was split into one module per concern (Chapter 17); a quarter of it is now tests. That is not an accusation — a custom text widget and an Elm-style router are both naturally large — but it tells you where to spend your reading time, and it explains why Chapters 11 and 17 are the long ones.

3.4 A first walk through the tree

Read these seven files, in this order, before you try to change anything. Skim; do not study. The goal is to have seen the names once.

#	FILE	WHY IT IS ON THE LIST
1	native/src/messages.rs	The complete event vocabulary of the application in one enum. Reading it is the fastest way to learn what the program can be asked to do.
2	native/src/main.rs	Short. CLI arguments, window geometry restore, the Linux desktop entry, and the call that starts the Iced application.
3	core/src/lib.rs	Ten lines. Tells you exactly which modules the engine has.
4	core/src/state.rs	The <code>AppState</code> struct, the whole SQL schema, and the database path logic. Chapters 5–7.
5	core/src/vault.rs	Find <code>write_file</code> and read it twice. Chapter 8 is about those forty lines.

#	FILE	WHY IT IS ON THE LIST
6	native/src/theme.rs	Every colour, size, spacing step and radius the UI is allowed to use. Small, and it will stop you writing a hardcoded <code>12.0</code> in your first pull request.
7	native/src/app.rs — the <code>update</code> signature and the <code>subscription</code> function only	Do not read all 4,462 lines. Read the shape: a match over <code>Message</code> , returning <code>Task<Message></code> . Chapter 11.

TRY IT

Run the test suite and read its output rather than just its exit code:

```
cargo test --workspace 2>&1 | grep -E "^test |test result"
```

215 test names is a surprisingly good table of contents for a codebase. Names like `write_file_never_leaves_a_truncated_file` and `blank_pages_reserve_space_in_total_height` tell you what the authors were afraid of, which is usually the most useful thing to know about a program.

3.5 Optional: desktop integration on Linux

Linux builds are portable by default and integration is explicitly opt-in:

```
./target/release/md-editor --install      # or --install-desktop
./target/release/md-editor --uninstall    # or --uninstall-desktop
```

`--install` writes `~/.local/share/applications/md-editor.desktop`, installs the icon at seven sizes under `~/.local/share/icons/hicolor/`, and refreshes the desktop and icon caches where those tools exist. Both flags exit non-zero with a message on other platforms. The whole round-trip has a test — `tests::test_linux_desktop_installation_roundtrip` — which runs against a temporary `$HOME`, so it is safe on your machine.

The language of this codebase

Twenty-three words you need, and the journey of one keystroke that uses most of them

Every codebase has private vocabulary, and misreading one word costs an hour. Here are the terms this one uses, roughly in the order you will meet them.

4.1 Vocabulary

TERM	WHAT IT MEANS <i>HERE</i>
Vault	The open folder. Not a container, not a database — a directory you chose with a folder picker. All paths in the app are relative to its root.
Path jailing	Canonicalising a path and refusing it if it does not lie strictly inside the vault root. <code>resolve_vault_path_checked()</code> in <code>vault.rs</code> .
Atomic write	Write to a sibling temp file, fsync, rename over the target. A reader sees the whole old file or the whole new one. Chapter 8.
Sidecar	Data <i>about</i> a file, stored outside it. Every PDF highlight is a sidecar record in SQLite.
<code>document_id</code>	SHA-256 of a PDF's first 1 MiB plus its length. The identity a highlight is attached to. Deliberately excludes mtime. Chapter 22.
Rope	The <code>ropey::Rope</code> data structure behind <code>DocBuffer</code> . Makes inserts and line lookups cheap on large files.
Transaction (editor)	A group of edit operations plus their inverses, so undo is a replay rather than a snapshot. Not a database transaction.
Undo run	Several consecutive keystrokes coalesced into one undo step, so <code>Ctrl+Z</code> takes back a word. Chapter 13.
Retained buffer	A parked <code>DocBuffer</code> — with undo stack, cursor and scroll — kept in an LRU registry of 32 when you navigate away from a note.
Conceal / reveal	Hiding markdown syntax markers when the cursor is elsewhere and showing them when it is on the line. The whole “hybrid preview” idea. Chapter 14.
<code>StyledSpan</code>	A run of characters with style, carrying both the raw <code>text</code> and an optional <code>display_text</code> . Concealing sets <code>display_text = Some("")</code> .
Fenwick / <code>HeightTree</code>	A binary indexed tree over per-line visual heights. Turns “which line is at y?” from $O(N)$ into $O(\log N)$. Chapter 15.
Culling	Drawing only the lines inside the viewport. The draw pass must never iterate the whole document. Chapter 17.
<code>Flow</code>	The inline layout of one line: its rows, and every run of text on them with its x and source columns. Built in <code>renderer/flow.rs</code> and nowhere else; painting, the caret and hit testing all read it. Chapter 17.
Caret affinity	Which side of a row break the caret is drawn on, when one column is both the end of a row and the start of the next. Owned by <code>DocBuffer</code> . Chapter 13.
Generation	A monotonically increasing id stamped on async work. A result whose generation no longer matches is discarded. Used by PDF rendering; markdown highlighting no longer needs one, because it runs synchronously (Chapter 14). Chapter 23.

TERM	WHAT IT MEANS <i>HERE</i>
Pending set	The set of in-flight requests of one kind (<code>pending_pages</code> , <code>pending_links</code> , <code>pending_text</code>). Cleaned up <i>before</i> the generation check, always.
Priority channel	The second mpsc channel into the PDFium worker, drained before each standard command, keeping only the newest request. Chapter 18.
Page slot	The fixed outer rectangle every PDF page occupies while scrolling, whether or not its bitmap has arrived. Chapter 24.
Synthetic TOC	An outline recovered from a PDF that has no embedded bookmarks, and marked as recovered in the UI. Chapter 20.
Reference link	A recognised in-prose mention like “Figure 1.1” turned into a clickable target, but only when the label matches exactly one target. Chapter 21.
Design token	A named constant in <code>theme.rs</code> or <code>motion.rs</code> . Views are not allowed to use anything else. Chapter 26.
Spring	The critically damped, closed-form motion in <code>motion.rs</code> for anything that follows a moving target — the editor scrolling to the caret, the caret gliding. Sampled at the frame's time, never stepped per frame. Chapter 26.
Subscription	An Iced stream of events the app opts into — and, in this codebase, one it opts <i>out</i> of the moment it has nothing to do. Chapter 12.

4.2 One keystroke, end to end

Here is the whole vocabulary doing its job at once. You press `e` in a note.

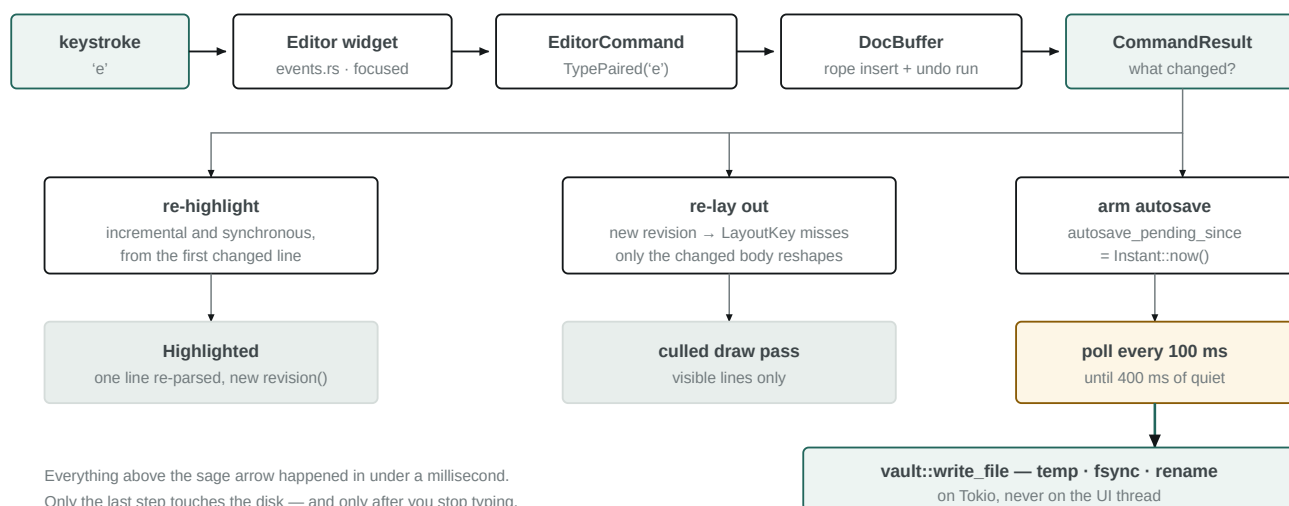


Figure 4.1 — The life of one character. Nine subsystems, one keypress. Notice the asymmetry: the immediate work is all in memory and the only I/O is deferred behind a debounce. Every chapter in Parts II and III is a zoom into one box of this figure.

WHY IT IS LIKE THIS

Notice what the diagram does *not* contain: a re-parse of the whole document, a full-document layout pass, or a synchronous write. Each of those would be the obvious first implementation, and each was deliberately replaced. Incremental highlighting, the height tree with its layout key, and the autosave debounce are the three replacements, and they are Chapters 14, 15-16 and 12 respectively.

The engine

md-editor-core owns everything that outlives the window: the files on disk, the database beside the executable, the link graph, and the identity of every PDF you have ever annotated. It compiles and tests with no display server at all.

1. AppState and the lock discipline
2. The database
3. Where the database lives
4. The atomic save
5. Wikilinks and the backlink graph
6. Full-text search

AppState and the lock discipline

Four mutexes, and the rule that keeps them from ever being held long

The whole engine hangs off one struct, shared as an `Arc`. It is small enough to quote in full, and the interesting part is not what it contains but how long its locks are held.

CORE/SRC/STATE.RS

```
pub struct AppState {
    pub vault_root: Mutex<Option<PathBuf>>,
    pub file_index: Mutex<FileIndex>,
    pub db: Mutex<Connection>,
    pub pdf_renderer: Option<PdfRenderer>,
}
```

Three `std::sync::Mutex` fields and one optional renderer handle. There is no `RwLock`, no async mutex, and no interior-mutability cleverness — because the design goal is not “fast locks” but **short** locks.

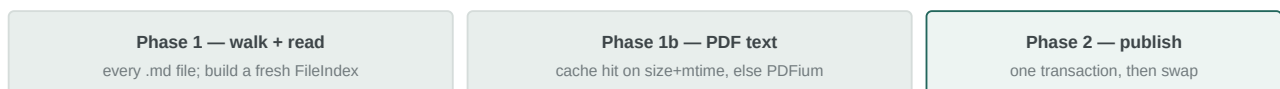
Two constructors exist. `AppState::new()` opens the on-disk database; `AppState::new_in_memory()` opens an in-memory one with no PDF renderer, which is what the test suite uses — that single line is why 56 core tests can run in a container with no display and no filesystem state to clean up.

5.1 The rule: do the slow work before you take the lock

The clearest example is `set_vault_root`, which runs when you open a vault. It has to walk the entire directory tree, read every markdown file, extract text from every PDF, build a fresh link index, and replace the full-text search table. On a large vault that is seconds of work. If any of it happened while holding `db` or `file_index`, the UI would freeze for exactly that long the moment anything else touched the engine.

So the function is deliberately split into phases, and only the last one takes a lock.

ELAPSED WORK →



LOCKS HELD



The trap this avoids

Read a file **while holding** the db lock and every other caller — autosave, search, the backlinks panel — waits on your slowest PDF. The freeze is proportional to the vault, not the edit.

Inside the short lock, only three things happen:

- 1 · publish the new vault root
- 2 · DELETE + INSERT all `file_search` rows in one transaction
- 3 · swap in the already-built FileIndex

Figure 5.1 — Two-phase indexing. The expensive 99% of the work happens with no locks held; the mutexes are taken only for the final swap. This is the pattern to copy whenever you add engine work: compute into a local, then lock, then assign.

WHY IT IS LIKE THIS

A Mutex in Rust is not slow. What is slow is the code you run while holding it. The compiler will happily let you `read_to_string` inside a guard's scope, and nothing will look wrong in review — the bug shows up only as “the app hangs for two seconds when I open a big vault”, which is exactly the kind of defect that never gets a bug report, only a shrug.

CHECK YOURSELF

You are adding “count the words in every note in the vault” to the engine. Sketch the shape.

Answer: lock `vault_root`, clone the `PathBuf`, drop the guard. Walk and count with no locks held. If you need to store the result, take the `db` lock only for the write. Three locks, each held for microseconds — and no lock held across a file read.

The database

Eight tables, two pragmas, and a migration strategy that fits in a paragraph

One SQLite file holds every piece of state that is not your content. It is opened once at startup, kept behind the `db` mutex, and never sharded, replicated, or versioned by hand.

6.1 Two pragmas

```
PRAGMA journal_mode = WAL;  
PRAGMA synchronous = NORMAL;
```

Both are applied best-effort at connection time and fall back silently if the platform does not support them. **WAL** (write-ahead logging) keeps writes fast and, critically, stops a writer from blocking a reader — which matters because the UI reads settings on paths a background task may be writing to. `synchronous = NORMAL` is the documented pairing with WAL: safe against an application crash, and it does not fsync on every commit.

NOTE

`synchronous = NORMAL` with WAL is *not* the same guarantee as the atomic file write in Chapter 8. Your notes are protected by the temp-file-and-rename protocol; the database is protected by WAL. Different mechanisms, because they protect different things: one is your writing, the other is your scroll position.

6.2 The schema

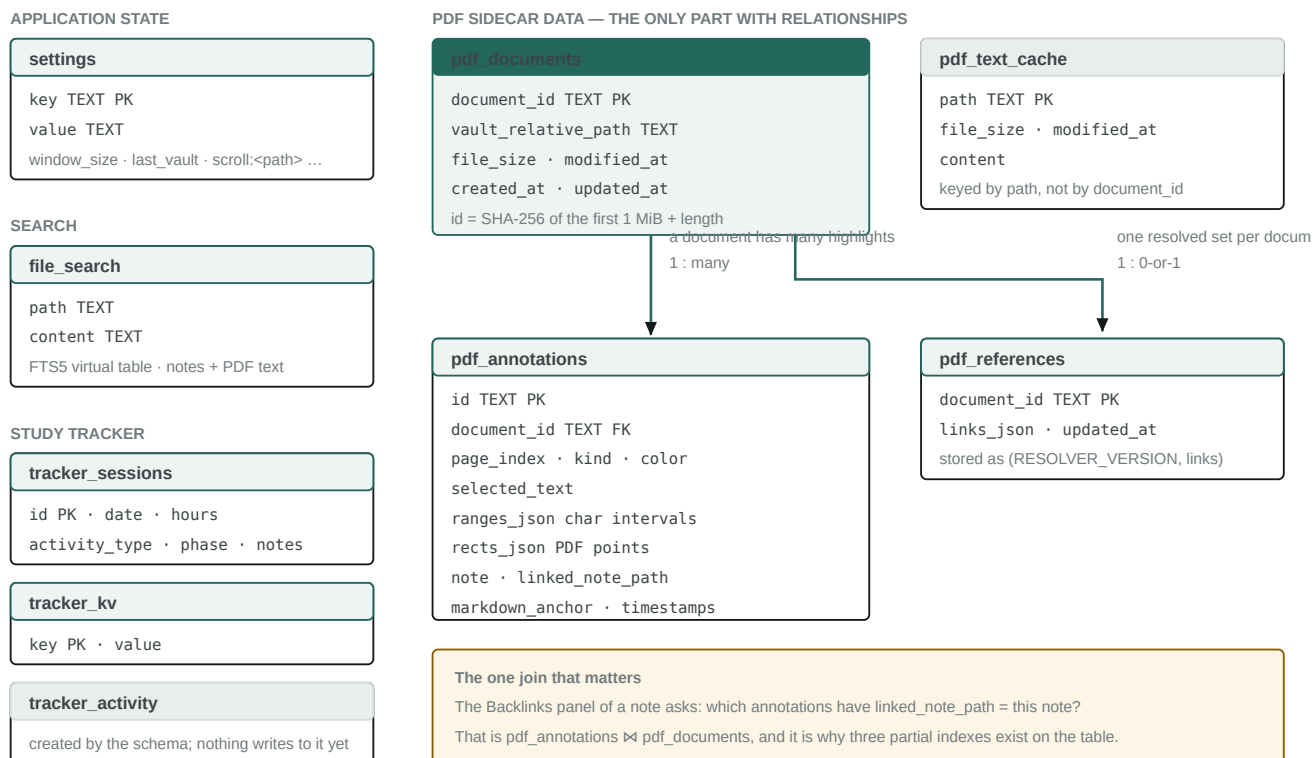


Figure 6.1 — The schema, grouped by what it is for. Seven of the eight tables are flat key-value or log tables; all the relational structure lives in the PDF cluster, hanging off `pdf_documents.document_id`. Note that `pdf_text_cache` is keyed by `path`, not by document id — it is a cache of extraction work, not a fact about a document.

What each table is for

TABLE	PURPOSE, AND THE THING THAT IS EASY TO GET WRONG
<code>settings</code>	Key/value application state. The keys actually written are <code>window_size</code> , <code>last_vault</code> , <code>last_file</code> , <code>scroll:<path></code> , <code>cursor:<path></code> and <code>tracker_config</code> . Window <i>position</i> is deliberately not stored — only size.
<code>file_search</code>	An FTS5 virtual table over <code>(path, content)</code> holding both markdown text and extracted PDF text, so one query covers both. Rebuilt wholesale inside one transaction when a vault opens — not incrementally.
<code>pdf_documents</code>	Known PDFs by content id. When a PDF is copied or restored and its id changes, rows recorded for the same path <i>and the same size</i> are adopted under the new id, so annotations are not orphaned. The size check is what stops a genuinely different file inheriting them.
<code>pdf_annotations</code>	The sidecar highlights and notes. <code>ranges_json</code> is character intervals on the page text layer; <code>rects_json</code> is bounding boxes in PDF points (Chapter 19 explains why points and not pixels).
<code>pdf_text_cache</code>	Extracted text by vault-relative path, invalidated by size and mtime. Empty results are cached too , so a scanned PDF with no text layer is not re-extracted on every vault open.
<code>pdf_references</code>	The resolved cross-reference list per document, stored as <code>(RESOLVER_VERSION, links)</code> so that changing the resolver transparently invalidates every cached result.

TABLE	PURPOSE, AND THE THING THAT IS EASY TO GET WRONG
<code>tracker_sessions</code> , <code>tracker_kv</code>	Logged study sessions, and the curriculum's checkbox and status flags.
<code>tracker_activity</code>	Created by the schema, available for an activity feed, and currently written by nothing. Left in deliberately; deleting it would be a migration for no benefit.

6.3 Migrations

Versioning uses SQLite's built-in `PRAGMA user_version`. There is no migration framework and no `migrations/` directory. The whole strategy is two functions:

- `init_schema()` runs the full idempotent DDL — every statement is `CREATE TABLE IF NOT EXISTS`.
- `apply_migrations()` runs afterwards and holds the additive steps, each bumping `user_version`.

TRAP

Adding a column by editing the base DDL **only affects fresh databases**. Every existing user keeps the old table and your code starts failing on a column that does not exist. New columns go in `apply_migrations()`, always — even when you are “sure” nobody has an old database yet, because you have one on your own machine from last week.

Where the database lives

Portable by default, with two fallbacks and one careful migration

Most desktop applications put their state in a per-user configuration directory and never think about it again. MD Editor puts it beside the executable, which is a deliberate and slightly unusual choice with a small amount of machinery behind it.

The goal is that the whole application is *one folder*: binary, PDFium library, icon, licence, and `md_editor_settings.sqlite`. Copy that folder to a USB stick and your settings, highlights and study history come with it. That only works if the executable's own directory is writable — which it is for a portable download, and is not for a system install under `/usr/bin`. Hence a ladder.

STEP 1 — CHOOSE A DIRECTORY (FIRST WRITABLE WINS)



STEP 2 — OPEN OR CREATE

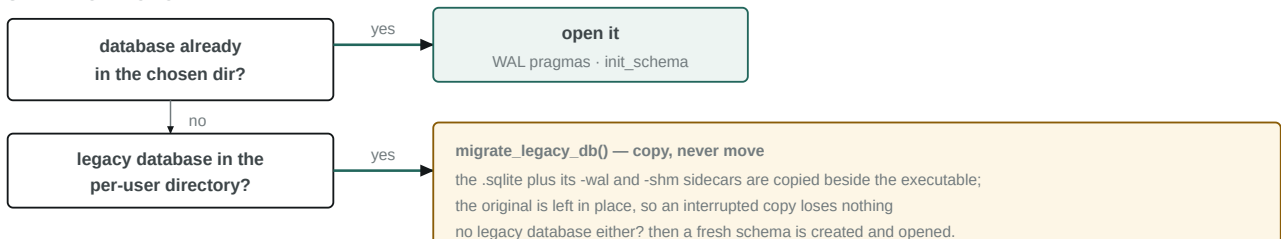


Figure 7.1 — Choosing the database directory. The ladder runs top to bottom on every launch. The lower half is the part worth remembering: an interim version of the app stored the database in the per-user directory, so there is a one-time copy back to the portable location — and it is a *copy*, with the `-wal` and `-shm` files brought along so not-yet-checkpointed writes survive.

WHY IT IS LIKE THIS

Copying rather than moving looks wasteful — you now have two databases. It is the right call anyway: a move that is interrupted halfway (a full disk, a killed process) leaves the user with neither file. A copy that is interrupted leaves the original untouched, and the next launch simply tries again.

TRY IT

Build a release binary, copy it plus `libpdfium.so` into an empty directory, run it, open a vault, and confirm `md_editor_settings.sqlite` appears *in that directory*. Then `chmod -w` the directory and run again: the database should be created under `~/.local/share/md-editor/` instead. That is the whole ladder, observable in two minutes.

The atomic save

Forty lines in `vault.rs` that decide whether the program is trustworthy

This is the most important function in the repository. If you read one thing carefully, read `vault::write_file` — and then read the four unit tests underneath it, which exist because every step below is load-bearing.

8.1 The problem

The naive save is `fs::write(path, content)`. It opens the destination, truncates it to zero bytes, and writes. If the process dies in the middle — a crash, a `kill -9`, a power cut — the file on disk is now half a note. The old version is gone and the new one never arrived. For an application whose entire pitch is “your work is safe in plain files”, that is not an edge case; it is the thing being promised against.

Worse, this app saves *constantly*: 400 ms after you stop typing, on every navigation, and on window close. A rare failure mode multiplied by a very high write frequency is a common failure mode.

8.2 The protocol

In the order the code performs it:

1. **Canonicalize the destination.** A symlinked note is written *through* to its target rather than replaced by a regular file.
2. **Create a sibling temp file** — `.<filename>.<uuid>.tmp`, in the destination's own directory, so the final rename can never cross a filesystem boundary. The UUID means two concurrent saves cannot collide.
3. **Write the bytes, then copy the destination's permissions** onto the temp file, so a note kept at `0600` does not come back world-readable.
4. `file.sync_all()` — flush the contents to physical storage.
5. `fs::rename()` — atomically replace the destination.
6. `sync_all()` **on the parent directory**, best-effort, so the rename itself survives a power loss.

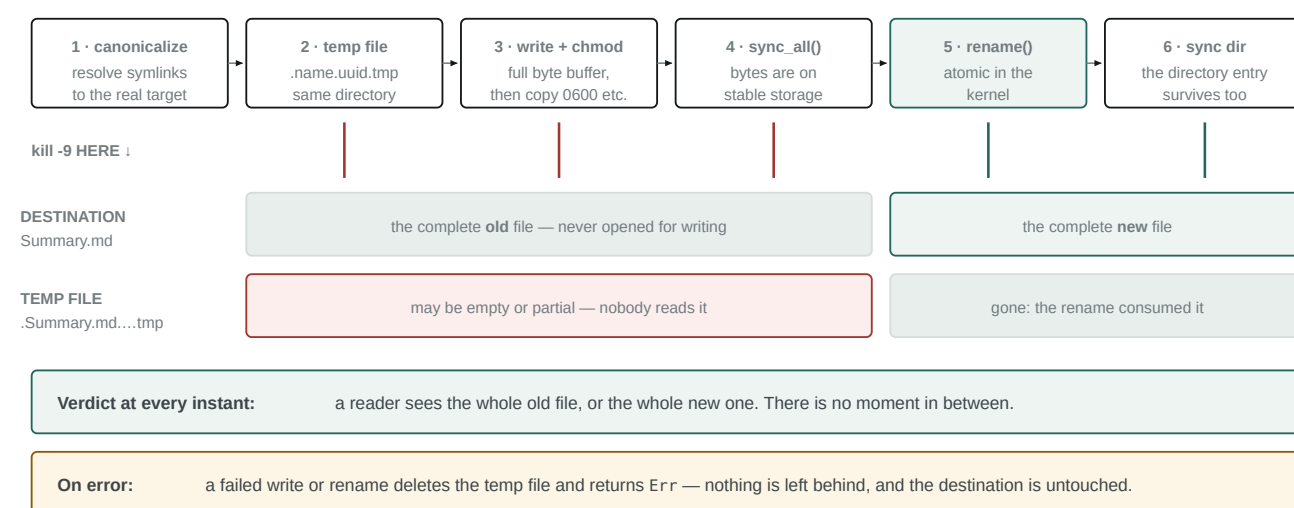


Figure 8.1 — Kill it anywhere. The top row is the protocol; the two lanes below show what is actually on disk at that instant. The destination is never open for writing, which is the whole trick — the new content becomes visible in one indivisible kernel operation, or not at all.

8.3 The tests that hold it in place

Four unit tests live in `core/src/vault.rs` and each maps to one property above. If you change this function, you will meet them:

- content replacement leaves **no** `.tmp` files behind;
- permissions are **preserved**;
- a symlink is written **through**, not over;
- `write_file_never_leaves_a_truncated_file` — the name is the specification.

8.4 Where saves are triggered from

Three call sites, with three different failure policies. The differences are deliberate.

TRIGGER	POLICY ON FAILURE
Autosave — 400 ms after the last keystroke, polled every 100 ms	Raise a toast the <i>first</i> time in a run of failures, keep the unsaved marker in the title, and restart the debounce. Toasting on every poll would be a strobe light.
Navigation — opening another note, an image, or a PDF	Refuse to navigate. The view does not change and the error is surfaced. Walking away from work that could not be persisted is the one thing the app will not do.
Window close	Best-effort, and it does <i>not</i> block the close. An app that refuses to quit because a write is failing is a worse outcome than losing the few hundred milliseconds autosave had not yet committed.

WHY IT IS LIKE THIS

The close-window exception looks like an inconsistency and is actually the harder call. The reasoning is about what the user can do next: a refused *navigation* leaves them in the editor, where they can fix the problem. A refused *quit* leaves them with a window that will not close, which is a support ticket and a force-kill — and a force-kill loses strictly more.

CHECK YOURSELF

Why must the temp file be a *sibling* of the destination, rather than in `/tmp`?

Answer: `rename()` is only atomic within a single filesystem. A temp file in `/tmp` is very likely on a different mount, so the “rename” would degrade into a copy-then-delete — which is exactly the non-atomic write the whole protocol exists to avoid, now with extra steps.

Wikilinks and the backlink graph

One regex, a three-step resolver, and why incoming is stored rather than derived

A vault is a graph as well as a folder. `core/src/file_index.rs` is 403 lines that keep that graph correct while files are created, renamed, edited and deleted underneath it.

9.1 Parsing

CORE/SRC/FILE_INDEX.RS

```
static WIKILINK_RE: LazyLock<Regex> =
    LazyLock::new(|| Regex::new(r"\[\[([^\]]+)\](?:\|([^\]]+)\]?)\]").unwrap());
```

Compiled once, reused for every file in the vault. It matches both the bare form `[[Note Name]]` and the aliased form `[[Note Name|Display Text]]`, and captures only the target — the alias affects display, never resolution.

NOTE

LazyLock matters more than it looks. Compiling a regex is expensive; compiling it inside the per-file loop would make indexing a 2,000-note vault noticeably slower for no reason at all. If you add a pattern to this module, add it the same way.

9.2 Resolution

`resolve_link(target, from_file)` tries three things in order.

THE VAULT

```
Lectures/
  Week1.md

Archive/
  2023/
    Note.md
    Note.md  ← shortest path
    Summary.md
```

In Lectures/Week1.md:
see `[[Note]]` for the proof

RESOLVE_LINK — FIRST MATCH WINS

1 · exact path match

handles `[[Archive/2023/Note]]` and a root-level `[[Summary]]`. No match here — “Note” is not a path.

2 · shortest basename match

names: `HashMap<String, BTreeSet<PathBuf>>` — lowercased stem → ordered candidates

`"note" → { Archive/2023/Note.md, Note.md } → Note.md wins`

3 · optimistic forward reference

no file with that stem is known yet? keep the path-based candidate anyway, so creating the note later makes the backlink appear with no re-index.

BOTH DIRECTIONS ARE STORED

outgoing

`Lectures/Week1.md → { Note.md }`
rebuilt from the file's own text whenever it changes

incoming

`Note.md → { Lectures/Week1.md }`
maintained explicitly, so the Backlinks panel is one map lookup

Figure 9.1 — Why the shortest path wins. Two files are called `Note.md`. A `BTreeSet` keeps the candidates ordered, so the choice is *deterministic* — the same file is picked by the link, by the click-through, and by the backlink. Any tie-break would do; what matters is that all three agree.

9.3 The bidirectional index

CORE/SRC/FILE_INDEX.RS

```
pub struct FileIndex {
    pub outgoing: HashMap<PathBuf, HashSet<PathBuf>>, // file → files it links to
    pub incoming: HashMap<PathBuf, HashSet<PathBuf>>, // file → files that link to it
    vault_root: PathBuf,
    names: HashMap<String, BTreeSet<PathBuf>>,          // lowercased stem → candidates
}
```

`incoming` could be derived from `outgoing` on demand — scan every file's outgoing set looking for the one you care about. It is stored instead, because the Backlinks panel is drawn on every relevant redraw and a map lookup is free where a full scan is not.

Two functions maintain it:

- `rebuild()` runs in **two passes**: register every filename first, then resolve links. This is not a style choice — a bare `[[Name]]` must be able to reach a file that appears later in the listing, or in a different subfolder. A single pass would resolve links against a half-built name map and produce different results depending on directory iteration order.
- `update_file()` does the same incrementally for one edited file, and its first action is to *remove that file's previous outgoing edges from every target's `incoming` set*. Forget that and deleting a link from a note leaves a permanent phantom backlink.

TRAP

Anything you change here must survive `core/src/massive_tests.rs`, which fuzzes wikilink variants, graph topologies (stars, chains, cycles, disconnected islands) and a dynamic create/edit/delete workload. That suite exists because this exact code is easy to get subtly wrong and impossible to eyeball.

Full-text search

One FTS5 table, one quoting rule, and backlinks that are not only wikilinks

Global search is a single SQL statement. The interesting parts are the quoting, and the fact that the index contains PDF text as well as markdown.

10.1 The query

CORE/SRC/VAULT.RS::SEARCH_VAULT

```
SELECT path, snippet(file_search, 1, '<b>', '</b>', '...', 15)
FROM file_search
WHERE content MATCH ?1
ORDER BY rank
LIMIT 100;
```

The query string is wrapped in double quotes before it reaches `MATCH`, with any embedded quotes doubled, so it is treated as an FTS5 *phrase* rather than as query syntax.

WHY IT IS LIKE THIS

FTS5's `MATCH` takes a small query language: `AND`, `OR`, `NEAR`, prefix stars, column filters. Passing raw user input straight in means someone searching for `NOT` or `a AND b` gets an unexpected result — or a syntax error where they expected zero hits. Quoting turns the whole input into one literal phrase, which is what a search box in a notes app should mean.

Results carry the path and a highlighted snippet, ordered by FTS5's own `rank`, capped at 100. On the native side the query is debounced by `SEARCH_DEBOUNCE = 200ms`, so a ten-character query runs one scan rather than ten (Chapter 12).

10.2 One index, two kinds of document

`file_search` holds `(path, content)` rows for markdown notes *and* for text extracted from PDFs. That is the entire reason global search covers papers as well as notes, and it is why vault indexing (Chapter 5) has a phase 1b that runs PDFium.

The table is rebuilt wholesale, inside one transaction, whenever a vault is opened — not maintained incrementally. For vault sizes this app targets that is simpler and fast enough, and it means the index can never drift out of sync with the filesystem in a way that requires a “rebuild index” command.

10.3 Backlinks beyond markdown

`get_mixed_backlinks(state, path)` answers “what points at the thing I am looking at?”, and the answer depends on what that thing is.

ACTIVE DOCUMENT	WHAT COMES BACK
A markdown note	<code>MarkdownFile { path }</code> — every note whose <code>[[wikilink]]</code> resolves to this one, straight from <code>FileIndex::get_backlinks</code>. <code>PdfAnnotation { document_path, annotation_id, page }</code> — every sidecar highlight whose <code>linked_note_path</code> is this note, joined from <code>pdf_annotations</code> to <code>pdf_documents</code>. Activating one opens that PDF at the exact page and focuses the highlight.
A PDF	<code>MarkdownFile { path }</code> — notes that wikilink to the PDF itself. <code>MarkdownFile { path }</code> again, but carrying the highlight's <code>selected_text</code> as context — notes reached <i>through</i> this PDF's own annotations, so the panel can show which passage produced the link.

NOTE

`BacklinkTarget::PdfDocument { path }` is defined in `core/src/types.rs` and rendered by `views/backlinks.rs`, but **no query currently emits it**. It is a wired-up variant with no producer — worth knowing before you spend an hour looking for the code path that creates one.

CHECK YOURSELF

A user searches for `1.5"` (with a quote character). What reaches SQLite?

Answer: `"1.5""` — the whole thing wrapped in quotes with the embedded quote doubled, which FTS5 reads as the literal phrase `1.5"`. Without the doubling the quote would terminate the phrase early and the rest would be parsed as query syntax.

The markdown editor

Not a text input with syntax colouring — a custom Iced widget with its own layout, hit testing, spatial index and caching strategy, built so that a 10,000-line note scrolls as smoothly as a ten-line one.

1. The shell: one loop, one router
2. Subscriptions and the zero-idle-CPU rule
3. DocBuffer, transactions, and word-sized undo
4. Hybrid preview and incremental highlighting
5. The Fenwick height tree
6. Measure once: the line cache and the layout key
7. The widget: rhythm, rows, and the draw pass

The shell: one loop, one router

The Elm architecture, and the rule that keeps app.rs from becoming everything

Iced is an Elm-architecture toolkit: state, a `Message` enum, an `update` that folds messages into state, and a `view` that renders state. MD Editor starts the application through Iced's function API rather than the older trait.

NATIVE/SRC/MAIN.RS

```
iced::application(
  move || app::MdEditor::new_with_startup_file(startup_path.clone()),
  app::MdEditor::update,
  app::MdEditor::view,
)
.title(app::MdEditor::title)
.theme(|state: &app::MdEditor| state.theme())
.subscription(app::MdEditor::subscription)
.window(iced::window::Settings { size: restore_window_size(), icon, platform_specific,
  ..Default::default() })
.run()
```

11.1 The loop

Startup is short and worth knowing, because two of its steps are easy to be surprised by: `read_startup_value("window_size")` opens *its own* database connection before `AppState` exists — the window has to be sized before the app is constructed — and the restored size is parsed and clamped, falling back to 1200×800 on anything nonsensical.

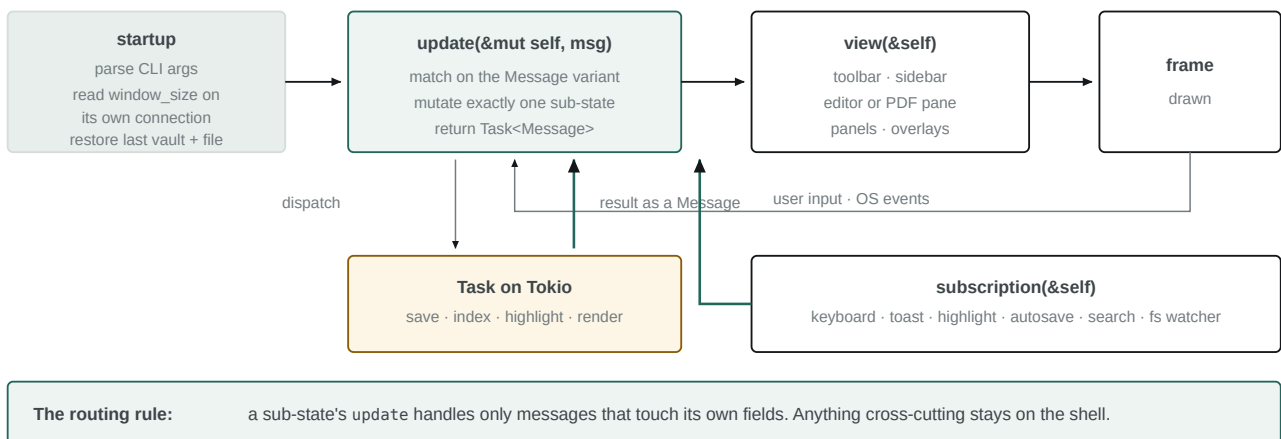


Figure 11.1 — One loop, three inbound edges. Messages arrive from user events, from finished background tasks, and from subscriptions. All three land in the same `update`, which is why the message enum is the best map of the program (Appendix C).

11.2 Sub-states, and why `active_path` is not one of them

`MdEditor` is not one giant struct of fields; it composes seven domain sub-states, each owning its own data and the messages that touch only that data.

SUB-STATE	OWNS
<code>EditorPane</code> <code>editor_state.rs</code>	The active <code>DocBuffer</code> ; its <code>Highlighted</code> lines, updated incrementally and synchronously on every edit; the image and math caches; the autosave timestamp, scroll offset, the pending caret-reveal request, TOC visibility, and the retained-buffer registry (<code>MAX_RETAINED_BUFFERS = 32</code>). <code>layout_revision()</code> names what the editor widget lays out, so it can skip layout on frames where nothing changed. Also defines <code>AUTOSAVE_DEBOUNCE = 400ms</code> and <code>AUTOSAVE_POLL = 100ms</code> .
<code>VaultState</code> <code>vault_state.rs</code>	Vault root, file entries, sidebar selection and folder expansion, the backlinks panel.
<code>UiState</code> <code>ui_state.rs</code>	Modals, the command palette and its highlighted row, the toast, split-view state and divider ratio, window size, and the display scale factor that drives PDF supersampling.
<code>SearchState</code> <code>search_state.rs</code>	Query and replacement text; the <code>regex</code> , <code>match_case</code> and PDF-only <code>loose</code> flags; vault and PDF results; and a memoized in-document match cache keyed by <code>DocMatchKey</code> so <code>view</code> never rescans the buffer. <code>SEARCH_DEBOUNCE = 200ms</code> .
<code>TrackerState</code> <code>tracker_state.rs</code>	Timer state, loaded sessions, tracker key-values, active tab, and the JSON config editor content.
<code>PdfPane</code> <code>pdf_pane.rs</code>	The PDF viewport, page caches, annotations, selection, and the render generation. Chapters 23–24.
<code>Motion</code> <code>motion.rs</code>	How far each animated panel has travelled, and the single <code>Instant</code> the current frame is sampled at. Chapter 26.

Three pieces of state deliberately stay on the shell itself: `active_path` (the current document, which the editor, search, and PDF pane all need), `active_panel` (which pane last received direct interaction, so split-view shortcuts route correctly), and `showing_pdf`.

WHY IT IS LIKE THIS

The shell is also the only place that owns the shared `Arc<AppState>`. That is not incidental: it means a sub-state *cannot* reach into the engine even if someone wants it to, so “open a file” or “create a vault entry” can only be written in the one place that can also update every other pane afterwards. Restricting access is how the coupling stays visible.

TRAP

When you add a message, the question to answer first is “does this touch fields of exactly one sub-state?”. If yes, its handler goes there. If it touches two — or touches the engine — it goes on the shell. Getting this wrong is how a 4,000-line `app.rs` becomes a 6,000-line one.

Subscriptions and the zero-idle-CPU rule

Six streams, five of which usually return nothing at all

A subscription in Iced is a stream of events the application opts into. Most apps arm a handful of timers at startup and leave them running. This one arms each stream only while it has work, and the result is a settled window that uses **0.0% CPU** — a property the project treats as a release blocker, not a nice-to-have.

12.1 The six streams

STREAM	ARMED WHEN	INTERVAL
Keyboard	Always — but <code>iced::event::listen_with</code> returns <code>None</code> for anything the app does not act on, so ordinary typing (handled by the focused editor widget) never spawns a redundant update cycle.	event-driven
Toast timeout	<code>ui.toast</code> is set	fires <code>ToastHide</code> after 3s
Autosave poll	<code>editor.autosave_pending_since</code> is set	100 ms tick; writes once 400 ms of quiet have passed
Search debounce	a query keystroke is pending	200 ms tick
Filesystem watcher	a vault is open	<code>notify</code> recursive watch, debounced 300 ms
Scroll interrupt	the editor is gliding to reveal the caret — a wheel turn or click fires <code>EditorScrollInterrupted</code> and hands the page back to you (Chapter 17)	event-driven

Animation frames are armed the same way, from `motion.is_animating()` (Chapter 26). The editor widget follows the same rule for the frames it schedules itself: it asks for one only while the caret glides or fades, and for none once the caret rests lit.

WHY IT IS LIKE THIS

The autosave *poll* (100 ms) is deliberately finer than the autosave *debounce* (400 ms). A single 400 ms timer would fire on its own phase, not on yours: stop typing 10 ms after a tick and you wait 790 ms for the save. Polling four times as fast bounds the error to 100 ms, at the cost of a few no-op wakeups *while you are typing anyway*.

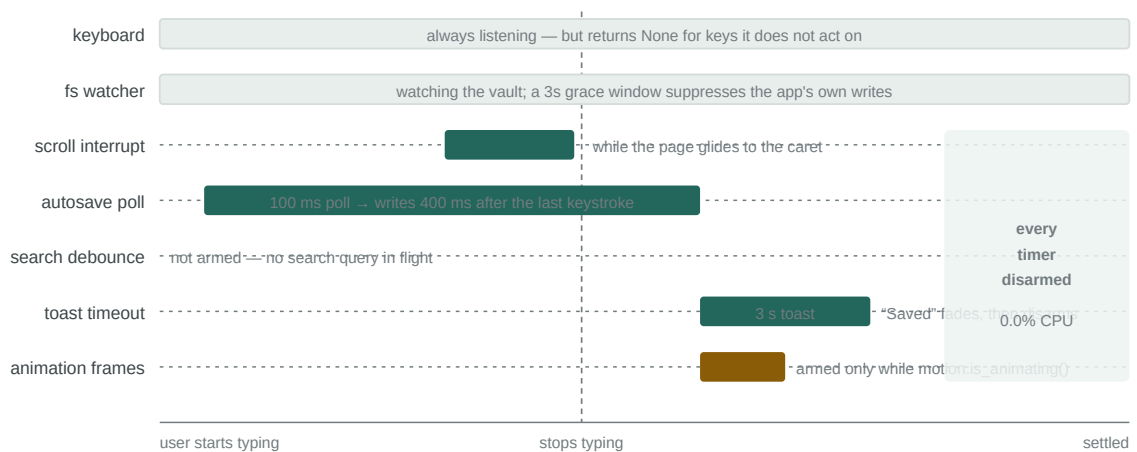


Figure 12.1 — Nothing is running when nothing is happening. Each bar is a subscription that returned something other than `Subscription::none()`. To the right of the settling point every lane is empty, which is the property the release checklist verifies with `top`.

12.2 The filesystem watcher

A `notify` recursive watch on the vault, debounced by `VAULT_WATCH_DEBOUNCE = 300ms`, emits `VaultFilesChanged` with the changed paths. External edits — a `git pull`, a change from another editor — refresh buffers, the file tree and the link index without a restart.

The subtlety is `SELF_WRITE_GRACE`: a 3-second window during which events caused by the app's *own* saves are suppressed. Without it, every autosave would bounce straight back as an external change notification, and the app would spend its time reacting to itself.

TRAP

Adding an always-on timer is a regression even when it looks cheap. “A 1-second tick to refresh the clock” costs a wakeup every second forever, on battery, in every window, and it defeats a property that took deliberate work to establish. If you need periodic work, arm it from the condition that makes it necessary and disarm it the moment that condition clears.

DocBuffer, transactions, and word-sized undo

Why one `Ctrl+Z` takes back a word, and how a quote knows it is an apostrophe

`DocBuffer` is the text model: a `ropey::Rope`, a cursor, a selection, and an undo stack of transactions. Everything the editor can do to text is an `EditorCommand` applied through `DocBuffer::execute()`.

NATIVE/SRC/EDITOR/BUFFER.RS

```
pub enum EditorCommand {
    InsertText(String), DeleteSelection, DeleteBackward, DeleteForward,
    MoveCursor { movement: Movement, extend: bool },
    SetCursor { line: usize, col: usize, affinity: Affinity },
    SetSelection { anchor_line: usize, anchor_col: usize,
                  focus_line: usize, focus_col: usize, affinity: Affinity },
    SelectAll, ToggleCheckbox { line: usize },
    FormatBold, FormatItalic, FormatInlineCode, InsertLink,
    TypePaired(char), Undo, Redo,
}
```

`execute()` returns a `CommandResult { text_changed, projection_changed, media_changed }`, so the shell knows whether to re-parse the document, refresh the image and math caches, or merely move the caret. Three booleans, and they save an enormous amount of unnecessary work: moving the cursor changes the projection but not the text, so nothing is re-highlighted.

WHY A ROPE

A `String` is one contiguous allocation. Inserting a character in the middle of a 2 MB note memmoves the tail — every keystroke. `ropey` stores the text as a tree of chunks, so an insert touches one chunk, and it also gives $O(\log n)$ conversion between character offsets and line/column, which the editor asks for constantly.

13.1 Human-sized undo

A keystroke-by-keystroke undo stack is technically correct and miserable to use: taking back one sentence costs a dozen presses. `commit_transaction` therefore *coalesces* consecutive edits into one undo step.

```
const UNDO_COALESCE_WINDOW: Duration = Duration::from_millis(300);
```

`can_coalesce(prev, next)` merges only when **every one** of these holds:

- the new transaction is a single operation;
- it is the same kind as the previous one — an insert continues inserts, a delete continues deletes;
- neither text contains a newline, so a run always breaks at a line boundary at the latest;
- the new character lands exactly where the previous one ended, so a cursor jump breaks the run;

- there was no active selection, so a selection-replacing edit is always its own step;
- less than `UNDO_COALESCE_WINDOW` has passed, so a deliberate pause starts a new step.

NATIVE/SRC/EDITOR/BUFFER.RS — THE INSERT ARM, LIGHTLY TRIMMED

```
(EditOp::Insert { char_offset: prev_at, text: prev_text },
 EditOp::Insert { char_offset: next_at, text: next_text }) => {
    next_text.chars().count() == 1
    && !next_text.contains('\n')
    && !prev_text.contains('\n')
    // The new character lands exactly where the previous one ended.
    && *next_at == prev_at + prev_text.chars().count()
}
```

KEYSTROKES →



Without coalescing

21 keystrokes → 21 undo steps.

Taking back "the cat sat" costs eleven presses, and each one redraws.

With coalescing

21 keystrokes → 6 undo steps.

One press takes back a word-sized burst, and the breaks land where a person would expect them to.

Figure 13.1 — Where a run breaks, and why. Every red bar is one of the six conditions in `can_coalesce` failing. Deletes coalesce by the same rules, which is why holding backspace is also a single undo step rather than forty.

13.2 Smart auto-pairing

Bracket and quote characters route through `TypePaired`; everything else inserts verbatim. Four behaviours, and the fourth is the one people notice when it is missing.

BEHAVIOUR	RULE
Brackets	<code>PAIRS = [('(', ')'), ('[', ']'), ('{', '}')]</code> . Typing an opener inserts the pair and places the cursor between them.
Selection wrapping	With a selection active, typing <code>(</code> , <code>[</code> , <code>{</code> , <code>"</code> , <code>'</code> or <code>`</code> wraps the selection rather than replacing it.
Skip-over	Typing a closer that already sits at the cursor steps over it instead of inserting a duplicate. Same for an identical quote directly ahead.
Contraction preservation	<code>"</code> , <code>'</code> and <code>`</code> pair by default — but a quote typed immediately after an alphanumeric character is left <i>single</i> , so <code>don't</code> , <code>it's</code> and <code>users'</code> do not sprout stray closers.

13.3 List continuation

SOURCE LINE	AFTER <code>ENTER</code>
- Buy milk	- Buy milk\n-
* [] Code task	* [] Code task\n* []

SOURCE LINE

AFTER ENTER

1. Step one

1. Step one\n2. — the number increments

- (an empty item)

the marker is removed and list mode ends

TRAP

The list parser works on **character offsets, not byte slices**. Slicing a Rust String at a byte index that falls inside a multibyte character panics — and a list marker followed by an accented letter or an emoji is not a hypothetical document. There is a regression test for exactly this. Any new text-scanning code in this file should use `chars()` and character indices for the same reason.

13.4 Grapheme clusters and caret affinity

Columns in `DocBuffer` are `char` offsets, but the caret never stops inside what a person would call one character. `←/→` step, and `Backspace/Delete` remove, one extended grapheme cluster (unicode-segmentation): 🐶 is three `char`s joined by a zero-width joiner, and one step. A line break — `\r\n` included — is one stop. `set_cursor`, `set_selection` and vertical movement snap a column that lands inside a cluster back to its start.

Wrapping creates a second ambiguity. Where a line breaks between rows, one column is *both* the end of the earlier row and the start of the later one, and the caret has to be drawn at exactly one of them.

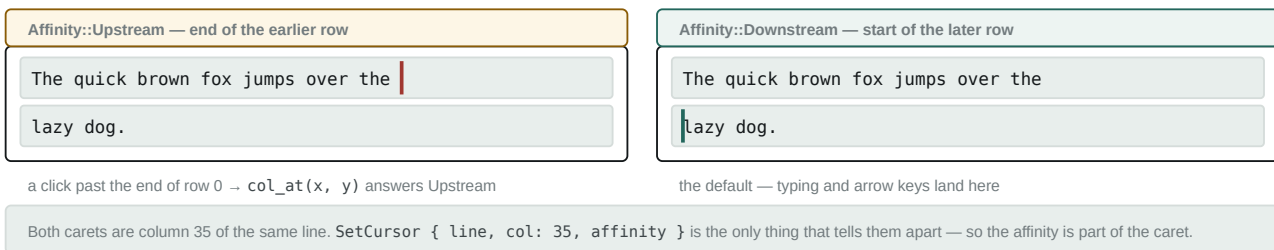


Figure 13.2 — One column, two places. Without affinity, clicking just past the end of a wrapped row would put the caret at the start of the next row — under a different word, on a different line of the screen — even though the click was on the first row.

Because the affinity describes one particular placement of the caret, the buffer owns it: `DocBuffer::cursor_affinity` is set only by `SetCursor` and `SetSelection`, and **reset by every other change**. It cannot outlive the placement it describes.

WHY IT IS LIKE THIS

If the widget kept its own copy of the affinity, an edit or an undo that moved the caret would leave that copy describing a placement that no longer exists, and the caret would be drawn at the end of the wrong row. Keeping every caret fact — position, selection, affinity — in `DocBuffer`, with no copy on the widget, makes that bug unrepresentable. Chapter 32 states it as a rule.

CHECK YOURSELF

You type `hello`, wait two seconds, type `world`, then press `Ctrl + Z` twice. What is on screen after each press?

Answer: `hello`, then empty. The two-second pause exceeded `UNDO_COALESCE_WINDOW`, so there are exactly two runs.

Hybrid preview and incremental highlighting

One data structure that holds both the source and what you see — kept in step with the text a line at a time

There is no preview mode and no edit mode. The document renders as formatted typography, and the line your cursor is on shows its raw markdown in place. The whole trick is that a span carries two strings.

14.1 The data structures

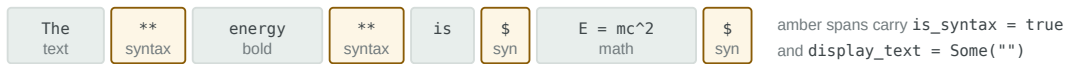
NATIVE/SRC/EDITOR/HIGHLIGHT.RS

```
pub struct StyledLine {
    pub spans: Vec<StyledSpan>,
    pub is_code_block: bool,
    pub is_math_block: bool,
    pub code_block_lang: Option<String>,
    pub is_blockquote: bool,
    pub block_id: usize,          // consecutive lines of one block share this
    pub is_block_fence: bool, // ``` or $$ — hidden in preview
    pub is_table_row: bool,
    pub table_cells: Vec<Vec<StyledSpan>>,
}

pub struct StyledSpan {
    pub text: String,                // the raw markdown source for this span
    pub display_text: Option<String>, // what preview shows; Some("") hides a marker
    pub color: Color,
    pub bold: bool, pub italic: bool, pub font_size: f32,
    pub is_code: bool,
    pub is_link: bool, pub link_target: Option<String>,
    pub is_heading: bool, pub heading_level: u8,
    pub is_checkbox: bool, pub is_checked: bool,
    pub is_list_marker: bool,        // bullet, number or checkbox: wrapped rows hang past it
    pub is_rule: bool,
    pub is_image: bool, pub image_path: Option<String>, pub image_alt: Option<String>,
    pub is_math: bool,
    pub is_syntax: bool,             // this span is a marker like ** or $ or ```
    pub id: Option<String>,
}
```

`text` always holds the exact source. Nothing is lost by concealing — the renderer simply chooses `display_text` instead. That is why editing works normally on a concealed line: the characters are still there, they are just drawn as nothing. Which spans on the active line actually reveal is decided per span, for the caret's column, in `renderer/spans.rs` — which is why the caret's column is part of the line cache key in Chapter 16.

THE SPANS — PARSED ONCE, THE SAME IN BOTH STATES



cursor elsewhere — CONCEALED

The **energy** is $E = mc^2$

markers render as empty; the reader sees typography

cursor on the line — REVEALED

The **energy** is \$E = mc^2\$

the exact source, editable in place — no mode switch

What actually changes between the two panels:

nothing in the parse. The renderer reads `display_text` for lines away from the cursor and `text` for the active one.
For code, math and table **blocks** the reveal covers the whole block — every line sharing that `block_id`.

Figure 14.1 — One parse, two projections. Concealment is a rendering decision, not a parsing one. That is what makes it cheap enough to do on every cursor move, and what makes editing a “preview” line behave exactly like editing source.

14.2 Incremental highlighting

A line's highlighting is a pure function of two things: its own text, and the **state it is entered in** — whether a code fence, a math block or a table is open above it.

NATIVE/SRC/EDITOR/HIGHLIGHT.RS

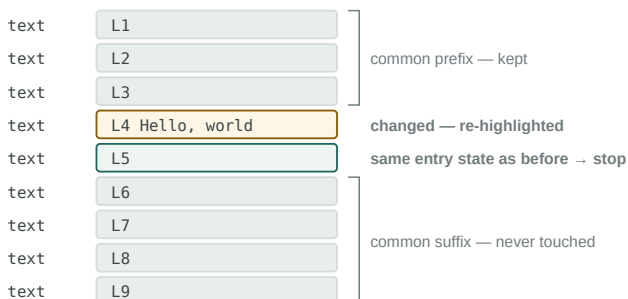
```
pub struct LineState { code: Option<CodeState>, in_math: bool, in_table: bool }
struct CodeState { lang: Option<String>, syntax: Option<(ParseState, HighlightState)> }
```

`Highlighted` keeps, for every line, its text, the `LineState` it was entered in, and how it opens, continues or closes blocks. `update(text)` then does the least work that can possibly be correct:

1. find the common prefix and suffix of the old and new line texts;
2. re-highlight from the first changed line, starting in that line's recorded entry state;
3. stop as soon as it reaches the kept suffix **in the same state** that line was entered in before — from there on nothing can differ — and splice the new lines in;
4. run `decorate` over the whole document: block numbering, math-block concealment and caption numbering, which depend on neighbours rather than on one line.

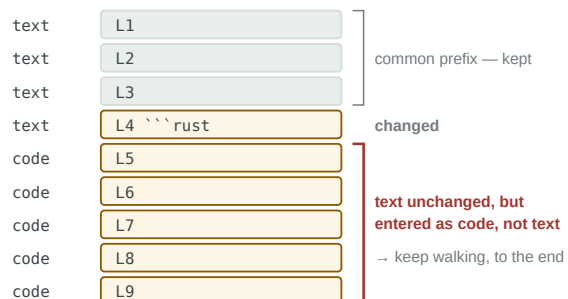
TYPING ONE LETTER ON LINE 4

entry state



OPENING A CODE FENCE ON LINE 4

entry state



The stop test compares `LineState`, not text: a line whose text is unchanged but whose entry state differs must still be re-highlighted.

Figure 14.2 — Where the walk stops. Typing an ordinary character re-highlights one line. Opening a code fence re-highlights until the fence closes, because only then is a line entered in the state it was entered in before.

Highlighting therefore runs **synchronously on every edit** — measured at about 0.03 ms per keystroke on a 700-line note and 0.34 ms at 7,000 lines. There is no debounce, no placeholder lines and no background task, so the painted lines always match the buffer. Each update also takes a new, process-wide unique `revision()`, which names that set of lines; Chapter 16 uses it to skip layout entirely on frames where nothing changed.

WHY IT IS LIKE THIS

An asynchronous highlighter needs a generation id so that a slow parse of old text cannot overwrite newer text — the pattern Chapter 23 describes for PDF renders. But even with the id, the lines on screen lag the buffer for a moment after each keystroke, and a click in that moment is hit-tested against lines that no longer describe the text: it lands on the wrong line. Making the work small enough to do synchronously removes the lag, the id, and that bug together.

NOTE

Two properties are pinned by tests. Fenced code coloured by syntect **preserves the full source text character for character** — a highlighter that quietly drops a character is a data-loss bug wearing a cosmetic disguise. And incremental highlighting **equals highlighting from scratch**, checked through seeded random edit scripts (`HIGHLIGHT_PROPERTY_CASES=N` raises the case count). A new markdown rule must keep both passing.

CHECK YOURSELF

A 500-line note contains no code. You type ````` at the start of line 10. How many lines are re-highlighted? Then you replace line 20 with a closing ````` — how many now? Then you type one ordinary letter on line 30?

Answer: 491, then 481, then 1. Opening the fence changes the entry state of every later line, so the walk never meets a line in its old state and runs to the end. Closing it on line 20 changes the entry state of line 21 onwards back again — every later line differs from a moment ago. The ordinary letter changes no state, so the walk stops at line 31.

The Fenwick height tree

Turning “which line is at this y ?” from $O(N)$ into $O(\log N)$

In a document where every line is the same height, finding the line at a scroll position is a division. In this editor lines are not the same height — a heading is taller, an image line is much taller, a wrapped paragraph is taller still, and a line's height changes when a LaTeX bitmap arrives. So the editor keeps a spatial index.

15.1 The problem

Three questions get asked constantly, on every scroll and every draw:

- How tall is the whole document? (for the scrollbar)
- What is the y offset of line L ? (to draw it)
- Which line is at y ? (for culling, and for every click)

Answering any of them by summing an array of heights is $O(N)$ — and it happens per frame. On a 10,000-line document that is a stall you can feel.

15.2 The structure

NATIVE/SRC/EDITOR/LAYOUT_TREE.RS

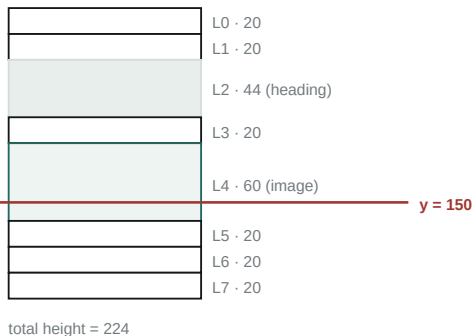
```
pub struct HeightTree { tree: Vec<i64>, heights: Vec<i64> } // fixed point, UNITS_PER_PX = 256

pub fn update_height(&mut self, idx: usize, new_height: f32); //  $O(\log N)$ 
pub fn prefix_sum(&self, idx: usize) -> f32; //  $O(\log N)$ 
pub fn find_line_at_y(&self, y: f32) -> usize; //  $O(\log N)$ , binary lifting
pub fn get_height(&self, idx: usize) -> f32; //  $O(1)$ 
```

A **Fenwick tree** (binary indexed tree) stores partial sums in a flat array. Cell i holds the sum of a range whose length is the lowest set bit of i . That single rule gives you prefix sums and point updates in logarithmic time with no pointers, no rebalancing, and one allocation.

THE DOCUMENT

eight lines, unequal heights



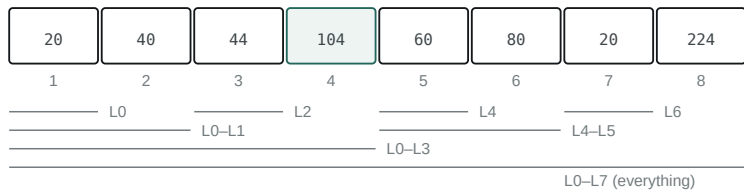
Where each line starts — what `prefix_sum` returns

L0	0
L1	20
L2	40
L3	84
L4	104 — y = 150 lands here
L5	164
L6	184
L7	204

the array is never materialised — these are computed on demand

THE FENWICK ARRAY

cell i holds the sum of the last $\text{lsb}(i)$ heights



Query A · `prefix_sum(6)` — where does line 6 start?

walk down by clearing the lowest set bit: $6 \rightarrow 4 \rightarrow 0$

```
i = 6      sum += tree[6] = 80      → sum = 80
i = 4      sum += tree[4] = 104     → sum = 184
i = 0      stop
```

line 6 begins at $y = 184$ ✓ ($20+20+44+20+60+20$) — two array reads, not six additions

Query B · `find_line_at_y(150)` — which line is under the cursor?

binary lifting: try the biggest power of two first, take the step if the sum still fits

```
step 8      tree[8] = 224          0 + 224 > 150          ✗ skip
step 4      tree[4] = 104          0 + 104 ≤ 150          ✓ take
step 2      tree[6] = 80           104 + 80 > 150          ✗ skip
step 1      tree[5] = 60           104 + 60 > 150          ✗ skip
```

answer: line 4, which spans y 104–164 — and 150 is inside it ✓

four probes for eight lines; seventeen for a 100,000-line document

Figure 15.1 — The same eight lines, two queries. Query A walks *down* the tree by clearing the lowest set bit; query B walks *across* it by trying powers of two from the top. Both touch $O(\log N)$ cells. The highlighted cell 4 is the one both queries happen to need — it covers lines 0–3 because 4’s lowest set bit is 4.

15.3 What it buys

OPERATION	BRUTE-FORCE SCAN	HEIGHTTREE
Total document height	$O(N)$	$O(\log N)$ — one <code>prefix_sum(n)</code>
Y offset of line L	$O(L)$	$O(\log N)$
Line at a Y coordinate	$O(N)$	$O(\log N)$, binary lifting on prefix sums
Update one line's height	$O(1)$	$O(\log N)$

Note the one row where the tree is *worse*: updating a single height. That is the trade, and it is obviously worth it — updates happen when a line is remeasured, queries happen several times per frame.

Three implementation details are worth copying:

- **Heights are stored as fixed-point integers** — `UNITS_PER_PX = 256`, so every sum is exact however many lines are added and removed. The public API still speaks `f32` pixels; only the storage is integral.
- `update_height` **short-circuits when the delta is zero**, so remeasuring a line to the same height costs nothing at all. Most remeasures produce the identical number, so this removes most of the update traffic — and in integer units “identical” is exact.
- Each stored height is a line's **whole extent**, not its text height: its collapsed top margin, a caption band on the first line of a code block or table (`BLOCK_CAPTION_HEIGHT = 24`), its body,

and a scrollbar gutter under a table's last row (`HORIZONTAL_SCROLLBAR_GUTTER = 16`). The tree's unit is “vertical space this line occupies”, not “font metrics”.

TRAP — AND THE BUG THAT MADE IT ONE

An earlier version of the tree stored `f32` heights. Floating-point addition is not exact, and a Fenwick cell is adjusted by every update beneath it, so over a long editing session rounding error accumulated in the cells until `prefix_sum` no longer agreed with the heights it was built from — and hit testing picked the wrong line. A property test found it. Integers do not drift.

TRY IT

Open `native/src/editor/layout_tree.rs` and read the tests at the bottom. They assert two things: that `prefix_sum` agrees **exactly** with a brute-force sum for random height vectors, and that `find_line_at_y` is monotonic and correct at boundaries. That is exactly the right pair of properties for a structure like this — an oracle test and an edge test — and it is a good template if you ever add a similar index. The renderer's property suite adds a third one level up: `height_tree_agrees_with_line_visual_y` checks the tree against an uncached model of the whole layout (Chapter 30).

CHECK YOURSELF

An image finishes loading and line 4 grows from 60 to 300 pixels. What has to happen?

Answer: `update_height(4, 300)`, which adds the delta of 240 to $O(\log N)$ cells — cells 5, 6 and 8 in our example. Every subsequent `prefix_sum` and `find_line_at_y` is then correct, with no rebuild. The *separate* question of how the editor knew to remeasure that line at all is Chapter 16.

Measure once: the line cache and the layout key

Text shaping is the expensive part, so the answer is remembered — and when nothing changed, not even asked for

Measuring a line means shaping glyphs: resolving the font, applying metrics, and breaking rows at the current width. It is the dominant cost of layout. The editor avoids it at two levels: `LineHeightCache` remembers each line's body height, and `LayoutKey` skips the layout pass altogether on a frame whose inputs have not changed.

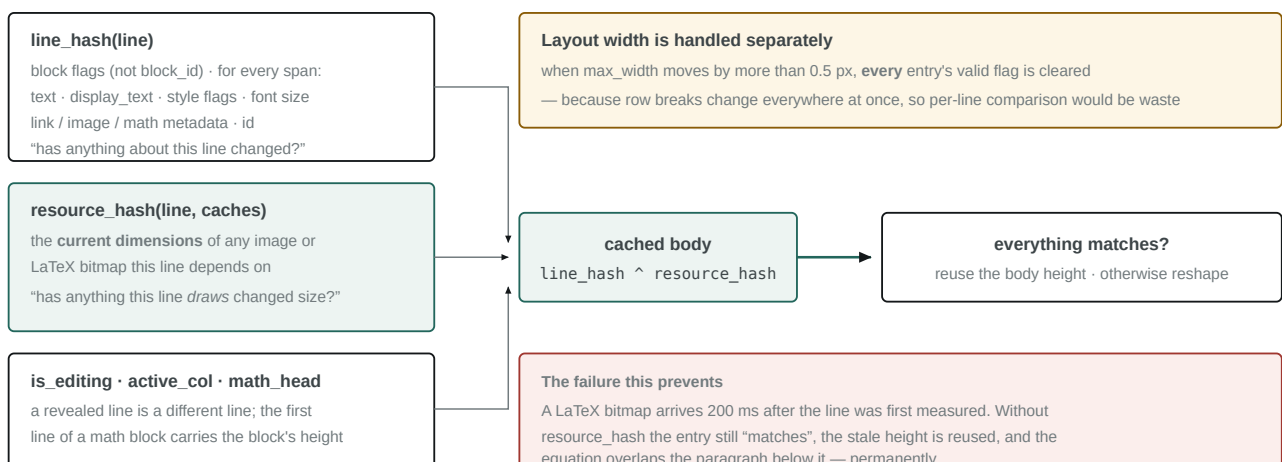
16.1 The line cache

NATIVE/SRC/EDITOR/LAYOUT_CACHE.RS

```
pub struct LineHeightCache {
    pub hash: u64,                // line_hash ^ resource_hash
    pub is_editing: bool,
    pub active_col: Option<usize>,
    pub math_head: bool,         // whether the line carries its math block's height
    pub height: f32,             // the body alone
    pub valid: bool,
}
```

The cache holds a line's **body** — its rows of text, its image, its equation — and nothing that depends on its neighbours. Margins do (Chapter 17), so they are never cached: a cheap walk recomputes them on every layout. A cached body is reused only when `valid`, the combined hash, `is_editing`, `active_col` and `math_head` **all** match. Miss any one and the line is remeasured.

WHAT GOES INTO THE KEY



Margins are **not** in the entry: they depend on the neighbouring lines, so a cheap walk recomputes them on every layout.
The hash is a `DefaultHasher`, chosen for speed — a cache key, not a cryptographic digest.

Figure 16.1 — What a cached body depends on, plus a global reset for width. The `resource_hash` half is the non-obvious one: it exists because asynchronous media changes a line's height *after* the line itself has stopped changing.

WHY BLOCK_ID IS NOT IN THE HASH

`block_id` numbers blocks from the top of the document, so inserting a line above a block renumbers it — and every block below — without changing anything a line's own height depends on. With `block_id` in the hash, pressing `Enter` near the top of a long note would remeasure every block beneath it.

16.2 The layout key: skipping layout entirely

The cache makes a layout walk cheap, but a walk over 37,000 lines is still 37,000 lookups — and Iced rebuilds the view on every scroll and animation frame. So the widget remembers what its last layout was computed from:

NATIVE/SRC/EDITOR/RENDERER/MOD.RS

```
struct LayoutKey {
    revision: u64,           // the app's revision of the lines, images and equations
    lines: usize,
    width: u32,             // bits of the width laid out at
    focused: bool,
    cursor: (usize, usize),
}
```

The revision is a promise from the app. `EditorPane::layout_revision()` hashes `Highlighted::revision` (Chapter 14), the number of images and equations in its caches, and the display scale, and hands the result to the widget through `Editor::layout_revision(u64)`. Images and equations are only ever added, or all dropped when the scale changes, so those counts identify them. While the revision, the width, focus and the caret are all the same as last time, **layout is skipped entirely**.

FRAME · 37,009-LINE DOCUMENT · RELEASE BUILD	COST
Cold layout	15.5 ms
Full walk, every body cached	15.4 ms
Nothing changed — layout skipped	47 ns
Drawing one screen	0.25 ms

Read the table from the bottom. The frames that matter most — scrolling and animation, which change nothing a layout depends on — cost 47 nanoseconds instead of fifteen milliseconds, which at 60 fps would have been almost the entire frame budget. The numbers come from an ignored test you can run yourself (Chapter 30).

One related detail: `Widget::size()` reports the editor's height as `Shrink`. The real height is only known once the document is laid out at the width the widget actually gets, so nothing ever measures the document at a guessed width.

TRAP

If you add state that affects a line's height, it has to reach both levels. Add it to `line_hash` or `resource_hash`, *and* make sure changing it changes `EditorPane::layout_revision` or `LayoutKey`. Miss the first and a stale height is reused; miss the second and the widget never even looks — it paints the previous layout. Neither failure produces an error message; both show up as “the layout is subtly wrong sometimes”.

CHECK YOURSELF

A LaTeX bitmap for line 900 arrives while you are scrolling. Which of the two levels notices, and how?

Answer: both, in order. The math cache on `EditorPane` gains an entry, so `layout_revision` changes and the `LayoutKey` no longer matches — the widget walks the lines again. During that walk line 900's `resource_hash` now includes the bitmap's dimensions, so its cache entry misses and that one body is remeasured; every other line is a hit. The next scroll frame matches the new key and skips layout again.

The widget: rhythm, rows, and the draw pass

One place computes each fact, everything else asks — and the draw pass only asks about what is visible

`Editor` implements `iced::advanced::Widget` — not `canvas::Program`, because it needs its own layout, hit testing and event handling, which the canvas abstraction does not provide. Content is capped at `MAX_CONTENT_WIDTH = 880.0` so prose keeps a readable measure in a wide window.

The widget used to be one 4,400-line `renderer.rs`. It is now `native/src/editor/renderer/`, one module per concern, and it rests on four principles worth reading before any of its code:

1. **One source of truth per fact.** Row breaks, baselines, margins, scroll extents and the caret's position are each computed in exactly one place; everything else asks.
2. **Layout is a pure function** of the lines, the width, the caret and the loaded resources. Every cache is checked against a cold recomputation by a property test.
3. **Invariants before code.** Behaviour is pinned by seeded property suites (Chapter 30) as well as by examples.
4. **Exact over estimated.** Positions come from the shaper and the font's own metrics; motion comes from closed-form springs (Chapter 26), not frame-by-frame integration.

17.1 The module map

TO CHANGE...	LOOK IN RENDERER/
how tall a line's body is	<code>layout.rs</code> — <code>line_body_height()</code>
space between lines and blocks	<code>layout.rs</code> — <code>requested_margins()</code> , <code>MarginWalk</code> ; values in <code>metrics.rs</code>
where rows break, row height, baselines	<code>flow.rs</code>
any size two passes must agree on	<code>metrics.rs</code>
when a span reveals its markdown source	<code>spans.rs</code>
where the caret lands, or what a click selects	<code>flow.rs</code> (inline lines), <code>caret.rs</code> (code lines, up/down)
how the caret moves and blinks	<code>glide.rs</code> — Chapter 26
how the page scrolls to the caret	<code>reveal.rs</code> (where) and <code>app.rs::reveal_caret()</code> (how)
block cards, captions, code lines, table rows	<code>draw/blocks.rs</code> , <code>draw/captions.rs</code>
paragraph text, images, equations, checkboxes	<code>draw/inline.rs</code>
the selection shape, search highlights, the caret's look	<code>draw/overlays.rs</code>
a key binding inside the editor	<code>events.rs</code>
horizontal scrolling of code, tables or math	<code>scroll.rs</code> — <code>scroll_extent()</code>

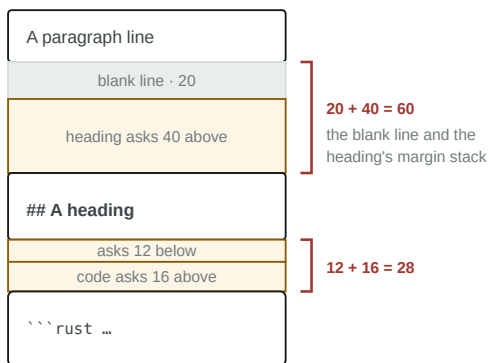
`mod.rs` holds `Editor`, its `State` and `LayoutKey`, and a thin `Widget` impl whose methods only delegate to the modules above. `measure.rs` does the shaping and reads font metrics, `selection.rs` extracts the selected text, and `testing.rs`, `tests.rs` and `properties.rs` belong to Chapter 30.

17.2 Typographic rhythm

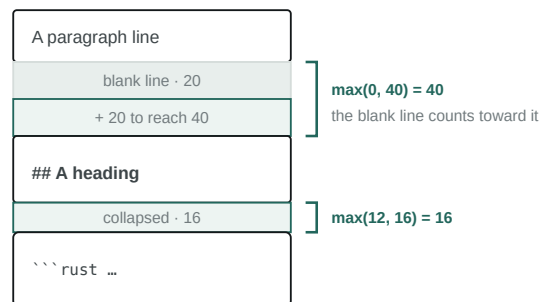
Every vertical measure in a document is a multiple of `GRID = 4px`, so baselines in different blocks share one grid. The numbers live in `renderer/metrics.rs`:

- **Row height** — `row_height(font_size)` interpolates line height from 1.6 at the 17px body size down to 1.2 at 34px, because large type needs less leading, and snaps to the grid. A body row is 28px; code rows are `row_height(15)`.
- **Blank lines** are paragraph gaps, `PARAGRAPH_GAP = 20px` — still tall enough to hold the caret.
- **Margins** — each line asks for space above and below by kind: headings from `HEADING_MARGINS` (48/16px for `#` down to 24/8px), and code blocks, tables, block math, images, rules and quote runs `BLOCK_MARGIN = 16px`.

SUMMED — WHAT THE EDITOR DOES NOT DO



COLLAPSED — WHAT IT DOES



The space between two lines is the larger of what they ask for, never the sum — and the first line of a document gets no top margin at all.

Figure 17.1 — Margins collapse. Summing margins is the obvious implementation, and it makes every heading that follows a blank line float away from its own section. Collapsing them is what lets a writer leave blank lines wherever they like without changing the page.

The whole collapsed margin is realised above the lower line, so a block's box starts exactly at its content, and the painters read margins from `State::line_margins` rather than deriving them again.

17.3 Inline layout: the Flow

Every line that is not a code line, a rendered table row or rendered block math is laid out by `Flow::build` in `renderer/flow.rs`, and **that is the only place rows are broken**. Line heights, painting, the caret, selection and search highlights, and click and hover hit testing all read the same `Flow` — so the caret always sits on the painted text, and a click lands where it is drawn.

SOURCE - Read **the flow** before painting anything

THE FLOW — ROWS, AND ITEMS IN SOURCE ORDER



amber ticks — concealed ****** markers: items with columns, zero width
dashed sage — hanging indent: row 1 starts under the text, not the bullet
dashed grey — one baseline per row, from the font's own metrics

layout.rs — the body height

row heights, summed once, for the height tree

draw/inline.rs — painting

each item at its x, on its row's baseline

caret(col, affinity)

the caret, the selection shape, search highlights

col_at(x, y) → (col, affinity)

clicks, drags and hover

Built once by `Flow::build`, read by everything. Nothing on the right wraps text itself — which is why the caret sits on the painted glyphs (invariant I1) and a click lands exactly where the caret is drawn (invariant I2).

Figure 17.2 — One line, one `Flow`, four readers. An item records its x, its row and the source columns it covers. Concealed markers are real items with no width, which is how a column inside ****** still has a place to put the caret when the line is revealed.

- **Shaping** — prose is shaped with `Shaping::Advanced` (kerning, ligatures); monospace keeps `Shaping::Basic` so columns line up as typed. Each word is shaped once per font and size, and the x of every character boundary inside it comes from the shaper's own `grapheme_position` (`measure.rs::word_offsets`), so the caret sits exactly between kerned glyphs and never inside a cluster.
- **Breaking** follows Unicode line-break opportunities (UAX #14, `unicode-linebreak`): between words, after hyphens and dashes, inside long URLs. Trailing whitespace hangs past the right edge; a word wider than a whole row breaks between grapheme clusters.
- **Hanging indents** — wrapped rows of a list item, task or numbered item start under its text, not under its marker (the `is_list_marker` flag from Chapter 14).
- **Balanced headings** — a wrapped heading nobody is editing is re-laid at the narrowest width that keeps the same number of rows (bisection, 14 steps), so its rows come out even instead of ragged. A heading being edited keeps greedy breaks, so text does not jump between rows while you type.
- **Row height and baselines** — a row is as tall as the larger of `row_height` for its largest text and its content (plus `INLINE_MATH_ROW_PADDING` with an equation on it), snapped up to the grid. `measure.rs::font_metrics` reads ascent, descent and x-height from the face the shaper uses; every item on a row shares its baseline, and checkboxes and inline equations centre on the **axis**, half an x-height above it. Painted text snaps to device pixels.
- **Columns ≠ x** — `caret(col, affinity)` places a column; `col_at(x, y)` returns a column *and* an affinity. Past the end of a row that wraps it answers `Upstream`, so the caret stays on the clicked row (Figure 13.2).

TRAP

Never measure or break inline text outside `flow.rs`. A second wrapper — in a painter, a hit tester, a highlight — will agree with the first on most lines and disagree on a few: a kerned pair, a hyphen, a checkbox. Where they disagree the caret is drawn beside the text instead of in it, or a click selects the character next to the one under the pointer. Laying inline text out once is how the caret, clicks and painting were made to agree. If you need to know where text is, ask the line's `Flow`.

17.4 Two passes with different budgets

Layout — `layout_lines` in `renderer/layout.rs` — runs only when the `LayoutKey` changed (Chapter 16). It walks all lines, taking each body from `LineHeightCache` unless something about the line changed, recomputing margins, and filling the height tree and `block_ranges`. Total height is `TOP_PAD + prefix_sum(n) + BOTTOM_PAD`, 24 and 80 px.

Draw — `renderer/draw/` — must never walk the document. It walks the viewport:

1. `find_line_at_y(viewport.y - bounds.y - TOP_PAD)` gives `visible_start`; the same call at the viewport's bottom edge gives `visible_end`;
2. `measure_blocks()` measures only blocks with a line in that range, and their chrome — cards, captions, language badges — is painted first;
3. each visible line is laid out **once** into a `LineBox`, its `Flow` included, and every step below reads those boxes;
4. the selection's pieces over text are painted under everything;
5. each line gets its search highlights, then one painter chosen by kind — rule, code line, table row, block math, or flow — then the caret;
6. the selection's pieces over tables, images and equations are painted in a layer of their own, and block scrollbars last.



Figure 17.3 — Culling. The draw pass never learns how long the document is. This is the rule stated in the contributor guidelines as “full-document work in the draw pass is a performance regression”, and it is the single easiest thing to break by accident when adding a new block type.

17.5 The selection, and why it needs layers

A selection is painted as **one continuous shape**: a full-height band per visual row it touches, bridged across the space between lines wherever neighbouring bands overlap, with corners rounded only where they stick out. A selection that continues past the end of a line gets an 8 px tail there, standing for the selected line break, so selected blank lines stay visible.

Tables, images and equations are selected whole, and their bands are painted **over** them — an opaque table header or picture would hide a band beneath it — while bands over text go

under it, where a translucent band on top would dim the glyphs. Doing that correctly depends on how renderers really paint.

TRAP — LAYERS, NOT CLIP RECTANGLES

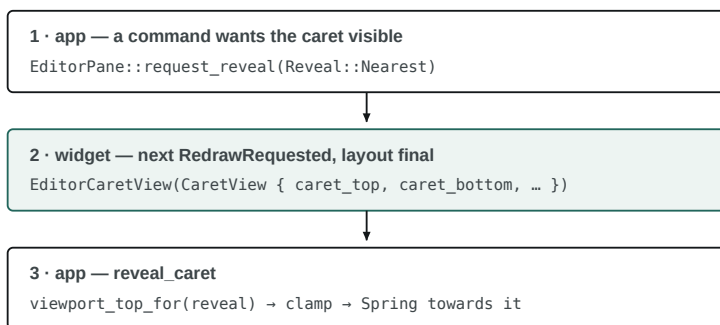
A renderer paints each layer's **quads, then its images, then its text**, whatever order they were drawn in. Anything that must appear over an image or text needs a *later layer*, not a later call.

Clipping has the same shape. Clip with a layer (`renderer.with_layer`), never with the clip rectangle passed to `fill_text` or `draw_image`: no iced backend clips an image to it, and a text clip rectangle *inside* the enclosing layer switches tiny-skia's clipping off. So wide code, tables and equations are painted inside a layer bounded by the block's viewport (`draw/blocks.rs::block_clip`), with their text drawn using the frame's viewport as its clip rectangle. The test recorder models both rules, so `Call::paint_order` and each call's `clip` show what a renderer really puts on screen.

17.6 Revealing the caret

Keeping the caret in view is a two-party problem. The widget knows exactly where the caret is drawn but cannot scroll the scrollable it sits in; the app can scroll, but cannot see where the caret is drawn. They meet halfway, and nothing is estimated — there is no per-keystroke centring and no guess at the editor's width.

THE HANDSHAKE



THE VIEWPORT · Reveal::Nearest

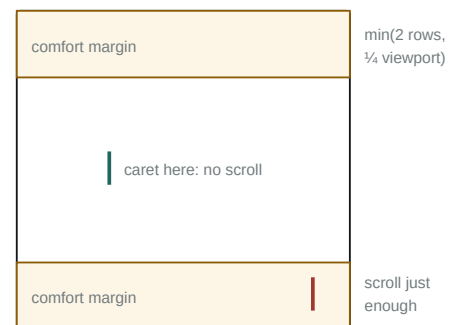


Figure 17.4 — Where the caret is, answered by the one party that knows. `Reveal::Center` skips the comfort test and puts the caret mid-view. A target more than two viewports away cuts to one viewport short and glides the rest.

1. A command that should keep the caret visible calls `EditorPane::request_reveal` — `Reveal::Nearest` for typing and caret movement, `Reveal::Center` for search navigation, table-of-contents clicks and heading links. This bumps `reveal_request`; while a request is unanswered the stronger mode wins, so a jump that centres is not downgraded by the cursor move made on the way.
2. On the next `RedrawRequested` — layout final, viewport known — the widget answers with a `CaretView`: the caret's top and bottom, the viewport's top and height, and its own height, all relative to the widget.
3. `app.rs::reveal_caret` asks `CaretView::viewport_top_for`. `Nearest` scrolls only if the caret is within the comfort margin of an edge, and then only by the amount that restores the margin; `Center` puts it mid-view.
4. The target is clamped to the scrollable range and reached with a `Spring` (`SCROLL_STIFFNESS`, Chapter 26), retargeted with its momentum if typing continues mid-glide. A wheel turn or

click fires `EditorScrollInterrupted` and hands the page back (Chapter 12).

17.7 Three more things the widget does

Inline and display LaTeX

`$...$` and `$$...$$` are rendered by `ratex-render` into bitmaps, cached on `EditorPane` by TeX source for the current display scale — flushed and re-rendered when the scale changes — and delivered back through `Message::MathRendered`. Display math is centred with its own padding; inline math centres on the row's axis. Because the render is asynchronous, its arrival changes the layout revision and `resource_hash`, and the affected line is remeasured — the loop closed in Chapter 16.

Wide blocks scroll independently

Tables, code blocks and wide equations neither wrap awkwardly nor stretch the layout. Each block keeps **its own horizontal scroll offset**, clamped to its own content width —

`scroll.rs::scroll_extent` is the one answer to how wide a block is and how much of it shows — so a wide table scrolls sideways under the mouse wheel while vertical document scrolling is unaffected. A code block scrolls by the least amount that keeps the caret `CODE_CARET_MARGIN` (24 px) inside its viewport, and an equation's number sits outside the equation's viewport, so a scrolled equation never runs under it.

Hit testing and visual movement

- Click and drag y positions map to line indices through `HeightTree`, then to a column and affinity through the line's `Flow`. A drag keeps tracking the pointer outside the widget.
- Horizontal wheel and scrollbar interaction resolve the target block via `block_ranges` plus prefix sums.
- Visual up/down moves **exactly one visual row**, starting from the caret's own row — its affinity included — and keeps a `desired_visual_x` across consecutive presses, so the caret does not drift left while moving through a run of short lines.

17.8 The maintenance contract

WHY IT IS LIKE THIS

These rules are stated in the wiki and repeated in the contributor guidelines, and they are really one rule seen from several angles: *each fact has one owner, and the pieces only work together*.

- Keep parsing in `highlight.rs`; do not add markdown rules to the renderer. A new rule must keep incremental highlighting equal to a fresh pass.
- Never measure or break inline text outside `flow.rs`.
- Caret facts — position, selection, affinity — live in `DocBuffer`. The widget keeps no copy.
- Keep height and invalidation logic in `layout_tree.rs`, `layout_cache.rs` and `renderer/layout.rs`.
- Put any size that layout, painting and hit testing must agree on in `renderer/metrics.rs`, never as a literal at one call site, and keep vertical sizes on the `GRID`.
- Keep the draw pass proportional to visible content, never to document length.
- Clip with layers, and remember that a layer paints its quads, then images, then text.
- When adding media that can affect height, include its dimensions in `resource_hash` and make sure a change to it changes `EditorPane::layout_revision`.
- When adding a block type, update block-range tracking, body height, margins, draw metadata, and hit testing and scrolling **together** — one contract split across five call sites.

CHECK YOURSELF

You add a “callout” block type that draws a tinted background and a label. Which places change?

Answer: the parser, then the five call sites. First `highlight.rs`, to parse it and assign a `block_id` — keeping incremental-equals-fresh passing. Then (1) `layout.rs::record_block_range`; (2) `layout.rs::line_body_height`; (3) `layout.rs::requested_margins`, probably with `BLOCK_MARGIN`; (4) `draw/blocks.rs::measure_blocks`, for the card, clamped to the visible range; and (5) `scroll.rs::scroll_extent`, so clicks and scrolling inside the callout resolve to the right line and column. Miss the last and clicks land in the wrong place — exactly the kind of bug that gets reported as “the editor feels weird”. Finish with a behaviour test in `renderer/tests.rs`, seen failing first.

The PDF subsystem

A single-threaded C library, a file format with an upside-down coordinate system, and documents that reference their own equations without linking to them. This is the most constrained part of the codebase, and the part with the most rules.

1. One thread, two queues
2. Two coordinate systems
3. Finding a table of contents that isn't there
4. Resolving “see equation (3.14)”
5. Sidecar annotations and document identity
6. Generations, pending sets, and the stranded page
7. Stable page slots
8. From highlight to note and back

One thread, two queues

Scheduling around a library that cannot be parallelised or interrupted

`pdfium-render` stores its bindings in a process-global cell. The application must never create a second `Pdfium` binding, and PDFium handles must not be shared across threads. Everything in this chapter follows from those two sentences.

CORE/SRC/PDF.RS

```
pub struct PdfRenderer {
    sender: Sender<PdfCommand>,           // standard work
    priority_sender: Sender<PriorityRender>, // the page the reader is looking at
    visible_range: Arc<Mutex<Option<(u16, u16, String)>>>,
}
```

`PdfRenderer::new()` spawns **one** worker thread that owns the binding and one cached open document. Every PDF operation in the process is serialized onto it. Since parallelism is off the table, the only lever left is *ordering* — and the design spends it entirely on one thing: the page the reader is looking at right now.

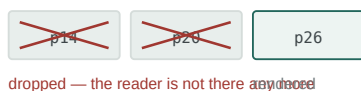
18.1 The scheduling rules

1. Receive a standard command; drain any others already available into `pending_commands`.
2. **Before each standard command, drain the priority channel.** If several priority requests are queued, only the newest is rendered — the reader has already scrolled past the others.
3. Process exactly one standard command, then repeat.

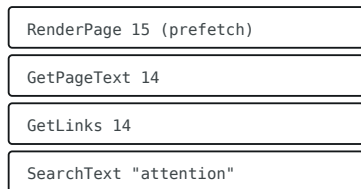
`PdfCommand::Wake` exists solely to unblock `receiver.recv()` after a priority request is sent; it performs no work of its own.

THE READER SCROLLS FAST: 3 → 14 → 20 → 26

priority channel
latest request wins



standard channel
everything else, in order



THE WORKER LOOP, IN ORDER



What this guarantees: the target page never queues behind a backlog of prefetches, link extractions and searches.

What it does not: interrupt work already running. A slow command still delays the next page by its full duration.

Figure 18.1 — Latest request wins. Priority does not mean pre-emption; it means the queue is drained newest-first before every standard command, and stale requests are discarded rather than rendered. That is why fast scrolling produces one render instead of twelve.

18.2 The command surface

Standard channel: `PageCount`, `ExtractText`, `PageSizes`, `RenderPage`, `GetToc`, `GetReferences`, `GetEmbeddedToc`, `GetLinks`, `SearchText`, `RenderLinkPreview`, `GetPageText`, `Wake`.

Priority channel: `PriorityRender { path, page_index, scale, resp }`.

Every command carries a `SyncSender` for its reply, so the async caller on the Tokio side blocks on a channel rather than on `PDFium`. That is the seam that keeps the FFI on one thread while still letting the rest of the app `await` results normally.

TRAP

Because a command cannot be interrupted, **the duration of the slowest command is the worst-case latency of every page render**. This is not theoretical: extracting the text of a whole textbook in one `ExtractText` takes hundreds of milliseconds, and while it runs the reader is looking at a blank page. Chapter 21 shows the fix — chunk the work into per-page commands so priority renders interleave.

Two coordinate systems

PDF measures from the bottom; screens measure from the top

Half the confusing bugs in a PDF viewer are one of these two conventions applied in the other one's space. The codebase's answer is not “convert everything” but “know which space each thing lives in, and convert at exactly one place”.

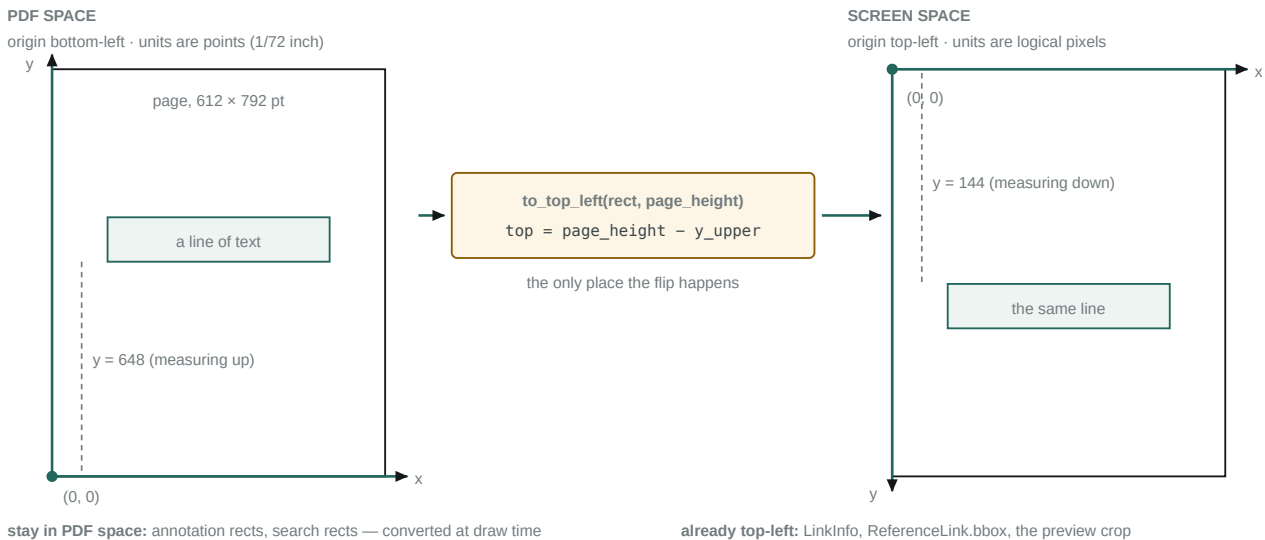


Figure 19.1 — One flip, in one function. Character bounding boxes arrive from PDFium in bottom-left origin; `to_top_left(rect, page_height)` is the single conversion, so hit testing has one convention to reason about.

19.1 Which space, and why it matters

- **Annotation and search rectangles stay in bottom-left PDF points**, and are converted at draw time by `search_rect_to_view_rect`.
- **Reference underlines are already top-left** and convert with a plain `* zoom`.
- **Page text geometry** is stored in PDF space, so it survives zoom changes.

WHY IT IS LIKE THIS

Rectangles are stored in *PDF points* rather than pixels, and that one decision is why a highlight you made last month at 100% zoom on a 1× display still lands on the right words today at 175% zoom on a HiDPI screen in a narrow split pane. Store a pixel rectangle and you have stored an accident of the window you happened to have open.

Finding a table of contents that isn't there

Four sources, in order of trustworthiness, and two guards against inventing one

Many PDFs — especially preprints and scanned books — carry no embedded bookmarks at all, and a reader without an outline is a reader who scrolls.

`get_toc(path)` returns `(Vec<TocEntry>, synthetic)` and works down a cascade of increasingly speculative sources.

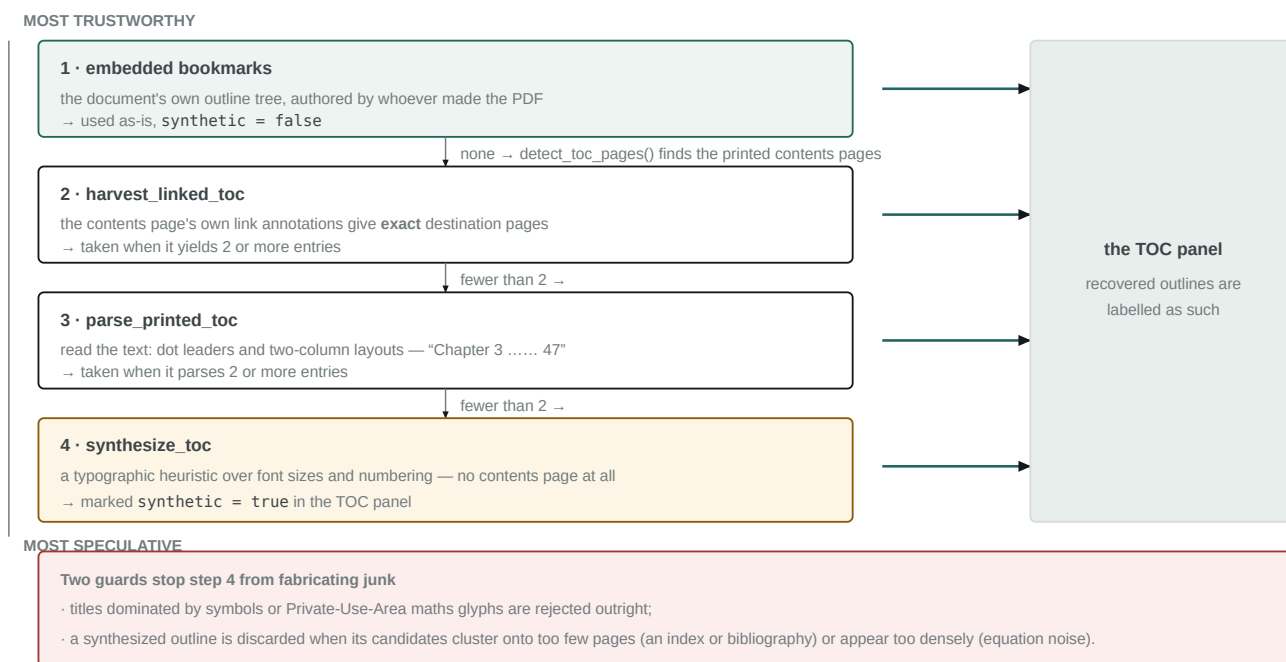


Figure 20.1 — Falling back, never fabricating. The cascade only descends when the level above produces too little, and the last level is allowed to produce *nothing*. A scanned or structureless PDF correctly yields an empty outline rather than a dump of page content — an empty panel is honest, a wrong outline is worse than none.

Two entry points exist: `recover_toc(doc, &pages)` runs the whole chain, and `recover_toc_from_texts(&pages)` runs the text-only subset with no document handle — which is what the reference resolver uses (Chapter 21), so it does not have to scan the document twice.

Resolving “see equation (3.14)”

A pure resolver, one hard precision rule, and the chunking that keeps it from blocking renders

Papers reference their own equations, figures, tables and sections by number all the time, and almost never carry link annotations for those mentions. `core/src/references.rs` reads the extracted text and turns the mentions into clickable targets.

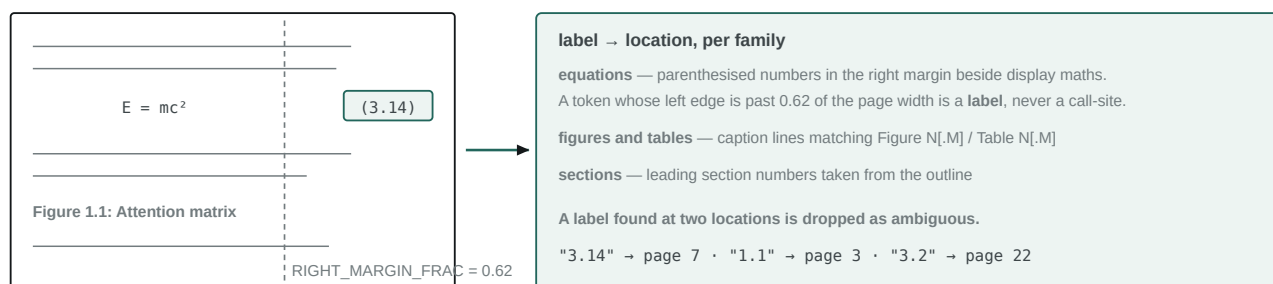
CORE/SRC/REFERENCES.RS

```
pub fn resolve_references(pages: &[PdfPageText], toc: &[TocEntry]) -> Vec<ReferenceLink>
```

The resolver is **pure**: it reads already-extracted page text plus the outline, touches no PDFium, and is therefore `Send`, cheap (roughly 15–30 ms for a full textbook) and unit-testable without a window. That signature is a design statement — the hard part of this feature is a text problem, not a rendering one, so it is kept where it can be tested.

21.1 Two passes and one gate

PASS 1 — BUILD THE TARGET MAP



PASS 2 — SCAN CALL-SITES, THEN THE GATE

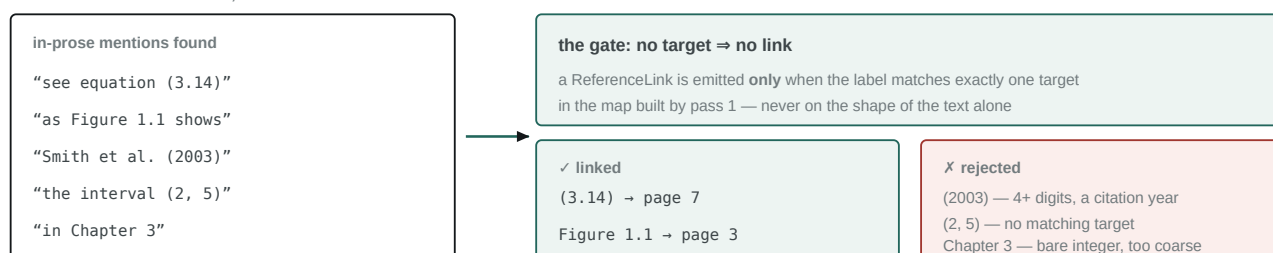


Figure 21.1 — Precision by construction. The gate is the whole design: a number only becomes a link if the document itself defines that number somewhere. Stray numbers — intervals, quantities, years — cannot become bogus links, because there is nothing for them to point at.

Three precision rules were learned from a real corpus and are worth stating explicitly, because each one is a bug that was observed before it was fixed:

- **Reject bare four-or-more-digit parenthesised numbers.** (2003) is a citation year, not equation 2003.
- **Capture dotted figure and table numbers** — otherwise Figure 1.1 and Figure 1.2 collapse into one target called "1" and both become ambiguous.

- **Require a dotted section number.** A bare chapter integer is too coarse to aim at.

21.2 Chunked scanning

References need the whole text layer. PDFium is single-threaded, and priority renders only drain *between* worker commands (Chapter 18) — so one monolithic full-document command would block page rendering for half a second or more. The native side therefore resolves references as a chunked background task:

1. `get_embedded_toc(path)` for section targets — cheap, no text scan;
2. extract text **one page at a time** with `get_page_text(path, i)`, so queued priority page renders interleave between pages;
3. if there were no embedded bookmarks, recover the outline from the text just collected with `recover_toc_from_texts(&pages)` — no second scan;
4. run `resolve_references(&pages, &toc)` on the async thread.

For bookmark-less PDFs the full scan already happens for TOC recovery, so references are effectively free. For bookmarked PDFs the chunking is what keeps first-open rendering responsive.

21.3 Caching, versioning, and aiming the preview

Results are cached in `pdf_references` keyed by `document_id`, stored as `(RESOLVER_VERSION, links)`. `get_pdf_references` discards entries whose version does not match the current resolver, so changing the algorithm invalidates stale results rather than serving them.

TRAP

Bump `references::RESOLVER_VERSION` whenever the resolver's output for the same input would change. It is currently 2. Forget it and every existing user keeps the old — now wrong — cached links for ever, while your machine, which has a fresh cache, looks perfect.

One more detail, and it is a nice example of caring about the actual reading experience: a caption sits *beside* its artwork, not on it. So `scan_caption_callsites` nudges the preview target off the caption — figures up (caption-below convention), tables down (caption-above) — by `CAPTION_PREVIEW_SHIFT = 110.0` points, about a third of the 300 pt preview window, so the figure fills the frame instead of being clipped.

Recognised references are drawn with a subtle accent **underline**. Those rects set `OverlayRect.thin`, which bypasses the minimum-size clamp that keeps highlight rectangles legibly tall, letting the underline render as a true hairline.

TRY IT

Two examples exist for working on this code:

```
cargo run --example dump_refs -- path/to/paper.pdf # resolved references with  
surrounding text  
cargo run --example bench_refs -- path/to/paper.pdf # text-scan cost per document
```

`dump_refs` is the precision spot-check: run it on a paper you know and read the surrounding text of every link it claims. A single wrong link is worth more attention than ten missing ones, because a wrong link makes the reader stop trusting the underlines entirely.

Sidecar annotations and document identity

What a highlight is attached to, when it survives, and how you find out it drifted

Highlighting text never modifies the `.pdf` on disk. That raises an immediate question: if the annotation is not *in* the file, what is it attached to?

22.1 The identity function

CORE/SRC/PDF.RS

```
pub fn compute_provisional_id(path: &Path) -> Result<(String, u64, Option<i64>), String>
```

The `document_id` is the SHA-256 of **the first 1 MiB of the file plus its length**, in hex. Two deliberate choices are buried in that sentence:

- **A bounded prefix, not the whole file.** Hashing 1 MiB keeps opening a 500-page PDF cheap. Adding the length makes truncation or extension detectable without reading the rest.
- **mtime is excluded on purpose.** Including it would orphan every annotation whenever the file was copied, restored from a backup, or re-downloaded. The mtime is still returned and recorded in `pdf_documents` for bookkeeping — it just is not part of identity.

THREE THINGS THAT CAN HAPPEN TO A PDF IN YOUR VAULT

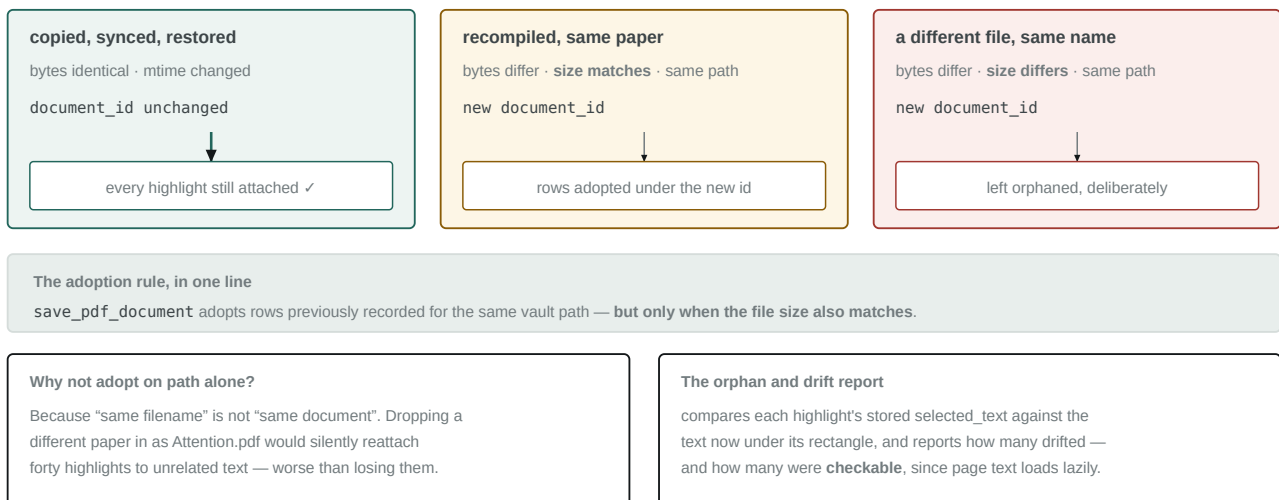


Figure 22.1 — Survive a copy, refuse a substitution. The size check in the adoption rule is the line between “helpful” and “dangerous”. Note the honesty in the report: it states how many annotations it could actually check, rather than implying it checked them all.

22.2 The annotation record

CORE/SRC/PDF.RS

```
pub struct PdfAnnotation {
  pub id: String,
  pub document_id: String,
  pub page_index: u16,
  pub kind: PdfAnnotationKind,      // Highlight | Note
  pub color: PdfAnnotationColor,    // Yellow | Green | Blue | Pink | Orange
  pub selected_text: String,
  pub ranges: Vec<PdfTextRange>,    // character intervals on the page text layer
  pub rects: Vec<PdfRect>,          // bounding boxes in PDF points
  pub note: Option<String>,
  pub linked_note_path: Option<String>,
  pub markdown_anchor: Option<String>,
  pub created_at: i64,
  pub updated_at: i64,
}
```

Two fields are stored where one might do. `ranges` are character intervals on the text layer — the semantic anchor, used by the drift report. `rects` are geometry in PDF points — what gets drawn. Keeping both is what lets the app detect that a highlight has moved rather than silently drawing it in the wrong place.

Colour cycling. `PdfAnnotationColor::next()` advances Yellow → Green → Blue → Pink → Orange → Yellow, and `PdfPane::next_highlight_color` steps through it after each quick highlight, so successive highlights stay visually distinct without the reader choosing a colour every time. The explicit `Message::PdfCreateHighlight(color)` path lets you pick one deliberately.

22.3 Text search, and the whitespace problem

CORE/SRC/PDF.RS

```
pub fn search_text(&self, path: &str, query: &str, regex: bool, match_case: bool)
  -> Result<Vec<PdfSearchMatch>, String>
```

The worker walks each page's text layer, builds the page string alongside a `char_indices` map back into PDFium's own character indices, finds matches (a compiled `regex` when `regex` is set, otherwise a literal), and converts each match into `PdfRect`s via `text_page.segments_subset`.

Loose whitespace mode is a native-side convenience built on the same regex path (`app.rs::search_pdf`):

```
let pattern = query.split_whitespace()
  .map(regex::escape)
  .collect::<Vec<_>>()
  .join(r"\s+");
```

Three lines that make PDF search actually usable on papers: a phrase still matches when the extracted text puts a line break or a run of spaces between the words, which is exactly what happens when a sentence wraps across a PDF line or column.

NOTE

It does **not** rejoin words hyphenated across a line break. A word split as algo- / rithm still has to be searched in halves. That is a known limit, stated in the release notes rather than hidden.

Generations, pending sets, and the stranded page

The bug this subsystem keeps trying to have, and the three-line shape that prevents it

If you read one chapter of Part IV twice, make it this one. Everything here is about asynchronous results arriving after the world has moved on, and the failure mode is a page that shows a loading state for ever.

23.1 The state involved

`PdfPane` (`native/src/pdf_pane.rs`) carries three in-flight request sets and one counter:

FIELD	MEANING
<code>pending_pages</code>	page images currently requested
<code>pending_links</code>	link metadata currently requested
<code>pending_text</code>	text geometry currently requested; bounded by <code>text_lru</code> to 12 pages
<code>render_generation</code>	bumped on a document change, a zoom change, or a navigation — so results from before the change can be recognised as stale

A pending set is not an optimisation. It is what stops the app requesting the same page fifty times while the first request is still in flight, so a page is scheduled only when the image is missing *and* the page is not already pending.

23.2 The invariant

NATIVE/SRC/APP.RS — THE SHAPE EVERY ASYNC PDF HANDLER MUST HAVE

```
Message::PdfRendered(generation, page, img) => {
    self.pdf.pending_pages.remove(&page);           // 1. always clean up first

    if generation != self.pdf.render_generation { // 2. then discard if stale
        return Task::none();
    }

    // 3. only now touch the cache
}
```

The same shape applies to `PdfRenderFailed`, `PdfPageLinksLoaded` and `PdfPageTextLoaded`. Swap steps 1 and 2 — the ordering a normal person would write, because “ignore stale results” feels like the first thing to do — and you get Figure 23.1.

THE SAME FOUR EVENTS IN BOTH COLUMNS

① page 14 is requested — `pending_pages = {14}`, `generation = 7` ② the reader zooms — `generation` becomes 8 ③ the old render finishes and arrives stamped 7 ④ page 14 scrolls t

GATE FIRST, CLEAN UP LATER — WRONG

③ the result arrives
`if generation != current { return }`
`7 ≠ 8` → returns immediately

`pending_pages = {14}` ← never removed

④ the page scrolls back into view
`if missing && !pending { request }`
the image is missing ... but 14 is pending,
so nothing is ever requested again.

page 14 loads for ever
and only a zoom or a re-open clears it

CLEAN UP FIRST, THEN GATE — RIGHT

③ the result arrives
`pending_pages.remove(&14);`
then: `7 ≠ 8` → discard the image

`pending_pages = { }` ← the slot is free

④ the page scrolls back into view
`if missing && !pending { request }`
missing, not pending → requested again,
this time at the correct zoom.

page 14 renders
one wasted render, no stuck state

Figure 23.1 — The stranded page. Both columns discard the stale image; only one frees the pending slot. `render_generation` exists to ignore stale *results* and must never be used to skip *cleanup* — that is the single easiest way to strand a page in a permanent loading state.

The corollary is just as important: **any code path that increments `render_generation` must also clear the in-flight state belonging to the previous generation** — `pending_pages.clear()` and `pending_links.clear()` — and a document change additionally clears `page_links`, `pages`, `dimensions` and `page_sizes`.

23.3 The deliberate exception

WHY IT IS LIKE THIS

`PdfReferencesLoaded` is gated on `document_id`, **never** on `render_generation`. References are a document-level artifact, and the chunked scan routinely outlives the fit-to-width zoom change that bumps the generation when a document opens.

Generation-gating them silently dropped every reference on first open. The tell was diagnostic gold: a *reopened* PDF worked — because it took the synchronous cache path — while a fresh one did not. `PdfTocLoaded` is gated on the document path for exactly the same reason.

CHECK YOURSELF

You add “extract the annotations layer” as a new async PDF result. What are the two questions to answer before writing the handler?

Answer: (1) Which pending set does it belong to, and is it removed *first*? (2) Is this a per-render artifact (gate on `render_generation`) or a document-level one (gate on `document_id`)? Getting question 2 wrong is the references bug again; getting question 1 wrong is the stranded page again.

Stable page slots

Why a rendered page must not change the space it occupies

A continuous PDF viewer has to answer “what is the scroll offset of page 300?” before page 300 has been rendered — and it has to give the same answer afterwards. Getting that wrong produces a bug that only appears deep in long documents, which is the worst kind.

24.1 The mechanism

Every page's unscaled size is loaded once with `page_sizes(path)` when the PDF opens — far cheaper than rendering, and available long before most pages enter the render queue. One of them (normally page 1) becomes `placeholder_page_size`. Then:

```
slot = (placeholder_width * zoom, placeholder_height * zoom)
```

Every page occupies that same outer slot while scrolling — including pages whose bitmaps have already rendered. Rendered bitmap dimensions are cached for image data and link maths, but they never change scroll geometry.

SLOT = WHATEVER THAT PAGE CURRENTLY IS

rendered pages use their bitmap size, unrendered ones use the placeholder



the model and the layout disagree by a few px per page
and the error **accumulates** — it is a running sum

jump to page 300

the computed offset is off by hundreds of pixels;
the reader lands on page 297, or mid-page, and the
viewer “corrects” itself with a visible jerk

SLOT = PLACEHOLDER SIZE × ZOOM, ALWAYS

the bitmap is drawn **inside** the slot; the slot never moves



the model and the layout are the same arithmetic
so there is nothing to accumulate

jump to page 300

offset = 299 × slot_height — exact, whether or not
any of those 299 pages has ever been rendered.
Rendering later changes nothing about where things are.

Figure 24.1 — Geometry that does not depend on what has loaded. The rule is not “placeholders are the same size”; it is that a page's *outer slot* is fixed, so scroll position is a function of the page index and the zoom alone. The unit tests in `app.rs` assert exactly this: target offsets map back to the same page, blank pages reserve space in the total height, and placeholder slots scale with zoom.

Page-size metadata drives loading placeholder dimensions, `pdf_page_offset(page)`, `pdf_page_at_scroll(scroll_y)`, viewport range detection, and the fit-to-width calculation — one source of truth feeding five consumers.

24.2 Rendering, supersampled

`render_pdf_page()` renders **only** the page image; it never extracts links. The PDFium render scale is *supersampled* relative to the displayed zoom — factoring in `ui.scale_factor`, the OS display scale — so a narrow split-view page stays sharp on HiDPI and fractional-scale displays without over-rendering on a 1× screen.

TRAP

Cached dimensions remain **logical display** dimensions, not raw bitmap dimensions. That distinction is what keeps link hit-testing and placeholder maths working in PDF-space/display-space geometry. Store the supersampled bitmap size there and every click lands in the wrong place on a HiDPI screen — and nowhere else, so you will not reproduce it.

On a successful `PdfRendered`: remove the page from `pending_pages`; discard if the generation is stale; store the handle in `pages` and `(width, height)` in `dimensions`; schedule low-priority link and text extraction for that page; and if the page is the active navigation target, compute the scroll offset and issue the programmatic scroll. A failure returns `PdfRenderFailed` rather than being swallowed by a generic tick.

24.3 Navigation and scrolling

TOC clicks, internal link clicks and PDF search-result clicks all funnel through `navigate_pdf_page(page)`:

1. clamp and store `current_page`;
2. clear `pending_pages` and `pending_links`;
3. increment `render_generation`;
4. store `toc_target_page`;
5. schedule link loading if the target page is already cached, otherwise request it through `render_pdf_page_direct()` — the **priority** channel;
6. compute the target offset from the same stable slot geometry the view uses;
7. set `programmatic_scroll = true` and issue `scroll_to`.

Scrolling then behaves differently depending on who caused it. **User scrolling** updates `scroll_y`, estimates the current page from `y + viewport_height * 0.33`, clears pending work if the page jump was large, and schedules viewport pages. **Programmatic scrolling** is acknowledged by the first matching `PdfScrolled`, which suppresses normal viewport scheduling for that event and schedules only a small range around the target.

WHY IT IS LIKE THIS

Without that suppression, one TOC click would schedule the target page, its neighbours, the whole viewport *and* link extraction, all at once — on a single-threaded worker. The jump would be slower than scrolling there by hand.

The viewport window itself is deliberately generous but bounded:

```
first = page_at(scroll_y - estimated_page_height)
last  = page_at(scroll_y + viewport_height + estimated_page_height)
range = (first - 2) ..= (last + 2)
```

Search navigation is the one exception to the page-top model: `navigate_pdf_search_to_index()` computes the offset from the *active match rectangle*, priority-renders the match page, schedules a small nearby range, and scrolls the match into view. It deliberately leaves `toc_target_page` unset, because the render-completion page-top scroll would otherwise override the match-precise offset.

From highlight to note and back

The pdf:// deep link, and the loop it closes

This is the feature the whole PDF subsystem exists to support: select a passage in a paper, attach it to a note, and be able to walk the connection in both directions afterwards.

25.1 What gets written

Selecting text and choosing **Link to Note** opens a searchable in-app vault picker (views/link_note_picker.rs): pick an existing note, or pick a folder and type a new note name. native/src/pdf_notes.rs generates the markdown.

```
---
type: pdf-note
source_pdf: papers/Introduction to Algorithms.pdf
---

# Divide and conquer

## Page 42

> A divide-and-conquer algorithm breaks down a problem into two or more
> sub-problems of the same or related type...

[Open highlight in PDF](pdf://papers/Introduction to Algorithms.pdf?
page=42&annotation=abcdef123456)

### Notes

This definition connects to [[Lecture 3 Notes]].
```

Six behaviours are worth knowing before you touch this file:

- A **new** note gets `type: pdf-note` / `source_pdf:` frontmatter and an `# H1` title derived from its filename (`shared pdf note.md` becomes `# Shared pdf note`).
- Reusing an **existing** note **appends** another `## Page N` section after a `---` rule, rather than overwriting the file.
- Re-linking the same highlight is **idempotent** — `append_linked_pdf_note_section` checks for the generated link and returns the file unchanged if it is already there.
- Page numbers in the heading and the link are **1-based**, while `page_index` is 0-based. Off-by-one bugs here are visible to the user, so the convention is worth internalising.
- The link target is the **vault-relative PDF path**, not the `document_id`; relative paths are resolved against the note's own location.
- The visible markdown carries **no debug identifiers**. The annotation id lives only inside the `pdf://` link, where it is needed for exact navigation.

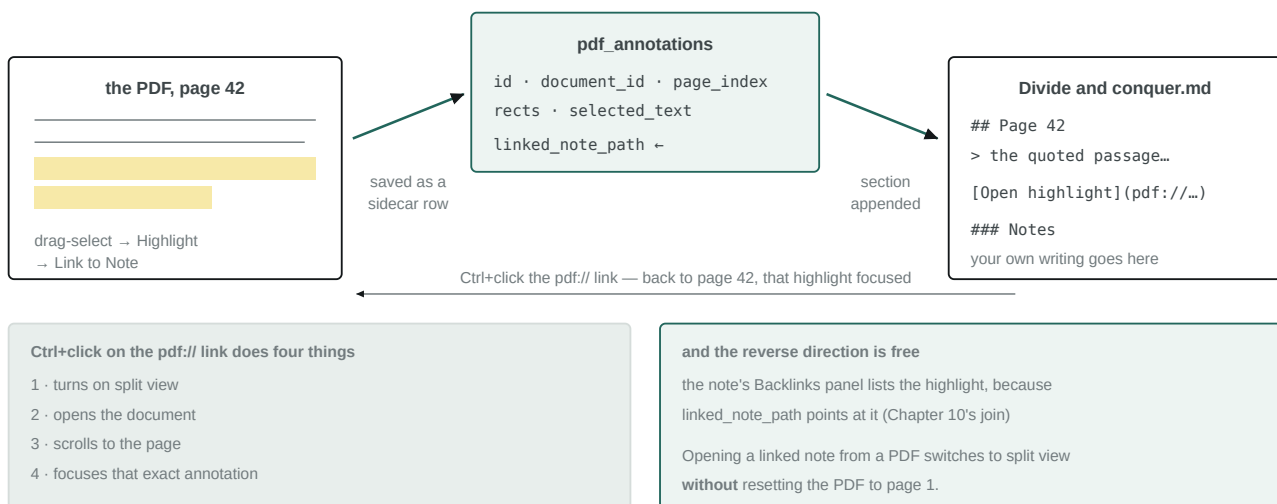


Figure 25.1 — A round trip with no central index. The forward direction is a link in the markdown; the reverse is a column in SQLite. Neither is derived from the other, which is why both keep working when you edit the note by hand or move the PDF.

TRAP

Linked-note writes must **append** to an existing note unless the exact highlight link is already present. Overwriting is the obvious implementation and it silently destroys a user's notes on the second highlight they link to the same file. The idempotency check exists for the third one.

The surface

Three small, strict systems that decide how the application feels: a token set that leaves no room for arbitrary numbers, a motion vocabulary of one curve, three durations and one spring, and a fuzzy matcher whose scores are comparable across commands and files.

1. Tokens and motion
2. The command palette and the fuzzy scorer
3. The study tracker

Tokens and motion

Six sizes, seven spacings, three radii, one curve, one spring — and the reason the lists are short

Everything the chrome draws — sidebar, toolbar, panels, modals, tracker — takes its colour, size, spacing and radius from `native/src/theme.rs`. The module's own comment records why: before the scales existed the app had **eleven** distinct font sizes and **seventeen** distinct spacing values.

WHY IT IS LIKE THIS

Eleven font sizes is the visual noise that reads as “unpolished” without ever being nameable. Nobody looks at a screen and thinks “that label is 13px and this one is 14px”; they think “this feels amateur”. A short scale removes the decision *and* the defect at once, and it makes review mechanical: a hardcoded number in a view module is wrong on sight, with no taste required.

Document typography — heading sizes and code or math scale *inside* a note — is a separate concern owned by the markdown renderer. Do not import `theme.rs` sizes into `highlight.rs`.

TYPE SCALE — SIX STEPS

TEXT_XS · 8	Fine print: badges, counters, gutter numbers
TEXT_SM · 10	Secondary: captions, metadata, tree affordances
TEXT_BASE · 11	Default UI text: controls, buttons, palette rows
TEXT_MD · 14	Emphasised UI text and section headings
TEXT_LG · 16	Panel and dialog titles
TEXT_DISPLAY · 42	MD Editor — the welcome wordmark, deliberately the only display size

SPACING — A 2 PX GRID, DRAWN TO SCALE

SPACE_0	0 — flush, no gap
SPACE_1	2 — hairline separation between tightly related items
SPACE_2	4 — within a control: icon to label
SPACE_3	6 — default gap between siblings; default control padding
SPACE_4	12 — between grouped rows; comfortable control padding
SPACE_5	16 — between groups within a panel
SPACE_6	24 — between major regions

RADII — THREE

2 · SM	chips, badges, inline markers
6 · MD	buttons, inputs, list rows
10 · LG	panels, cards, modals

The palette is one dark theme registered with iced as “MD Editor Premium Dark” — ground `#0d0e10`, text `#e3e5ed`, accent `#b1ccc6`, danger `#ee7d77`. `fade(color, opacity)` scales a colour's own alpha, so a half-faded overlay stays internally consistent instead of some parts leading others.

Figure 26.1 — The whole vocabulary on one page. Sizes are shown at the figure's own scale, so read the ratios rather than the millimetres. The chrome's type is deliberately small, so that it recedes behind the document; what matters is that there are six sizes and each has a named job, so choosing a size means looking up a role rather than picking a number.

26.1 Motion

Before `native/src/motion.rs` the app had no motion at all: panels appeared and vanished between one frame and the next. Nothing is *missing* from a snapshot of an instant UI — it just feels abrupt, because the eye gets no continuity between the two states.

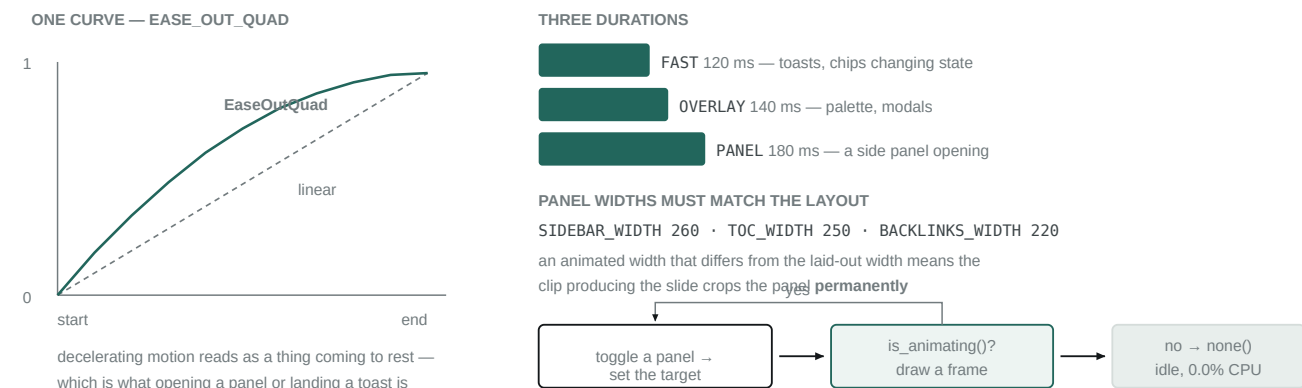


Figure 26.2 — One curve, three durations, and a loop that ends. The per-frame redraw subscription is armed *strictly* while `motion.is_animating()` holds; the moment every transition settles it returns `Subscription::none()` and the process sleeps.

`Motion` holds an `iced::Animation<bool>` mirroring each visibility flag — `sidebar`, `toc`, `backlinks`, `toast`, `palette` — plus two things worth noticing:

- `now: Instant`, the timestamp the current frame is drawn for. `view` reads *this* rather than calling `Instant::now()` itself, so every animation in a frame is sampled at the same moment. Without it, panels drift out of step within a single frame.
- `toast_text: String`, retained after `ui.toast` clears — so the toast has something to draw *while it fades out*. An animation that outlives its data needs somewhere to keep it.
- `editor_scroll: Option<ScrollGlide>`, the editor gliding to reveal the caret, while it is (Chapter 17).

The booleans themselves stay where they live (sidebar visibility on the vault state, TOC on the editor pane); `Motion` only tracks how far each has travelled.

26.2 Springs: motion that follows something

A duration and a curve are right for a transition between two known states: the panel is closed, it will be open, and 180 ms is how long that takes. They are wrong for motion whose destination keeps moving — the page scrolling to keep a caret in view while you type, or the caret itself sliding to where an arrow key put it. Restarting an ease on every keystroke would stall the motion each time it began again. That kind of motion uses a **critically damped spring**.

NATIVE/SRC/MOTION.RS

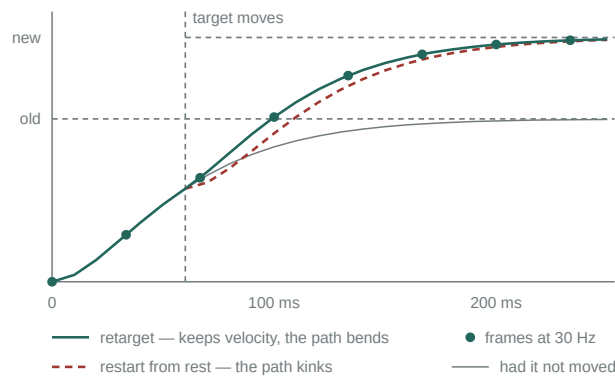
```

pub struct Spring { target: f32, x0: f32, v0: f32, start: Instant, stiffness: f32 }

// displacement x0 and velocity v0 at `start`; ω = stiffness
x(t) = target + (x0 + (v0 + ω·x0)·t)·e^(-ωt)
  
```

The spring is **solved in closed form**, not integrated frame by frame. Its position is a function of the frame's time, `Motion::now`, so it traces exactly the same curve at 30 Hz or 240 Hz, and a dropped frame costs nothing. Critical damping is the fastest approach to the target that does not overshoot from rest — a scroll that bounced past the caret and back would be unreadable.

SCROLL_STIFFNESS = 32 · THE TARGET MOVES AT 60 MS



Sampled, not stepped

position is a function of the frame's time, so the dots at 30 Hz sit exactly on the curve a 240 Hz display draws — and a dropped frame is simply a dot that was not drawn

Stepped (pos += vel · dt) — what is not done

the result depends on dt, so a slow frame jerks the motion and a fast display moves differently from a slow one; and it has to run on every frame just to stay correct

Critical damping

the fastest approach that never overshoots from rest

Figure 26.3 — A spring that follows. At 60 ms the target jumps. `retarget` keeps the spring's position and velocity, so the path bends; restarting from rest throws the velocity away and the motion visibly stalls before setting off again. The dots are what a 30 Hz display would draw — the same curve, fewer samples.

METHOD	WHAT IT DOES
<code>retarget</code>	heads for a new target from wherever the spring is, keeping its velocity, so a new destination bends the path instead of kinking it
<code>shift</code>	displaces the spring while keeping its velocity: the target stays, and the spring sets off again from further away
<code>is_settled</code>	true once remaining distance plus momentum is below a tenth of a pixel — the moment the frame subscription may disarm

STIFFNESS	TO 95%	USED BY
<code>SCROLL_STIFFNESS = 32</code>	~150 ms	<code>motion.rs</code> — the editor scrolling to reveal the caret
<code>GLIDE_STIFFNESS = 80</code>	~60 ms	<code>editor/renderer/glide.rs</code> — the caret gliding to its new place

TRAP

Never write `pos += vel * dt` for motion in this codebase. Step-by-step integration makes an animation depend on the frame rate, and it has to run on every frame to stay correct, which fights the zero-idle-CPU rule. Sample the closed form at the frame's time instead.

26.3 The caret: glide and blink

The caret is the one piece of motion inside the editor widget, and it lives in `editor/renderer/glide.rs`.

- **Glide** — the caret is painted where layout puts it, plus an offset that springs to zero. When it moves, the offset starts as the distance moved, so the eye sees it travel. Moves of more than `MAX_GLIDE = 96px` vertically cut instead, and layout moving under a still caret — an image loading above it — never glides.
- **Blink** — lit for `BLINK_DELAY = 500ms` after it last moved, then a soft fade: a cosine over `BLINK_PERIOD = 1060ms`, stretched by `BLINK_SHARPNESS = 2.5` and clamped, so it holds fully on and fully off with quick fades between.
- **Rest** — after `BLINK_CYCLES = 18` blinks, about twenty seconds, the caret stays lit.

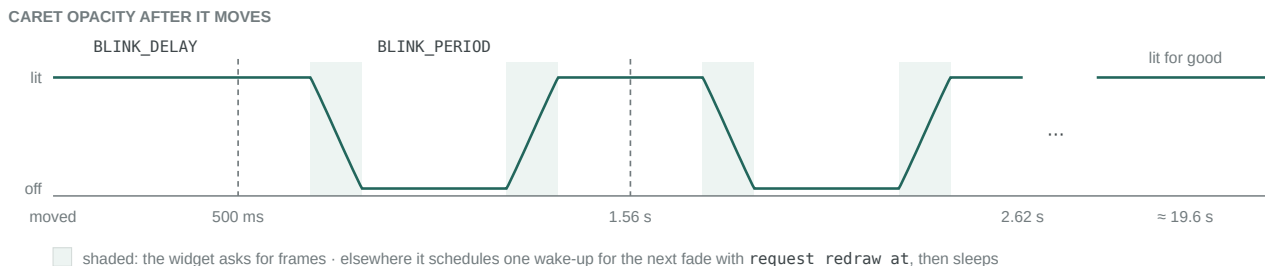


Figure 26.4 — The blink, and the frames it costs. A clamped cosine spends most of each period fully on or fully off, where nothing on screen changes and no frame is needed. Only the four short fades per two blinks cost frames.

The widget schedules its own frames by the same discipline as the shell's subscriptions: `shell.request_redraw()` only while the caret glides or fades, `request_redraw_at` for the exact instant the next fade begins, and nothing once the caret rests lit.

CHECK YOURSELF

Why does the caret stop blinking after about twenty seconds, when most editors blink forever?

Answer: because a blinking caret needs frames, and frames are CPU. A caret that blinked forever would make durability invariant 5 — zero idle CPU with a note open — impossible to meet. Eighteen blinks is long enough to find the caret after looking away, and then the window can sleep.

The command palette and the fuzzy scorer

Making a command and a file comparable on one numeric scale

`Ctrl` + `P` opens a palette that searches commands and vault files *together*. That is only possible if both can be scored on the same scale — which is what `native/src/fuzzy.rs` is for.

NATIVE/SRC/FUZZY.RS

```
pub fn score(haystack: &str, query: &str) -> Option<i32>
pub fn score_path(path: &str, query: &str) -> Option<i32>
```

`score` walks the query left to right, advancing through the haystack to each next matching character, case-insensitively. `None` means the query does not appear in order at all — characters must be in order, but need not be adjacent, which is what makes initials work.

RULE	EFFECT
Base	+10 per matched character
Word start	+18 when the character is at index 0 or follows a boundary — <code>space / \ - _ . ([</code>
CamelCase	+10 for an uppercase letter inside a word (mutually exclusive with the word-start bonus)
Adjacency	+12 when it directly follows the previous match, rewarding runs
Distance penalty	-min(gap, 8) for characters skipped since the previous match
Length normalization	-(haystack.len() / 12) overall, preferring the tighter of two otherwise-equal matches

QUERY: "sv"

command · "Split View"

Split View	s → index 0, a word start	+10 base +18 word start	= 28	≈ 51 – 0 length + 8 command bonus
	v → index 6, follows a space	+10 base +18 word start –5 gap	= 23	

file · "Systems/vector-notes.md"

Systems/vector-notes.md both are word starts, but the path is long: the length penalty and the gap penalty both bite, and there is no command bonus

file · "notes/archive/sv-draft.md"

notes/archive/sv-draft.md adjacency (+12) helps a lot — but `score_path` also scores the filename alone with a +24 bonus and takes the better of the two, which is what lifts it above the path match

The two thumb-scales that make the ranking feel right

+24 for matching a file's own name rather than a folder on its path · +8 for a command at equal relevance, because the palette is primarily a way to *act*

Figure 27.1 — Why `sv` finds Split View. The bonuses are small integers chosen so that intuitive expectations win: initials beat substrings, filenames beat folders, and actions beat documents when everything else is equal. Concretely, `tocfg` finds *Tracker Configuration*.

27.1 The command registry

Every action the palette can run is declared in **one** registry in `views/command_palette.rs`, so a command exists in exactly one place and the palette and the shortcut list cannot drift apart.

COMMAND	SHORTCUT	DIRECT KEY BINDING?
New File	<code>NewFile</code>	Ctrl+N
Open Vault	<code>OpenVault</code>	Ctrl+O
Save File	<code>Save</code>	Ctrl+S
Search Vault	<code>Search</code>	Ctrl+F
Toggle Sidebar	<code>ToggleSidebar</code>	Ctrl+B
Table of Contents	<code>TableOfContents</code>	Ctrl+T
Toggle Backlinks	<code>ToggleBacklinks</code>	none — palette only
Study Tracker	<code>StudyTracker</code>	none — palette or toolbar
Split View	<code>SplitView</code>	none — palette or toolbar
Focus Mode	<code>FocusMode</code>	none — palette only

`Shortcut` is the indirection that lets the keyboard subscription and the command palette trigger exactly the same handler. Add a command by adding a `Shortcut` variant and a registry entry; do not add a second code path.

27.2 Ranking and interleaving

- With an **empty query** the palette lists commands only — a menu of what the app can do. Files would just be an arbitrary slice of the vault.
- With a query, commands are scored with `fuzzy::score` and files with `fuzzy::score_path`, then **interleaved by relevance** rather than split into fixed sections.
- Directories are never offered as results.
- The list is capped at `MAX_RESULTS = 40` — past that the answer is a longer query, not more scrolling.
- `↑/↓` move the highlighted row, `Enter` activates it, `Escape` closes. Changing the query resets the highlight to the first row, since keeping the old index would leave it on an unrelated result.

The palette's ranking is covered by eight unit tests, including that `Enter` activates exactly the highlighted row and that `result_count` matches what is rendered — the two properties a keyboard-driven list can silently get wrong.

The study tracker

A timer, a log, and a curriculum the app knows nothing about

The tracker is the most self-contained feature in the app: three tables, one settings key, six tabs, and a deliberate refusal to model any particular subject.

28.1 Storage

WHERE	WHAT
<code>tracker_sessions</code>	one row per logged interval, whether timed or entered by hand: <code>id</code> , <code>date</code> , <code>hours</code> , <code>activity_type</code> , <code>phase</code> , <code>notes</code>
<code>tracker_kv</code>	the checkbox and status state of the curriculum — project statuses (<code>proj_<id></code>), gate ticks (<code>gate_<id>_<n></code>), reading ticks (<code>read_<section>_<n></code>). Written through <code>tracker::set_kv</code> , an upsert.
<code>tracker_activity</code>	created by the schema, available for an activity feed; the current UI does not write to it.
<code>settings['tracker_config']</code>	the curriculum itself, as JSON. It lives in <code>settings</code> rather than <code>tracker_kv</code> because it is <i>one document, not a set of flags</i> , and it is only accepted after <code>parse_config</code> validates it.

NOTE

`get_sessions` skips a malformed row **and logs it**. Silently dropping study history would hide corruption — and a tracker whose totals quietly shrink is worse than one that complains.

28.2 The timer, and what it refuses to be

One button. **Start** records `Instant::now()` and toasts “*Study timer started*”. **Stop** computes `elapsed.as_secs_f32() / 3600.0`, floors it at `0.01` hours so a very short session still records something, and saves a `StudySession` **immediately** — `date` from `chrono::Local::now()` as `"%Y-%m-%d %H:%M"`, `activity_type = "Study"`, `phase = "Focus"`, no notes.

There is no pause, no pomodoro cycle, and no post-stop dialog. To record an activity type or notes you use the Log tab's manual entry, which takes a date, an hours value and free text.

WHY IT IS LIKE THIS

A dialog after every session is a tax on the behaviour the feature is trying to encourage. Saving immediately with defaults means the worst case is a row with a generic label — recoverable — rather than a session you abandoned because you did not want to fill in a form.

28.3 The configurable curriculum

Four collections, and the tracker understands nothing beyond their shape:

```
{
  "PHASES":  [{ "id": "1A", "title": "Mathematics", "year": "Year 1", "months": "Months 1-4"
}],
  "PROJECTS": [{ "id": "1.1", "phase": "1A", "name": "SVD from scratch" }],
  "GATES":    [{ "id": "1A", "title": "Gate 1A - Mathematics",
                  "items": ["Derive SVD from eigendecomposition"] }],
  "READING":  [{ "section": "Textbooks - Mathematics",
                  "items": [{ "priority": "critical", "title": "Linear Algebra Done Right" }] }]
}
```

Out of the box it is seeded with a concrete four-year roadmap toward research in efficient machine learning — 17 phases (1A Mathematics through 4C Original Research), 20 projects, 6 checkpoint gates, and a reading list. **That is a starting point, not a structure.** The Config tab replaces it wholesale, and a language course, a thesis, or a certification path fits the same four collections exactly as well.

`TrackerConfigSave` runs `parse_config` first, which requires at least one phase and at least one project. An invalid document is rejected rather than saved, and a stored config that fails to parse falls back to the built-in default — so a bad edit costs you the edit, never the app.

Two lifecycle details: `TrackerState::new` loads sessions and key-values once at startup and validates the stored config before adopting it; opening the panel (`TrackerToggle`) reloads from disk, so an external change is picked up. Side effects that belong to the global UI are emitted as `Message::ShowToast` tasks rather than reaching back into the shell — the tracker never touches another pane's state, which is the sub-state rule from Chapter 11 in practice.

NOTE

Gates are a **self-assessment checklist**, not an enforcement mechanism. Nothing in the app blocks progress on an unticked gate, and nothing should — the tracker's job is to show you where you are, not to hold a door shut.

Working here

How the build finds a C library it downloads at compile time, what the 215 tests are actually protecting, the five properties that block a release, and the rules — each with the bug that produced it.

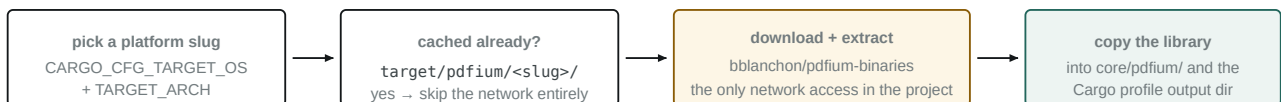
1. The build system and PDFium
2. The test suite, and what each part protects
3. The five durability invariants
4. The rules, and the bug behind each one
5. Shipping a build

The build system and PDFium

A native library that is downloaded at compile time and looked up at run time

PDFium is a large C++ library from Chromium. Building it from source is out of the question for a project this size, so `core/build_pdfium.rs` fetches a prebuilt binary, and `bind_pdfium` finds it again at run time. Two separate mechanisms, and most “PDFs don't work” reports are one of them.

BUILD TIME — `core/build_pdfium.rs`

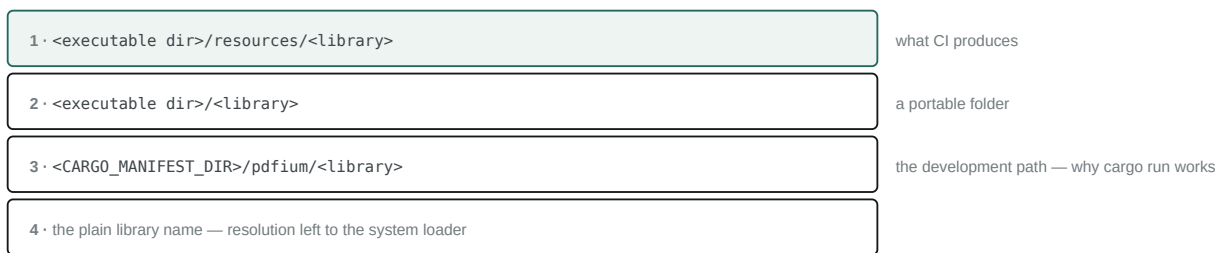


Two opt-in environment variables — both unset by default

`PDFIUM_RELEASE` pins a release tag, e.g. `chromium/6996` (default: `latest`) · `PDFIUM_SHA256` is the expected digest of the platform .tgz

With `PDFIUM_SHA256` set, a mismatch **panics the build**. Unset, the script warns that the archive is unverified — you are trusting whatever the network returned.

RUN TIME — `core/src/pdf.rs::bind_pdfium`, once per process behind a `OnceLock`



Unsupported platform? The build script prints a warning and skips the download. **PDF support is simply unavailable and the rest of the app still builds.**

Figure 29.1 — Two lookups, once each. The build-time half runs once per cache; the run-time half runs once per process, behind a `OnceLock`, because exactly one `Pdfium` binding may exist. If PDFs fail to open in a packaged build, candidate 1 is almost always the answer.

PLATFORM	ARCHITECTURES	SLUG	LIBRARY
Linux	x86_64 / aarch64	linux-x64 / linux-arm64	libpdfium.so
Windows	x86_64 / aarch64	win-x64 / win-arm64	pdfium.dll
macOS	x86_64 / aarch64	mac-x64 / mac-arm64	libpdfium.dylib

TRY IT

For a reproducible, verified build, set both variables:

```
PDFIUM_RELEASE=chromium/6996 PDFIUM_SHA256=<hex digest> cargo build --release
```

Leaving them unset is fine for local development and is *not* fine for a release you are asking other people to run.

29.1 Continuous integration

`.github/workflows/windows-build.yml` — named for its original scope, it now covers the whole release pipeline. It runs on pushes to `main`, on `v*` tags, and on manual dispatch, with three jobs:

JOB	RUNNER	PRODUCES
<code>build-windows</code>	<code>windows-latest</code>	<code>cargo build --release</code> , then a package of <code>md-editor.exe</code> plus <code>resources/pdfium.dll</code> (downloaded separately from <code>bblanchon/pdfium-binaries</code>), zipped as <code>md-editor-windows-x64.zip</code>
<code>build-linux</code>	<code>ubuntu-latest</code>	the same layout using the <code>libpdfium.so</code> the build script already placed in <code>target/release/</code> , as both <code>.tar.gz</code> (which keeps the executable bit) and <code>.zip</code>
<code>publish-release</code>	<code>ubuntu-latest</code>	downloads both artifacts and publishes a GitHub Release — <code>v*</code> tags are final, everything else publishes as <code>latest</code> and is marked pre-release

TRAP

CI does not run `cargo test`, `cargo fmt --check`, or `cargo clippy`. It builds and packages. Everything in Chapter 30 is the contributor's responsibility *before* opening a pull request — a green checkmark on your PR means “it compiled on two runners”, nothing more.

Give a first build on either runner extra time: `rusqlite`'s bundled SQLite compiles from source and the PDFium archive downloads, both exactly once per cache.

The test suite, and what each part protects

215 tests and three opt-in tools, no display server, a few seconds

The suite is 56 tests in `md-editor-core` and 159 in `md-editor-native`, plus three opt-in renderer tools that `cargo test` skips (30.2). Read as a list of names it is a good table of contents; read as a map of fears it is better still.

`cargo test --workspace`

WHAT COULD BREAK

a save leaves half a file on disk
or drops permissions, or replaces a symlink

a wikilink resolves differently after a rename
or a backlink survives the link being deleted

TOC recovery invents an outline
or PDF search misses a wrapped phrase

undo takes back one character again
or Backspace splits an emoji, or a list marker panics

the highlighter drops a character
or an edit highlights differently from a fresh parse

the height tree drifts on long documents
or a margin, a caption number or a glide is off

a click lands off the text the caret is drawn on
or a selection has gaps, or wide code shows past its edge

a cache agrees with a cold layout, until it doesn't
on a document nobody thought to write by hand

a TOC jump lands on the wrong PDF page
once blank pages are involved

the palette opens the row you did not select
or a spring never settles and the window never idles

--install leaves junk in someone's \$HOME
or a nonsense window size is restored

WHAT CATCHES IT

`core/src/vault.rs` — 4 atomic-write tests
no .tmp left · permissions preserved · symlink written through

`core/src/massive_tests.rs` — 8 combinatorial suites
link variants · graph topologies · index fuzzing · file lifecycle

`core/src/pdf.rs` + `references.rs`
page count · text · search · TOC recovery · target maps · precision rules

`native/src/editor/buffer.rs`
coalescing · graphemes · affinity · multibyte markers · stress test

`native/src/editor/highlight.rs`
source preserved · incremental = fresh · a reference implementation

`layout_tree.rs` + `renderer/` unit tests
exact prefix sums · margins · captions · reveal policy · blink schedule

`renderer/tests.rs` — through the Widget API
draw calls out: caret on glyphs · clicks · clipping · layout skipped

`renderer/properties.rs` — seeded invariants I1–I12
generated documents, 40–1400 px wide, shrunk to a minimal case

`native/src/app.rs` — page-slot tests
offsets map back to their page · blank pages reserve space · slots scale

`fuzzy.rs` · `views/command_palette.rs` · `motion.rs`
ordering · initials · Enter opens the highlighted row · springs settle

`native/src/main.rs`
CLI parsing · window-size round-trip · a full desktop install/uninstall cycle

Nothing here needs a window, a GPU, or a network. That is deliberate: a test you cannot run in CI is a test that stops being run.

Figure 30.1 — The suite as a map of fears. Read right to left when you are adding a test and are not sure where it belongs: find the failure you are preventing, and the suite that already owns that failure is the one to extend.

30.1 Property suites: the editor's invariants

The editor's geometry is pinned by invariants rather than by examples alone.

`renderer/properties.rs` generates documents from a seed — every block and inline kind, pathological nesting, CRLF, widths from 40 to 1400 px — using the generator it shares with the highlighter's suite (`editor/test_docs.rs`). A failing case is shrunk line by line and reported with its seed and a minimal document.

INVARIANT	TEST
I1 — the caret lies on painted glyphs of its row, on either side of a row break	<code>caret_lies_on_painted_glyphs</code>
I2 — hitting where the caret is drawn yields a position drawn at exactly the same place	<code>hit_testing_inverts_caret_placement</code>
I3 — caret positions advance in reading order	<code>caret_order_follows_logical_order</code>
I5 — rows tile a line; painted content stays inside its line box	<code>rows_tile_their_line_and_contain_their_items</code> <code>painted_content_stays_inside_line_boxes</code>
I6 — cached and early-out layouts equal a cold layout through edits and caret moves	<code>cached_layout_equals_cold_layout</code>
I8 — the height tree, the widget height and the uncached model agree	<code>height_tree_agrees_with_line_visual_y</code>
I9 — up and down move exactly one visual row	<code>vertical_moves_step_one_visual_row</code>
I10 — no input panics	<code>arbitrary_input_never_panics</code>
I11 — scroll offsets stay in range and thumbs follow drags	<code>scrollbars_stay_in_bounds_and_follow_drags</code>
I12 — drawing touches only visible lines	<code>drawing_visits_only_visible_lines</code>

```
cargo test -p md-editor-native properties # default case counts
RENDER_PROPERTY_CASES=3000 cargo test -p md-editor-native properties # before a renderer
change lands
RENDER_PROPERTY_SEED=<seed> cargo test -p md-editor-native properties # replay a reported
failure
HIGHLIGHT_PROPERTY_CASES=3000 cargo test -p md-editor-native highlight
```

WHY INVARIANTS

An example test checks the document its author thought of. A generator writes thousands its author did not — a kerned pair at a row break, a checkbox beside an equation in a 40 px column — and the shrinker turns the one that fails into something a person can read. The height tree's floating-point drift (Chapter 15) was found this way. I12 is Chapter 17's culling rule made executable: it fails the moment a draw pass touches a line outside the viewport.

30.2 Three opt-in tools

Three renderer tests are marked `#[ignore]`, so `cargo test` skips them. Each is a tool you run on purpose.

The render transcript harness

`editor/render_snapshot_tests.rs` drives the `Editor` widget purely through its `Widget` API — layout, draw, update, mouse interaction — against a document that exercises every block and inline kind, at three widths. It records every quad, text run and image drawn, every message published, clipboard writes and mouse-cursor shapes, into a plain-text transcript. Drawing goes to a recording renderer that borrows the real renderer's text measurement, so no window or GPU is needed.

```

RENDER_SNAPSHOT=/tmp/render.txt RENDER_SNAPSHOT_WRITE=1 \
  cargo test -p md-editor-native render_snapshot -- --ignored # before the change: a baseline
RENDER_SNAPSHOT=/tmp/render.txt \
  cargo test -p md-editor-native render_snapshot -- --ignored # after: compare
diff /tmp/render.txt /tmp/render.txt.actual | less

```

- **For a refactor**, the comparison must pass. That is how splitting `renderer.rs` into `renderer/` was verified to change nothing.
- **For a visual fix**, the diff should contain the draw calls you meant to change, and nothing else.

The transcript is tens of megabytes and pins current behaviour, known bugs included, so keep it out of the repository.

Render previews

`editor/render_preview.rs` renders sample scenes to PNG through iced's tiny-skia software `renderer` — a caret mid-paragraph and after a checkbox, selections across wrapped rows, across lines and across a table, an equation and an image, a caret in equation source, a wide unfocused view — so rendering can be looked at without opening a window. It uses only the widget's public API, so the same file dropped into an older checkout renders a before-and-after pair.

```

RENDER_PREVIEW_DIR=/tmp/previews cargo test -p md-editor-native render_preview -- --ignored

```

The timing report

Prints what frames cost on a 37,000-line document — the table in Chapter 16. Run it in a release build; debug numbers mean nothing.

```

cargo test -p md-editor-native --release layout_timing -- --ignored --nocapture

```

30.3 Where a new test goes

- A bug fix in the markdown parser, undo grouping, or auto-pairing needs a regression test in `native/src/editor/buffer.rs` or `editor/highlight.rs`. A highlighter change must keep the incremental-equals-fresh property passing.
- A rendering, caret or hit-testing bug in the editor needs a test in `renderer/tests.rs`, written against the draw calls and messages the widget produces rather than its internals, and **seen failing before the fix**. When the bug breaks a general rule, strengthen the matching invariant in `renderer/properties.rs` too, and run it at `RENDER_PROPERTY_CASES=3000` before landing.
- A change to wikilink parsing or resolution must be validated against `core/src/massive_tests.rs`.
- A change to the reference resolver's output for the same input **must bump** `references::RESOLVER_VERSION`.

- A change to PDF scroll or page geometry needs coverage in the page-slot tests in `native/src/app.rs`.
- New core functionality belongs behind a headless test. **If it cannot be tested without a window, it probably belongs in `md-editor-native`** — that sentence is a design check as much as a testing rule.

TRY IT

Break something on purpose and watch which test catches it. Change `UNDO_COALESCE_WINDOW` to `0` and run `cargo test -p md-editor-native`; then put it back and change the atomic write to a plain `fs::write` and run `cargo test -p md-editor-core`. Ten minutes, and you will remember which suite owns what for the rest of the project.

The five durability invariants

Pass/fail engineering requirements, not judgement calls

These five properties are checked before every release, by hand, because no automated test fully replaces them. **If any one fails, the build does not ship, regardless of what else works.**

Invariant 1 · Kill it mid-edit

Scenario — type paragraphs into a note, then `kill -9` the process without pressing `Ctrl + S`.

Guarantee — reopening the vault shows no corrupted characters and no truncated or empty file. The sibling-temp-file protocol means the destination holds either the complete previous version or the complete new one, never a mix. No stray `.tmp` files are left in the folder either.

Mechanism — `vault::write_file` (Chapter 8), plus the unit test `write_file_never_leaves_a_truncated_file`.

Invariant 2 · Switch with unsaved edits

Scenario — edit a note and immediately click another file, an image, or close the window, before the 400 ms debounce fires.

Guarantee — the buffer is flushed *before* the view changes. If the write fails — disk full, permission revoked — navigation is refused and the error is surfaced, so unsaved edits are never abandoned. Window close is the one deliberate exception: best-effort, non-blocking.

Invariant 3 · Undo is word-sized

Scenario — type a multi-word sentence, press `Ctrl + Z` once.

Guarantee — a run of typing disappears, not a single character.

Mechanism — `UNDO_COALESCE_WINDOW = 300ms` plus `can_coalesce`, which breaks a run at a pause, a newline, a cursor jump, a direction change, or a selection edit (Chapter 13).

Invariant 4 · Navigation preserves undo

Scenario — edit `FileA.md`, go to `FileB.md`, edit it, come back to `FileA.md`, press `Ctrl + Z`.

Guarantee — undo walks back through the edits made before navigating away, and the cursor and scroll position are where you left them.

Mechanism — the retained-buffer registry: `MAX_RETAINED_BUFFERS = 32`, least-recently used evicted. `take_retained_matching(path, disk_text)` returns a parked buffer *only* when the file on

disk still matches what the buffer was based on; otherwise the stale buffer is dropped, so an external edit is never silently overwritten by a resurrected undo stack.

Invariant 5 · Zero idle CPU

Scenario — leave the app open with no typing, no mouse movement, and no animation in flight.

Guarantee — the process consumes **0.0%** CPU.

Mechanism — every subscription is armed only while it has pending work, and the per-frame subscription only while `motion.is_animating()` holds (Chapters 12 and 26). A settled window has no live timers at all. The editor widget obeys the same rule for its own frames: it requests them only while the caret glides or fades, and after `BLINK_CYCLES = 18` blinks the caret stays lit so the window can sleep.

CHECK YOURSELF

Which invariant does “opening another note flushes the current one first” protect, and which one does it *also* depend on?

Answer: it protects invariant 2 directly, and it depends on invariant 1 — flushing before navigating is only a guarantee if the flush itself is atomic. A non-atomic flush would turn “save before switching” into “corrupt before switching”, which is strictly worse than not saving at all.

31.1 Verifying them by hand

- Type into a note, then `kill -9`. Reopen: no lost characters, no truncated file, no `.tmp` files in the folder.
- Edit a note, click straight to another file, come back. The edits are on disk and `Ctrl + Z` still walks back through history from before the switch.
- Type a sentence, press `Ctrl + Z` once. A run disappears, not a character.
- Leave a note by *every* route with unsaved edits: click another note, click an image, close the window. In each case the edits reach disk and the undo history survives.
- Quit and relaunch: same window size, same file, same scroll offset and cursor position.
- Leave the settled window alone and check `top`: 0.0%.

The rules, and the bug behind each one

Seven architectural invariants a reviewer will check

Each of these exists because something concrete broke without it. They are stated in the contributor guidelines as rules; here they are stated as consequences, which is the form that actually sticks.

Rule 1 · Keep the crates decoupled

Never import `iced`, `winit`, `ratex-render`, or any GUI or text-shaping crate into `md-editor-core`.

→ if you do: the 56 core tests stop running in a container with no display, and the engine stops being testable at all.

Rule 2 · Use the design tokens

No hardcoded dimensions, padding, font sizes or hex colours in view modules. Sizes from `TEXT_XS`...`TEXT_DISPLAY`, gaps from `SPACE_0`...`SPACE_6`, radii from `RADIUS_SM`/`MD`/`LG`, colours from the palette. Animations use `EaseOutQuad` and one of `FAST`, `PANEL`, `OVERLAY` — a new duration needs a reason, not a preference. Motion that follows something moving — the editor scrolling to the caret, the caret gliding — uses the closed-form critically damped `Spring`, sampled at the frame's time; never integrate motion step by step per frame. A panel's animated open width must match the width it actually lays out at.

→ if you do: eleven font sizes and seventeen spacings, and a panel permanently cropped by its own slide animation.

Rule 3 · Keep the draw pass proportional to what is visible

The draw pass in `renderer/draw/` must never iterate the whole document: clamp strictly between `visible_start` and `visible_end`, and stop once `y` passes the viewport. The layout pass runs only when its `LayoutKey` changes, and when it runs every line must be a cache hit unless something about it actually changed — state that affects a line's height goes into `line_hash` or `resource_hash`, and must change `layout_revision` or `LayoutKey`. Vertical sizes sit on the 4 px `GRID`, and space between lines goes through `requested_margins`, where it collapses — never pad a painter to fake a gap. A size two passes must agree on goes in `metrics.rs`; inline text is broken into rows only in `flow.rs`; caret facts live only in `DocBuffer`. Clip with layers, never with a primitive's clip rectangle. When adding a block type, update block-range tracking, body height, margins, draw metadata, and hit testing and scrolling **together**.

→ if you do: the caret drifts off the text, the editor paints a stale layout, or it silently makes $O(N)$ shaping calls per frame — and nobody can tell you which commit did it.

Rule 4 · PDFs are immutable

Never write to a user's `.pdf`. Highlights, notes, bookmarks and cross-references belong in `pdf_annotations` and `pdf_references`, keyed by the content-derived `document_id`. Only ever create one `Pdfium` binding per process, and reach it only through `core/src/pdf.rs`.

→ if you do: you have modified a user's source material, and a second binding is undefined behaviour in a C++ library.

Rule 5 · Do not weaken durability

Never replace the atomic save sequence with a direct truncating write. Navigation must keep flushing the active buffer before switching, and must keep **refusing to navigate** when that write fails. Undo histories must keep surviving navigation through the 32-document retained registry, and a parked buffer must keep being discarded when the file changed on disk.

→ if you do: invariants 1, 2 and 4 all fail at once, and the failure only shows up on someone else's machine, at the worst moment.

Rule 6 · Zero idle CPU

Arm the per-frame subscription **only** while `motion.is_animating()` is true. The same discipline applies to the toast timer, autosave poll, search debounce and the editor's scroll interrupt: each returns `Subscription::none()` when it has nothing pending. Widgets schedule their own frames the same way — the editor asks for the next frame only while the caret glides or fades, schedules the start of the next fade with `request_redraw_at`, and asks for nothing once the caret rests lit.

→ if you do: a new always-on timer is a regression even if it looks cheap, and it silently costs every user battery for ever.

Rule 7 · Async results clean up before they are gated

In every PDF result handler, remove the page from its pending set **before** checking `render_generation`. Any code that increments `render_generation` must clear the previous generation's pending sets. Document-level artifacts — resolved references, the TOC — are gated on `document_id` or the document path, **never** on `render_generation`. Markdown highlighting has no async result to gate: it runs incrementally and synchronously on every edit, so the painted lines always match the buffer. Keep it that way — lines that lag the text are how clicks land on the wrong line.

→ if you do: Chapter 23's stranded page, or references that vanish on first open and appear on second.

32.1 The full PDF rule list

The PDF subsystem carries fifteen rules of its own. They are worth reading in full before changing anything in `pdf_pane.rs` or the PDF handlers in `app.rs`:

1. Never create more than one `Pdfium::new()` binding in the process.
2. Navigation-target renders must use `render_page_priority()`.
3. Priority rendering may use `Wake`, but `Wake` must not do real work.
4. Remove pending page/link/text state **before** the generation check.
5. Stale results must never update UI caches.
6. Link extraction must stay decoupled from page-image pending state.
7. Navigation schedules the target first, then scrolls, then schedules neighbours.
8. Programmatic scroll events must not trigger the full viewport scheduling path.

9. Rapid navigation should behave as latest-request-wins wherever practical.
10. Never “fix” a problem by rendering every page, sleeping, clearing all cached pages on every scroll, or skipping generation checks.
1. Search and annotation overlays stay in canvas layers; never expand them into one widget per rectangle.
2. Linked-note writes append to an existing note unless the exact highlight link is already present.
3. Reference resolution must not issue a monolithic full-document worker command from the UI path; extract text per page so priority renders interleave.
4. Reference results are gated on `document_id`, never on `render_generation`.
5. Bump `references::RESOLVER_VERSION` whenever the resolver's output for the same input changes.

32.2 Before you open a pull request

```
cargo fmt --check           # must be clean
cargo check --workspace
cargo test --workspace      # 215 tests, all passing
cargo clippy --workspace    # baseline 10 warnings – do not add to it
```

And one more, easy to forget: **a change that alters behaviour described on a wiki page updates that page in the same pull request.** The wiki is the only documentation tree in the project — there is no `docs/` directory — and every claim on it is meant to be checkable against the file named beside it. Prefer naming the constant or function over paraphrasing its value, so a rename breaks a search rather than rotting quietly.

WHY IT IS LIKE THIS

Recording *why* alongside *what* is a documented expectation of this project, not a nicety. Most of the non-obvious code in the repository already carries its reasoning in module comments; this handbook exists because that reasoning is worth reading in order at least once.

Shipping a build

The checklist, the packaging layout, and the constraints you state rather than fix

33.1 Build and test

```
cargo fmt --check           # must be clean
cargo check --workspace
cargo test --workspace      # expect 215 passing: 56 core + 159 native
RENDER_PROPERTY_CASES=3000 cargo test -p md-editor-native properties
cargo clippy --workspace    # baseline is 10 warnings; confirm it has not grown
cargo build --release
```

For a reproducible release, pin and verify the PDFium download:

```
PDFIUM_RELEASE=chromium/6996 PDFIUM_SHA256=<hex digest> cargo build --release
```

33.2 The durability checks come first

Chapter 31, in full, before anything below. They are pass/fail.

33.3 Motion, caret and palette checks

- Toggle the sidebar, table of contents and backlinks. Each **slides** rather than snapping, and the window goes idle once the transition finishes.
- Trigger a toast by saving a file. It fades in and out rather than blinking.
- Press `Ctrl + P` and type immediately without clicking — the query field already has focus.
- Type a few letters of a note's name; the note appears among the results. Move the highlight with `↑/↓` and open it with `Enter`.
- Type a command's initials (`sv`) and confirm it ranks first. `Escape` closes.
- Type near the bottom of a long note. The page glides just far enough to keep the caret clear of the edge — it never re-centres on every key.
- Jump to a search match or a table-of-contents heading. It glides to the middle of the view; turning the wheel mid-glide stops it.
- Move the caret with the arrows. It glides between nearby places and cuts across long jumps; at rest it fades in and out, and after about twenty seconds it stays lit and the window goes idle.
- Click past the end of a wrapped row. The caret stays at the end of that row.
- Drag a selection across several paragraphs. It is one shape, with no gaps between lines and rounded corners only where it sticks out.
- Type 🐼, then press `Backspace` once. The whole emoji goes.

33.4 Smoke test — use a fresh temporary vault

VAULT AND MARKDOWN

- Open a vault; the sidebar indexes folders, markdown files, PDFs and images.
- Create, edit, save, reopen and delete a markdown file.
- Confirm `.git` and `node_modules` are not listed.
- Edit a file outside the app and confirm the watcher picks the change up.

SEARCH

- `Ctrl` + `F` in markdown: highlighted matches plus next/previous navigation.
- Toggle regex and match case; run a replace-all and confirm `Ctrl` + `Z` undoes it.
- Global search from the toolbar: both markdown and PDF results appear.

PDF

- Open a PDF: continuous rendering, fit-to-width (which stops at 200%; the zoom buttons still reach 400%), zoom, TOC, internal links, scrolling.
- Open a PDF **without** embedded bookmarks and confirm a recovered outline appears, marked as synthetic — or is correctly empty for a scanned document.
- Right-click a numbered cross-reference and confirm the preview shows the target.
- Select PDF text, copy it, paste it elsewhere.
- Search inside the PDF; toggle **Loose WS** and confirm a phrase that wraps across a line break matches.
- Create several highlights on different pages and confirm the colour cycles.
- Add a quick note to a highlight.
- Run the orphan report and confirm it states how many annotations were checkable.

LINKED NOTES AND SPLIT VIEW

- Link one highlight to a **new** markdown note through the picker.
- Link another highlight to the **same** note; a new section is appended, not overwritten.
- Re-link the first highlight and confirm nothing is duplicated.
- `Ctrl` + click the generated `pdf://` link and confirm the PDF opens at the target page with the highlight focused.
- Confirm opening a linked note in split view does **not** reset the PDF to page 1.
- In split view, interact with the PDF pane and press `Ctrl` + `F` — PDF search opens. Then interact with the markdown pane and press it again — markdown search opens.
- Check the Backlinks panel of the linked note lists the highlight.

OTHER

- Open image files and confirm the preview renders.
- Open the study tracker, start and stop a session, edit the configuration, restart the app, and confirm the state persisted.
- On Linux, run `--install` and `--uninstall` and confirm the launcher appears and disappears.

33.5 Packaging

Ship the platform library beside the executable — `pdfium.dll` (Windows), `libpdfium.so` (Linux), `libpdfium.dylib` (macOS) — in a `resources/` folder, which is the first path `bind_pdfium` checks. For portable distribution ship the executable, the PDFium library, the app icon and the licence together.

Then the first-run portability check:

- Run the packaged build from a fresh directory and confirm `md_editor_settings.sqlite` is created **beside the executable**, not in a system configuration directory.
- Move the whole folder somewhere else and confirm settings, highlights and study history come with it.

33.6 Known constraints

State these in release notes. They are design decisions, not defects — and saying so plainly is part of the same honesty that makes the app mark a recovered outline as synthetic.

- **PDF annotations are sidecar records.** The original files are never modified — which also means annotations do not travel with a PDF to another application.
- **PDF text selection, search and reference recognition require an embedded text layer.** Scanned or image-only PDFs need OCR outside the app.
- **PDFium operations run on one worker thread.** Navigation prioritizes the newest target page, but an operation already in flight cannot be interrupted.
- **One theme.** The app ships a single dark palette.
- **Window position is not restored**, only window size.

Reference

The lookup material: every binding, every named constant, the complete message vocabulary, a graded list of first tasks, and a short note on where to go next.

- A** Keyboard shortcuts
- B** Every named constant
- C** The message vocabulary
- D** Starter tasks, graded
- E** Where to go next

Keyboard shortcuts

Two layers, and knowing which is which explains the behaviour

Shortcuts are handled in two places. The **application layer** (`app.rs::subscription`) sees every key press regardless of focus and returns `None` for anything it does not act on, so ordinary typing does not spawn a redundant update cycle. The **editor layer** (`editor/renderer/events.rs`) only fires while the markdown editor widget is focused. On macOS, `Cmd` works everywhere `Ctrl` does — both layers test `modifiers.command() || modifiers.control()`.

Application layer

SHORTCUT	ACTION
<code>Ctrl + P</code>	Open the command palette
<code>Ctrl + O</code>	Open a folder as a vault
<code>Ctrl + N</code>	New file
<code>Ctrl + S</code>	Save the active note immediately, with a confirming toast
<code>Ctrl + F</code>	Search — routed to the active pane, see below
<code>Ctrl + B</code>	Toggle the sidebar
<code>Ctrl + T</code>	Toggle the table of contents
<code>Ctrl + C</code>	Copy the PDF text selection, when the PDF pane holds one
<code>Escape</code>	Close the topmost overlay, in priority order
<code>Enter</code>	Submit the open name modal
<code>↑ / ↓</code>	Scroll the PDF pane by 64 px
<code>PageUp / PageDown</code>	Scroll the PDF pane by 520 px

What `Ctrl + F` does

It follows the active pane rather than opening one fixed panel:

1. **Split view with a file open** — if the PDF pane was the last one you interacted with (by scrolling, clicking, or selecting text) it opens PDF search; otherwise in-note search.
2. **PDF only** — PDF search.
3. **A note open** — in-note search.
4. **Nothing open** — global vault search.

What `Escape` closes

In order, closing exactly one thing per press: the PDF text selection → the focused annotation → the PDF link preview → the open modal → the study tracker → in-file search → global search → the command palette → the table of contents.

Editor layer

SHORTCUT	ACTION
<code>Ctrl</code> + <code>Z</code>	Undo one word-sized run
<code>Ctrl</code> + <code>Y</code>	Redo
<code>Ctrl</code> + <code>A</code>	Select the whole document
<code>Ctrl</code> + <code>C</code> / <code>X</code> / <code>V</code>	Copy / cut / paste
<code>Ctrl</code> + <code>B</code>	Toggle bold around the selection
<code>Ctrl</code> + <code>I</code>	Toggle <i>italic</i>
<code>Ctrl</code> + <code>E</code>	Toggle inline code
<code>Ctrl</code> + <code>K</code>	Insert a link
<code>Tab</code>	Insert four spaces
<code>Enter</code>	New line, continuing a list marker where applicable
<code>Backspace</code> / <code>Delete</code>	Delete backward / forward
<code>←</code> / <code>→</code> / <code>↑</code> / <code>↓</code>	Move the cursor; hold <code>Shift</code> to extend the selection
<code>Home</code> / <code>End</code>	Start / end of line; <code>Shift</code> extends
<code>Ctrl</code> + click	Follow a link, including a <code>pdf://</code> deep link

TWO THINGS THAT SURPRISE PEOPLE

`Ctrl` + `B` is bound on **both** layers. With the editor focused it also formats the selection bold; the sidebar toggle fires regardless.

`Ctrl` + `Y` is the **only** redo binding. `Ctrl` + `Shift` + `Z` is not bound.

Palette-only commands

COMMAND	ALSO ON THE TOOLBAR?
Toggle Backlinks	no
Study Tracker	yes
Split View	yes
Focus Mode	no

Focus Mode is a one-shot action, not a toggle: it hides the sidebar, the backlinks panel and the table of contents, and closes the tracker, leaving just the document. **Split View** requires both a note and a PDF to be open; otherwise it raises a toast explaining why.

Mouse and trackpad

GESTURE	CONTEXT	ACTION
Wheel	Editor / PDF	Scroll vertically

GESTURE	CONTEXT	ACTION
Wheel over a wide block	Editor	Scroll that table or code block horizontally, independently of the page
Click and drag	Editor	Select text
Click and drag	PDF	Select PDF text
Right-click a reference	PDF	Preview the target equation, figure, table or section in place
<code>Ctrl</code> +click a link	Editor	Follow a wikilink, a file link, or a <code>pdf://</code> deep link
Drag the divider	Split view	Resize the panes

Every named constant

Read once; grep when you need it

Values below were read from the source tree. If one disagrees with the code, the code is right.

Timing

CONSTANT	VALUE	WHERE · WHAT IT DOES
<code>AUTOSAVE_DEBOUNCE</code>	400 ms	<code>editor_state.rs</code> — quiet time before a write
<code>AUTOSAVE_POLL</code>	100 ms	<code>editor_state.rs</code> — poll interval, finer than the debounce on purpose
<code>SEARCH_DEBOUNCE</code>	200 ms	<code>search_state.rs</code> — one scan per query, not one per keystroke
<code>UNDO_COALESCE_WINDOW</code>	300 ms	<code>editor/buffer.rs</code> — a pause longer than this starts a new undo run
<code>VAULT_WATCH_DEBOUNCE</code>	300 ms	<code>app.rs</code> — filesystem watcher coalescing
<code>SELF_WRITE_GRACE</code>	3 s	<code>app.rs</code> — suppresses watcher events caused by the app's own saves
<code>FAST</code>	120 ms	<code>motion.rs</code> — toasts, chips
<code>OVERLAY</code>	140 ms	<code>motion.rs</code> — palette, modals
<code>PANEL</code>	180 ms	<code>motion.rs</code> — side panels
<code>EASE</code>	EaseOutQuad	<code>motion.rs</code> — the only easing curve
<code>SCROLL_STIFFNESS</code>	32	<code>motion.rs</code> — the editor scrolling to reveal the caret, ~150 ms to 95%
<code>GLIDE_STIFFNESS</code>	80	<code>editor/renderer/glide.rs</code> — the caret gliding, ~60 ms to 95%
<code>MAX_GLIDE</code>	96 px	<code>glide.rs</code> — vertical caret moves further than this cut instead of gliding
<code>BLINK_DELAY</code>	500 ms	<code>glide.rs</code> — the caret stays lit this long after it moves
<code>BLINK_PERIOD</code>	1 060 ms	<code>glide.rs</code> — one blink
<code>BLINK_SHARPNESS</code>	2.5	<code>glide.rs</code> — how far the cosine is stretched before clamping
<code>BLINK_CYCLES</code>	18	<code>glide.rs</code> — blinks before the caret stays lit and the window can idle

Limits and thresholds

CONSTANT	VALUE	WHERE · WHAT IT DOES
<code>MAX_RETAINED_BUFFERS</code>	32	<code>editor_state.rs</code> — parked documents with their undo stacks
<code>MAX_WALK_DEPTH</code>	32	<code>core/vault.rs</code> — directory recursion cap
<code>MAX_RESULTS</code>	40	<code>views/command_palette.rs</code> — palette result cap

CONSTANT	VALUE	WHERE · WHAT IT DOES
<code>MAX_CONTENT_WIDTH</code>	880.0	<code>editor/renderer/metrics.rs</code> — readable measure for prose
<code>SPAN_WIDTH_CACHE_CAP</code>	8 192	<code>editor/renderer/measure.rs</code> — memoized span widths, dropped wholesale past the cap
<code>WORD_OFFSET_CACHE_CAP</code>	16 384	<code>editor/renderer/measure.rs</code> — memoized per-word character offsets from the shaper
<code>PDF_SUPERSAMPLE_MAX</code>	3.0	<code>app.rs</code> — ceiling on PDF render oversampling
<code>PDF_FIT_MAX_ZOOM</code>	2.0	<code>app.rs</code> — fit-to-width stops here; manual zoom still reaches 4.0
<code>PDF_PAGE_BITMAP_EVICT_MARGIN</code>	6	<code>pdf_pane.rs</code> — pages kept either side of the viewport

Editor rhythm and geometry

In `editor/renderer/metrics.rs` unless noted — the values layout, painting and hit testing must agree on.

CONSTANT	VALUE	WHAT IT DOES
<code>GRID</code>	4 px	every vertical measure is a multiple of it
<code>DEFAULT_FONT_SIZE</code>	17 px	body text; <code>row_height()</code> gives it a 28 px row
<code>PARAGRAPH_GAP</code>	20 px	a blank line
<code>HEADING_MARGINS</code>	48/16 ... 24/8	above/below: # 48/16, ## 40/12, ### 32/8, deeper 24/8
<code>BLOCK_MARGIN</code>	16 px	code blocks, tables, block math, images, rules, quote runs
<code>BLOCK_CAPTION_HEIGHT</code>	24 px	the caption band above a code block or table
<code>HORIZONTAL_SCROLLBAR_GUTTER</code>	16 px	space under a table's last row for its scrollbar
<code>CODE_FONT_SIZE</code> · <code>CODE_CARET_MARGIN</code>	15 · 24 px	code text · how far inside its viewport a code block keeps the caret
<code>TABLE_ROW_HEIGHT</code>	36 px	a body row plus padding
<code>TOP_PAD</code> · <code>BOTTOM_PAD</code>	24 · 80 px	above the first line · below the last
<code>UNITS_PER_PX</code>	256	<code>editor/layout_tree.rs</code> — fixed-point heights, so prefix sums are exact

PDF and references

CONSTANT	VALUE	WHERE · WHAT IT DOES
<code>RESOLVER_VERSION</code>	2	<code>core/references.rs</code> — bump whenever resolver output changes
<code>RIGHT_MARGIN_FRAC</code>	0.62	<code>core/references.rs</code> — past this fraction of page width, a number is a label
<code>CAPTION_PREVIEW_SHIFT</code>	110.0 pt	<code>core/references.rs</code> — nudge the preview off the caption onto the artwork
<code>PREVIEW_WINDOW_PT</code>	300.0 pt	<code>core/pdf.rs</code> — the right-click preview crop
<code>MAX_TOC_SCAN</code>	24	<code>core/pdf.rs</code> — pages examined when looking for a printed contents page

CONSTANT	VALUE	WHERE · WHAT IT DOES
<code>MAX_TOC_LEVELS</code> / <code>MAX_LEVELS</code>	4	outline depth caps
<code>PDF_PAGE_SPACING</code>	20.0	<code>views/pdf_viewer.rs</code> — gap between page slots

Design tokens

TYPE SCALE	SPACING (2 PX GRID)	RADII
<code>TEXT_XS</code> 8 · <code>TEXT_SM</code> 10 · <code>TEXT_BASE</code> 11	<code>SPACE_0</code> 0 · <code>SPACE_1</code> 2 · <code>SPACE_2</code>	<code>RADIUS_SM</code> 2
<code>TEXT_MD</code> 14 · <code>TEXT_LG</code> 16 · <code>TEXT_DISPLAY</code> 42	4	<code>RADIUS_MD</code> 6
	<code>SPACE_3</code> 6 · <code>SPACE_4</code> 12 · <code>SPACE_5</code>	<code>RADIUS_LG</code> 10
	16 · <code>SPACE_6</code> 24	

COLOUR	HEX	ROLE
<code>BG_PRIMARY</code>	<code>#0d0e10</code>	Window and panel ground
<code>BG_SECONDARY</code>	<code>#181a1d</code>	Raised surfaces
<code>BG_TERTIARY</code>	<code>#23262b</code>	Inputs, hovered rows
<code>BG_SURFACE</code>	<code>#334b47</code>	Selected / active surface
<code>BORDER</code> · <code>BORDER_SUBTLE</code>	<code>#45484e</code> · <code>#1d2024</code>	Standard borders · hairline dividers
<code>TEXT_PRIMARY</code>	<code>#e3e5ed</code>	Body and default UI text
<code>TEXT_SECONDARY</code> · <code>TEXT_MUTED</code>	<code>#a9abb2</code> · <code>#9d9ea3</code>	Secondary · captions, placeholders, disabled
<code>ACCENT</code> · <code>ACCENT_SECONDARY</code>	<code>#b1ccc6</code> · <code>#cde8e2</code>	Primary accent · lighter accent
<code>ACCENT_GLOW</code> · <code>ACCENT_DIM</code>	accent @ 50% · @ 20%	Focus rings, rules · subtle fills
<code>DANGER</code> · <code>SUCCESS</code> · <code>WARNING</code>	<code>#ee7d77</code> · <code>#d9f2d2</code> · <code>#bfdad4</code>	Destructive · confirmations · cautions

Panel widths and file names

<code>SIDEBAR_WIDTH</code> / <code>TOC_WIDTH</code> / <code>BACKLINKS_WIDTH</code>	260.0 / 250.0 / 220.0 — must match the width the panel lays out at
<code>DB_FILE_NAME</code>	<code>md_editor_settings.sqlite</code>
<code>PAIRS</code>	<code>[('(', ')'), ('[', ']'), ('{', '}')]</code> — quotes are handled separately, so contractions survive
<code>IMAGE_EXTENSIONS</code> <code>core/src/vault.rs</code>	<code>["jpeg", "jpg", "png", "svg", "webp", "avif"]</code>

TRAP — AND A WORKED EXAMPLE OF “THE CODE IS RIGHT”

The wiki's user guide and the project README both list the supported image types as `.png .jpg .jpeg .gif .bmp .webp`. The single source of truth, `core/src/vault.rs:8`, says `["jpeg", "jpg", "png", "svg", "webp", "avif"]` — so `.gif` and `.bmp` are *not* indexed, and `.svg` and `.avif` *are*.

This is exactly the drift the documentation convention is meant to catch: when a page and the code disagree, the code is right and the page is the bug. Check the constant before you trust a list.

The message vocabulary

Everything the application can be asked to do, in one enum

`Message` in `native/src/messages.rs` is the complete event vocabulary. Reading it top to bottom is the fastest orientation available in this codebase, because a program's message enum is the honest version of its feature list.

GROUP	VARIANTS
Vault	<code>OpenVaultDialog</code> , <code>VaultOpened</code> , <code>VaultIndexed</code> , <code>CreateFileDialog</code> , <code>CreateFolderDialog</code>
Sidebar	<code>SidebarToggle</code> , <code>SidebarFileClicked</code> , <code>SidebarFolderToggled</code>
Navigation	<code>GlobalSearchOpen</code> , <code>SearchQueryChanged</code> , <code>SearchDebounceElapsed</code> , <code>SearchRegexToggled</code> , <code>SearchMatchCaseToggled</code> , <code>SearchReplaceAll</code> , <code>CommandPaletteOpen</code> , <code>NameModalSubmit</code> , <code>DeleteFileDialog</code>
Editor	<code>EditorCommand</code> , <code>EditorCommandNoScroll</code> , <code>EditorSave</code> , <code>EditorCheckboxToggle</code> , <code>EditorScrolled</code> , <code>EditorCaretView</code> , <code>EditorScrollInterrupted</code> , <code>AutosaveElapsed</code> , <code>AnimationTick</code>
PDF	<code>PdfZoomChanged</code> , <code>PdfFitToWidth</code> , <code>PdfLoaded</code> , <code>PdfPageSizesLoaded</code> , <code>PdfRendered</code> , <code>PdfRenderFailed</code> , <code>PdfRenderSkipped</code> , <code>PdfScrolled</code> , <code>PdfLeftClicked</code> , <code>PdfRightClicked</code> , <code>PdfTocLoaded</code> , <code>PdfPageLinksLoaded</code> , <code>PdfReferencesLoaded</code> , <code>PdfSearchResult</code> , <code>PdfLinkPreviewResult</code>
PDF study	<code>PdfDocumentIdComputed</code> , <code>PdfPageTextLoaded</code> , <code>PdfSelectionChanged</code> , <code>PdfSelectionFinished</code> , <code>PdfCopySelection</code> , <code>PdfCreateHighlight</code> , <code>PdfQuickHighlight</code> , <code>PdfDeleteHighlight</code> , <code>PdfSearchLooseToggled</code> , <code>PdfOrphanReport</code> , <code>PdfAddQuickNote</code> , <code>PdfLinkNote</code> , <code>PdfOpenLinkedNote</code> , <code>PdfAnnotationFocused</code>
Tracker	<code>TrackerToggle</code> , <code>TrackerStart</code> , <code>TrackerStop</code> , <code>TrackerTabSelected</code> , <code>TrackerGateToggled</code> , <code>TrackerReadingToggled</code> , <code>TrackerProjectStatusChanged</code> , <code>TrackerConfigEdited</code> , <code>TrackerConfigSave</code> , <code>TrackerManualAdd</code> , <code>TrackerSessionDelete</code>
Toast & media	<code>ShowToast</code> , <code>ToastHide</code> , <code>MathRendered</code>
System	<code>KeyboardShortcut(Shortcut)</code> , <code>ToggleTOC</code> , <code>TocClicked</code> , <code>SplitViewToggle</code> , <code>SplitViewDragStart</code> , <code>SplitViewDragging</code> , <code>SplitViewDragEnd</code> , <code>WindowResized</code> , <code>WindowOpened</code> , <code>WindowRescaled</code> , <code>WindowCloseRequested</code> , <code>VaultFilesChanged</code>

Two supporting enums live beside it:

```
pub enum Shortcut {
    Save, OpenVault, NewFile, Search, CommandPalette,
    ToggleSidebar, ToggleBacklinks, FocusMode,
    TableOfContents, StudyTracker, SplitView, Escape,
}

pub enum TrackerTab { Dashboard, Log, Projects, Gates, Reading, Config }
```

READING THE ENUM FOR CLUES

Three details in that list are worth noticing, because each is a design decision made visible:

`EditorCommand` *and* `EditorCommandNoScroll` — some edits should not move the viewport, and that is expressed as a separate message rather than a boolean parameter.

`PdfRendered`, `PdfRenderFailed` *and* `PdfRenderSkipped` — three distinct outcomes, so a failure is never silently indistinguishable from a deliberate skip (Chapter 23).

`KeyboardShortcut(Shortcut)` rather than one variant per binding — the indirection that lets the keyboard subscription and the command palette trigger the same handler (Chapter 27).

Starter tasks, graded

Six ways in, roughly in order of difficulty

Each of these is a real, self-contained piece of work in this codebase. For each one: the files you will touch, the tests you should add, and the rules from Chapter 32 that apply. None of them is busywork — they are all things the program would genuinely be better for.

1 · Add a command to the palette WARM-UP

Something small and useful: *Copy File Path*, which puts the active document's vault-relative path on the clipboard and raises a toast.

Touch	<code>messages.rs</code> (a <code>Shortcut</code> variant), <code>views/command_palette.rs</code> (the registry), <code>app.rs</code> (the handler)
Test	extend the palette ranking tests so the new command is findable by its initials
Rules	one registry, one code path; the toast uses the existing <code>Message::ShowToast</code>
Why it teaches	you will touch the message enum, the router, a sub-state and a view in one change — the whole shape of Chapter 11 in twenty lines

2 · Decide what the image list should be JUDGEMENT, SMALL

Appendix B's trap: the wiki and README used to claim `.gif` and `.bmp` were supported and did not mention `.svg` or `.avif`. Those pages have been corrected to match `IMAGE_EXTENSIONS` — but the interesting half of the question is still open. The docs promised GIF and BMP for a reason; should the code deliver them, or was dropping them deliberate?

Touch	<code>core/src/vault.rs</code> — and, if you change it, <code>wiki/User-Guide-and-Feature-Manual.md</code> , <code>wiki/Architecture-and-Philosophy.md</code> and <code>README.md</code> in the same pull request
Test	if you extend the list, a test that a file with each extension is indexed as an image — and check the renderer actually draws it before you promise it
Why it teaches	the documentation rule is not decoration, and “make the docs true” is only half a fix when the docs described something a user reasonably wanted

3 · Rejoin hyphenated words in PDF search SMALL FEATURE

Loose whitespace mode (Chapter 22) matches a phrase across a line break, but a word split as `algo-` / `rithm` still fails. Extend the generated pattern so an optional hyphen-plus-whitespace can appear *inside* a word.

Touch	<code>app.rs::search_pdf</code>
Test	a core test over synthetic page text containing a hyphenated break

Watch out	do not break genuinely hyphenated words (<code>state-of-the-art</code>) and do not make the pattern quadratic; the constraint is precision, exactly as in Chapter 21
------------------	--

4 · Give tracker_activity a producer, or remove it JUDGEMENT

The table is created by the schema and written by nothing (Chapter 28). Either wire an activity feed to it — session started, session saved, gate ticked — or make the case for dropping it. Both are legitimate; the work is in arguing for one.

Touch	<code>core/src/tracker.rs</code> , <code>views/tracker.rs</code> , and — if you remove it — <code>apply_migrations()</code> rather than the base DDL
Why it teaches	Chapter 6's migration rule, and the discipline of deciding whether unused code is a plan or a leftover

5 · Add a block type to the editor THE FIVE-CALL-SITE CONTRACT

A callout / admonition block (`> [!note]`) that draws a tinted background and a label. This is the task that teaches the layout system properly.

Touch, together	<code>editor/highlight.rs</code> — parse it, assign a <code>block_id</code> ; then in <code>renderer/</code> : (1) <code>layout.rs::record_block_range</code> ; (2) <code>layout.rs::line_body_height</code> ; (3) <code>layout.rs::requested_margins</code> ; (4) <code>draw/blocks.rs::measure_blocks</code> for the card, clamped to the visible range; (5) <code>scroll.rs::scroll_extent</code> for hit testing and scrolling. Any new size goes in <code>metrics.rs</code>
Test	a behaviour test in <code>renderer/tests.rs</code> against the draw calls it produces; add callouts to the document generator in <code>editor/test_docs.rs</code> so every property suite exercises them; keep incremental-equals-fresh and source preservation passing in the highlighter
Rules	Chapter 32 rule 3, all of it. If the new block's height depends on anything new, it goes into <code>line_hash</code> or <code>resource_hash</code> and must change <code>layout_revision</code> . Run the render transcript harness (Chapter 30) before and after, and read the diff

6 · Make the FTS index incremental ENGINE, HARD

Today `file_search` is rebuilt wholesale when a vault opens (Chapter 10). On a very large vault that is the slowest part of opening. Update rows from the filesystem watcher instead, keeping the wholesale rebuild as a fallback.

Touch	<code>core/src/vault.rs</code> , <code>app.rs</code> 's <code>VaultFilesChanged</code> handler
Test	extend the FTS suite in <code>core/src/massive_tests.rs</code> : create, edit, rename, delete — and assert the index matches a full rebuild afterwards
Watch out	the current design's virtue is that the index <i>cannot</i> drift out of sync. Any incremental scheme has to be at least as trustworthy, or it needs a “rebuild index” command — which is a feature admitting a bug
Rules	Chapter 5's lock discipline: compute into a local, then lock, then assign

WHICHEVER YOU PICK

Run the four pre-PR commands (Chapter 32.2), compare the clippy count before and after, and if your change touches saving, navigation, animation or the PDF pane, walk the relevant items in Chapter 33 — particularly the durability checks, which no automated test fully replaces.

Where to go next

Five things to remember, and what to read when you forget the rest

If you remember only five things

1. **The engine is headless and the interface owns the screen.** When you cannot decide where code goes, ask whether it can be tested without a window. That answer is almost always the right one.
2. **Slow work happens before the lock, never inside it.** Compute into a local, then lock, then assign.
3. **Writes are atomic and navigation flushes first.** Never replace the temp-file-and-rename protocol with a truncating write, and never navigate away from a failed save.
4. **The draw pass is proportional to the viewport, not the document.** Everything in Chapters 15–17 exists to keep that true.
5. **Async results clean up before they are gated.** Remove from the pending set, *then* check the generation — and gate document-level artifacts on the document, not the generation.

Reading order, when you come back

WHEN YOU NEED TO...	READ
understand a failure in saving or navigation	Chapters 8, 31 · <code>core/src/vault.rs</code>
make the editor faster, or stop making it slower	Chapters 15–17 · <code>editor/layout_tree.rs</code> , <code>layout_cache.rs</code> , <code>renderer/layout.rs</code> , <code>renderer/draw/</code>
fix a caret, click or selection bug	Chapters 13, 17 and 30 · <code>renderer/flow.rs</code> , <code>caret.rs</code> , and a failing test in <code>renderer/tests.rs</code> first
debug a PDF page that will not load	Chapter 23 · the pending sets in <code>pdf_pane.rs</code>
debug a PDF jump that lands in the wrong place	Chapter 24 · the page-slot tests in <code>app.rs</code>
work on cross-reference recognition	Chapter 21 · <code>core/src/references.rs</code> , and <code>cargo run --example dump_refs</code>
add anything to the UI	Chapter 26 · <code>theme.rs</code> , <code>motion.rs</code>
review someone else's pull request	Chapters 31 and 32

The habit worth copying

The most transferable thing in this repository is not the Fenwick tree or the priority channel. It is that almost every non-obvious decision carries its reasoning next to it — in a module comment, in a test name, in a wiki page. `write_file_never_leaves_a_truncated_file` is a better

specification than three paragraphs of prose, and it fails loudly when someone stops believing it.

When you add something here, add the *why* with it. That is the practice this handbook is trying to hand on more than any particular fact inside it.

Colophon

Written from the seventeen pages of the project wiki and verified against the source tree.

Set in Bitstream Charter, with Liberation Sans for headings and DejaVu Sans Mono for code.

Every figure was drawn for this book as hand-authored SVG; none is a generated graph.

Built by `wiki/handbook/build.py` – edit the HTML in `wiki/handbook/` and re-run it to regenerate this PDF.

MD Editor is licensed under GPL-3.0; this handbook ships with it.